Drishti दृष्टि

Drishti: see, aggregate and analyse any data, live, in one platform

An entity terminal, a BI and analytics platform, and risk computed on demand, on the bank's own data

See what your data hasn't told you yet.

Your data is a treasury; Drishti is its sentinel.

Ashutosh Sinha

November 2026

Java 21 or later · console on any device

IN THIS DECK

1 Why one platform

2 The platform in one picture

3 See: every record, live

4 Drishti Elements

5 Aggregate: any scope

6 Analyse: BI and analytics

7 Compute: risk on demand

8 Pre-deal and one limit rule

9 Explain and trust

10 Modules, lenses and access

11 Case studies

12 Data in

13 Feeds and public data

14 Security

15 Evidence by the numbers

16 Operating and deploying

17 Where it is going

A Appendix: reference

TL;DR

Executive summary

What is Drishti?	Explainable and controlled risk aggregation over the engines you already have: every record live, any aggregate of it, analysed and explained, on the bank's own lakes.
What does it replace?	The screen coded per product, the data mart built per question, the BI tool on copies and the risk engine with its own store: one engine and one grammar over the data instead.
How does it work?	Sutras describe screens; scopes and hierarchies aggregate any set of trades from its members; models serve BI; VaR, ES and exposure are computed when asked, with their working shown.
What is there today?	4 modules, 10 lenses, 17 packs, 248 Sutras, 19 source plugins, 13 feeds, 11 libraries; VaR by any hierarchy, sharded; exposure and pre-deal; FRTB-SA, SIMM; playgrounds.
Where is it going?	Lean & streamlined: trust and controls with licensing, memory and speed; then counterparty engines and stress, collateral and margin, CVA, hardening.

WHERE DRISHTI SITS

The real-time risk and analytics layer next to the bank's trading systems

What it is

- the trader's risk workstation: VaR, ES, exposure, sensitivities, on any book, desk or ad-hoc set of trades, live
- pre-trade checks: the pre-deal check against credit limits and tenor buckets, audited
- a live, editable blotter: one engine for blotters, FORM results and lists, edits with four eyes
- every figure explained: where it came from, why it moved

What it is not

- not an order or execution system: it never routes, books or amends a trade at a venue
- not the system of record: the trading and pricing systems stay the source; Drishti reads their files, streams and databases
- not a second risk silo: one engine, one grammar, the same answer on every screen

Trading and pricing systems
own the trades and the prices

›

Feeds
files, Kafka, databases: bound by configuration

›

Drishti kernel
lakes, engines, one grammar

›

Workstation
risk, pre-trade, explain, live

Next to the trading systems, not inside them: read-only on their data, so Drishti can be added, upgraded or removed without touching an order path.

Why one platform

01

Today a new question means a new screen, a new mart and a new report

1 A new screen
a developer codes a page for the product or the desk; it is right on the day it ships, then rots

→

2 A new data mart
the rows are copied out of the systems of record, reshaped and loaded overnight for that question

→

3 A new report
a BI tool reads the copy; its totals, filters and masks are written again, in its own language

→

4 A new risk store
the risk engine keeps its own finished numbers; nobody can re-derive one of them later

→

5 The numbers disagree
the screen, the report and the risk figure each read a different copy, at a different time

What it costs

Time
each question waits for a screen, a mart and a report to be built in turn, by three teams

Trust
a figure in the morning meeting has three versions; reconciling them is a job of its own

Security
every copy needs its own entitlements and masks, and each is a new place for data to leak

Change
a new product or desk means new code in every layer; a hierarchy change means a rebuild

Key insight One platform reads the bank's own data once, and every screen, aggregate and analysis is computed from it.

THE PROBLEM

Every entity kind gets a bespoke screen — and the screen rots

A screen per product, in the documents' own words

- **Hundreds of entity kinds**
  “Front-office, risk and operations staff look at hundreds of entity kinds.”
- **Written once, rotting for ever**
  “Each one traditionally gets a bespoke screen, written once and rotting forever.”
- **Data outruns screens**
  “The data changes shape faster than screens can be rebuilt.”

And what people do in the meantime

- **Tab between systems**
  “A trade lives in one system, its netting set in another, its curve in a third … the desk tabs between them.”
- **Wait for a screen**
  “Every new product type waits for a screen.”
- **Read raw JSON**
  “A new feed or product has no screen at all, so on day one people read raw JSON or wait.”

Key insight **Nobody here is short of data — they are short of a screen for it that is dense, live and linked.**

WHAT DRISHTI REPLACES

Four things a bank builds again and again, and what takes their place

What a bank builds today	What Drishti does instead	Where it shows
The bespoke screen per product	A Sutra: a short YAML file per product family; inference lays out what it leaves out. One pipeline draws every entity.	219 Sutras in 11 packs
The data mart per question	Scopes, hierarchies and groupings computed from the lake's own rows when asked; Export data downloads any dataset.	Delta or Iceberg, read in place
The BI tool on copies	Rūpaka BI on the same data: datasets rooted in packs, one model per report, row rules and masks before any sum.	the same entitlements as a view
The risk engine with its own store	VaR, ES, exposure and the attributes computed for any scope from trade-level inputs; official records kept, never edited.	any scope, bank to one trade

Key insight Not four products glued together: one engine, one grammar, one set of rules over the bank's own lakes.

THE SHAPE OF THE FIX

From a coded screen per product — to one grammar and inference

Your systems

Trades, curves, tickets, variants

→

A screen per kind

Coded by a developer, per product

→

A backlog

New products wait for a screen

→

Drift

The data changes; the screen does not

Your systems

Read where they are, through plugins

→

DRISHTI

Sutra ⊕ inference

A short YAML Sutra describes the view; scored, explainable rules read the data's shape and fill the rest

→

A live view

Dense, linked, explained — day one

Key insight A brand-new product with no Sutra still renders a usable view on day one; a Sutra makes it beautiful.

Six goals the design is held to

1

One grammar

Every view is a Sutra in Rachana, or inferred; no screen code.

2

Any source

Sources plug in through a small, Spring-free interface.

3

Linked

Every id on a screen is navigable, across systems.

4

Live

Changes reach the open view; p99 is measured and shown in the top bar.

5

Provenance

Every view shows how it was built and where its data came from.

6

Keyboard first

<MNEMONIC> <ID> <GO>, F-keys for panels, F9 for raw JSON.

Key insight Six goals, each one testable — the p99 is not a claim on a slide, it is in the top bar of every view.

Each family does part of the job, and is good at it

Family	Examples	Good at	What it leaves you
Market data terminals	Bloomberg Terminal, LSEG Workspace, FactSet	Dense, keyboard-driven, live screens	The vendor's screens, for the vendor's data
Trading and risk platforms	Murex, Calypso, in-house systems	Deep screens for every product they book	A coded screen per product type, for that platform's data
Low-code internal tools	Retool, Appsmith, Power Apps	Internal screens assembled quickly	A person still assembles each screen
BI and dashboards	Tableau, Power BI, Grafana	Aggregates and charts	Weaker at one entity, live, with its links

Key insight **The screens that are dense and live are coded per product; the screens that are quick to make are not terminals.**

BENEFITS

What a grammar of screens buys

Day one, every product	Inference shows a new feed at once; nobody reads raw JSON.
Screens as reviewed data	A Sutra is versioned, tested, diffed and approved like code — without being code.
One place to look	A trade, its curve and its netting set from three systems on one linked screen.
Trust in the number	Provenance and freshness on every view; masks applied for every reader.
Anywhere it is needed	The console on any device, the API, the CLI, and the same view inside other apps.

Key insight **The cost of a new screen falls from a project to a paragraph — and to nothing at all on day one.**

The platform in one picture

02

THE PLATFORM IN ONE PICTURE

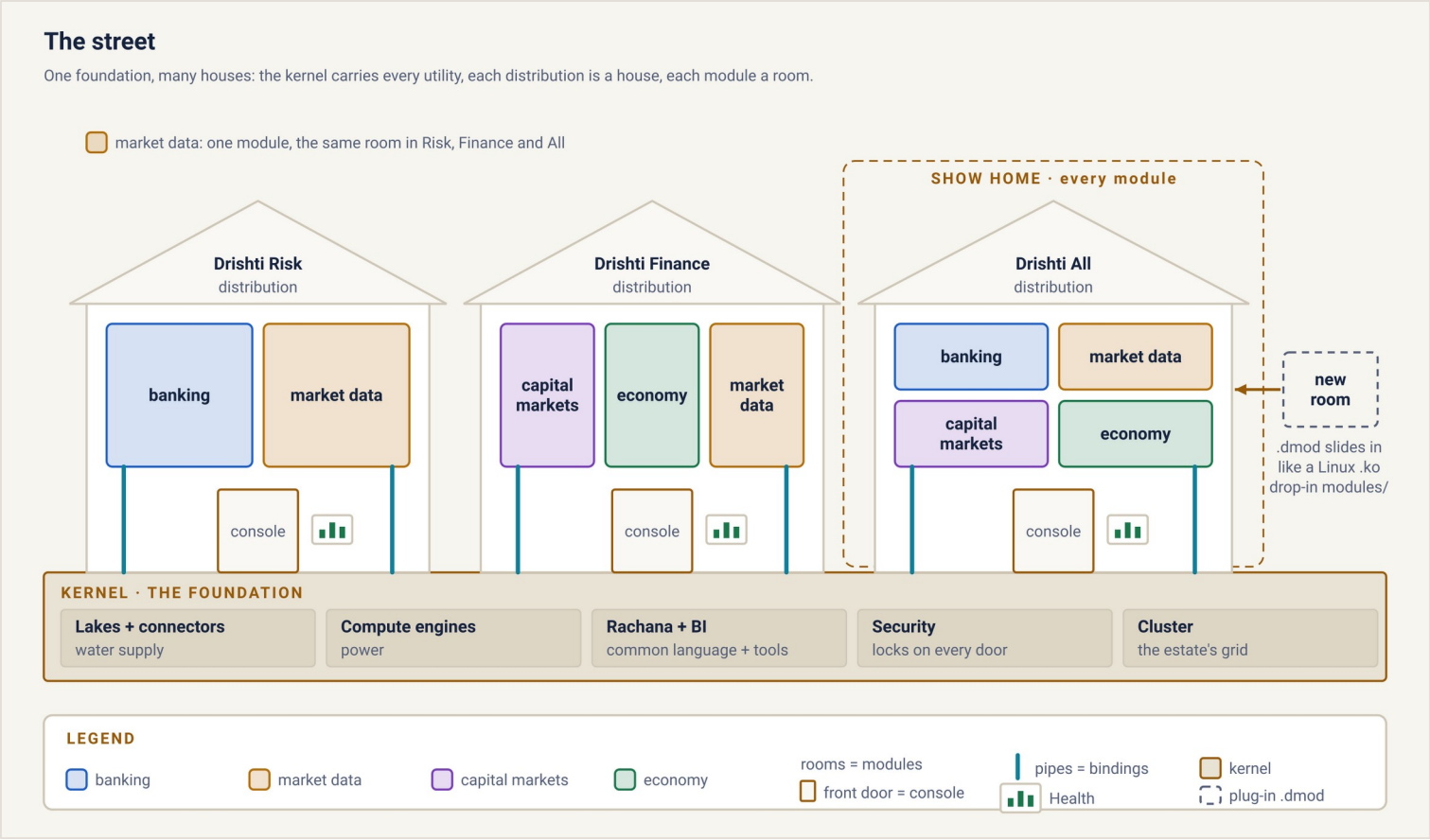
Four layers: data in, one engine, many experiences, and trust around them all

Experiences what people use	**Terminal** any record, live, linked	**BI** tables, models, reports, boards	**Playgrounds** every figure as formulas	**Lenses** Risk, Trading, Market, World	**Phone** the same views, tiered	**Elements** views inside other apps
Engine one grammar	**Rachana** Sutras and inference	**Aggregation** scopes, hierarchies, totals	**Computed measures** VaR, ES, exposure, groupings	**Live** one subscription per entity	**Drishti*net** several servers as one	
Data the bank's own	**Lakes** Delta and Iceberg, local or S3	**JSON Lines** the one data standard	**Contracts** one per command	**Connectors** 19 source plugins	**Public packs** SEC, GLEIF, World Bank	

Trust, around every layer sign-in and roles · masks before arithmetic · audit of who saw what · data contracts · official records that never change · the evidence: tests, drills, benchmarks

Key insight **Every experience asks the same engine, which reads the same data under the same rules.**

One foundation, many houses: kernel, modules, distributions



The kernel

lakes and connectors, compute engines, Rachana and BI, security, the cluster. It knows no industry.

Modules

banking, market data, capital markets, economy: .dmod packages with packs, compute, command words, lenses and pages.

Distributions

Drishti Risk for traders and risk, Drishti Finance for research and economics, Drishti All with every room: one kernel build, one console build.

One rule

every function has one room, every room one door, every setting one switch.

Like Linux: the kernel, drivers (.dmod modules built apart and checked against the kernel), and distributions with their own name, theme and set of modules.

See a record, aggregate any scope, analyse a model: the same request path

See

```
TRD MX-20000001
```

- a kind and an id
- the Sutra matched, the layout from cache
- links fetched in parallel
- a ViewModel in JSON, live by patches

Aggregate

```
VAR BANK - DESK:CREDIT
```

- a scope written as text
- resolved to ordinals on the date
- parts summed by position runs
- VaR, ES and contributions that add up

Analyse

```
BI · a report on a model
```

- one model per report
- row rules and masks injected first
- one query plan, live or as of a date
- the browser only draws

Shared by all three, so they always agree

- the same connectors and business dates; the same entitlements, masks and audit; the same data versions and caches keyed by them
- the same answer on every member of a cluster (Drishti*net), and the same answer over the REST API, the CLI and in the console

Key insight Seeing, aggregating and analysing are three questions to one engine, not three systems.

Eight primitives carry the whole model

Primitive	What it is	Where you meet it
Pack	A folder for one domain: kinds, mnemonics, links, connectors, Sutras, samples, help text; packs inherit	config/packs/ — 11 shipped, e.g. trading, market-risk, funds
Sutra	A short, versioned YAML description of one view, in the Rachana grammar	219 Sutras in the shipped packs; fragments from the UI kit
Match	kind + where (a closed expression) + priority; the highest match wins	match: { kind: trade, where: …, priority: 10 }
Strip	Up to eight headline figures across the top of a view	Source, side, notional, status …
Panel	One of 22 kinds on a 12-column grid, each with an F-key	table, ladder, pivot, surface, map …
Inference	Explainable rules that read the data's shape and lay out what no Sutra covers	infer: true; 'completed by inference'
ViewModel	Plain JSON the server returns and the browser draws	GET /api/v1/views/{kind}/{id}
Frame	A patch of what changed, sent over one live channel per browser	Server-sent events

ARCHITECTURE

From your systems to a view, in one direction

Stores and lakes

Delta, JDBC, Aerospike, Cassandra, MongoDB, Redis, Iceberg, DuckDB

Streams and queues

Kafka, ActiveMQ, RabbitMQ

Files and services

JSON Lines, CSV (deprecated), S3, REST

Public market data

SEC EDGAR, GLEIF, N-PORT, NY Fed, ECB, FRED

DRISHTI SERVER · JAVA 21 · ONE OF A CLUSTER

Fetch, match, infer, link, bind

Virtual threads fetch in parallel; the engine matches a Sutra, merges inference, links ids and binds values; BI answers on DuckDB; VaR and exposure engines compute any scope — 19 source plugins

SAME SERVER

Identity, packs, audit, Drishti*net

Users, roles and masks; audit; a signed pack registry; commands switched off for everyone; peers by gossip on their own port, HTTP/2 between servers

Console

Web terminal, any device

REST API

/api/v1, OpenAPI at /api/docs

CLI

tools/drishti.py, 22 groups

Other apps

<drishti-view>, <drishti-report>

SEVEN STAGES

One command, seven stages, one view

1

Command

TRD MX-20000001 <GO>, or plain words

2

Fetch

Sources in parallel on virtual threads

3

Classify

Which kind is this record?

4

Fingerprint

The data's shape, hashed

5

Layout

Sutra ⊕ inference, cached by shape

6

Bind

Values into panels, in parallel

7

Link

Every id navigable, time-boxed

0.6 ms
Server work for a warm view

3.0 ms
Warm view over HTTP, p99 (gate: 50 ms)

7.2 ms
Structured search over 750 trades

Key insight **The expensive part — deciding the layout — happens once per shape, not once per request.**

A swap's whole screen, in one short file

murex/irs-fixfloat.v1.sutra.yaml (trimmed)

```
rachana: 1
sutra: irs-fixfloat
doc: { summary: "Exchanges fixed for floating …" }
match: { kind: trade, priority: 10, where:
  "$.SOURCE_SYSTEM == 'MUREX' &&
   $.PRODUCT_TYPE == 'IRS_FIXFLOAT'" }
title: { pill: "Rates · Interest rate swap",
         id: $.TRADE_ID }
strip:
  - { label: Source, bind: $.SOURCE_SYSTEM }
  - { label: Maturity, bind: $.MATURITY_DATE }
panels:
  - { use: trading/deal-ticket@1 }
  - { use: trading/legs@1, with: { rate-fmt: pct4 } }
  - { use: data-provenance@1 }
  - { use: risk-links@1 }
```

- **When it applies**
  A MUREX trade whose PRODUCT_TYPE is IRS_FIXFLOAT; the highest priority match wins.
- **What it shows**
  A linked header, eight headline figures and fragments: the pack's deal ticket and legs, the UI kit's provenance and risk links; 132 pack files reuse the deal ticket.
- **Documented in itself**
  Every Sutra and fragment carries a doc: block (summary, params, examples) that the designer, the help and AI read.
- **Shipped through review**
  Rachana-EL has no side effects; a bad edit keeps the last good version live; in production a second person approves.

LIVE, END TO END

One subscription per entity, one channel per browser

Source
subscribe()

→

TopicHub
One topic per entity; latest wins

→

ViewStream
One per open view; cached layout

→

PatchDiffer
Old vs new ViewModel

→

Browser
Patches over one SSE channel

2.4 ms
Tick to frame, p50

11.2 ms
Tick to frame, p99 (gate: 40 ms)

50 ms
A topic sends at once if it has not sent within this window

1
Live channel per browser, however many views

Key insight A view does not poll and does not redraw — only the figures that changed travel, and only to the views that show them.

Several servers, one Drishti

Console
Given one server; learns the rest; fails over

→

Any node
Signed calls; your roles checked

→

Capability table
Who renders each kind, who holds its data

→

Forward or fetch
Newest pack served; masked on both nodes

→

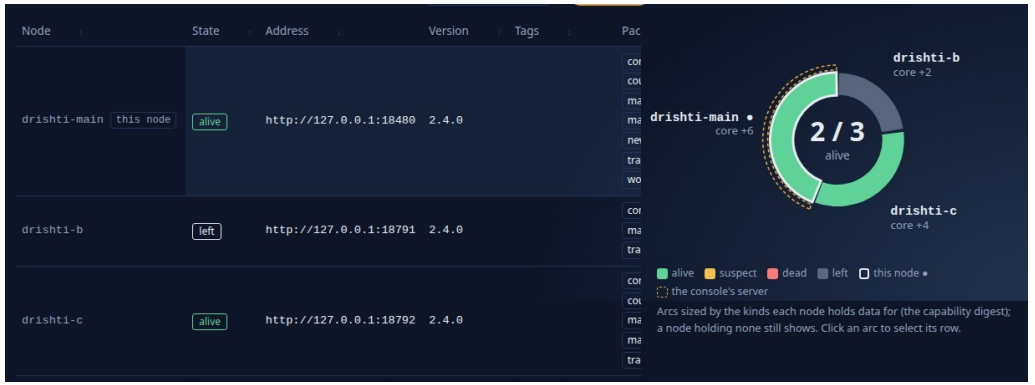
Scatter-gather
A search asks every shard, once

52 ms
Three servers converge after the third starts

0 wrong
of 146,498 answers checked in a 2-hour soak with kills

117 ms
p99 between servers on HTTP/2 (370 ms on HTTP/1.1)

13,542
Signed calls through a key rotation, none refused



Key insight Each node keeps its own packs, connectors and data — the user cannot tell which one answered; an administrator can.

See: every record, live

03

JAVA 21 OR LATER · ANY DEVICE

Drishti — overview

11
Packs: core, trading, market data and risk, counterparty risk, funds, US equities, GLEIF, economy, news

219
Sutras shipped in those packs, built from the UI kit

106
Commands, each with a machine-readable data contract

19
Source plugins behind one interface

- **A terminal on any device**

Command line, history ranges, FUNC, DATAMAP, Knowledge and WHAT over production-shaped trades from ENDUR, CALYPSO and MUREX and public data from SEC, GLEIF and N-PORT; phones.

- **Risk computed, never stored**

VaR and exposure for any scope, baskets and pre-deal what-ifs, official end-of-day records, in BI; VaR from scenario PVs in any reporting currency.

- **Build and analyse without code**

The four-mode Sutra designer and the UI kit; BI on one table layer, edited with its YAML in view; the Notebook; Python on the server; forms that write back under four eyes.

- **Run it, on one server or several**

Pack bundles with rollback, data-load signals, JSON Lines in and out, Delta and Iceberg side by side, a CLI for every step, Drishti*net clusters.

What is a live entity terminal?

- **One record, one dense screen**
  A swap, an FX trade, a netting set, a ticket or a gene variant, with its headline figures in a strip and its detail in up to twenty-two kinds of panel.

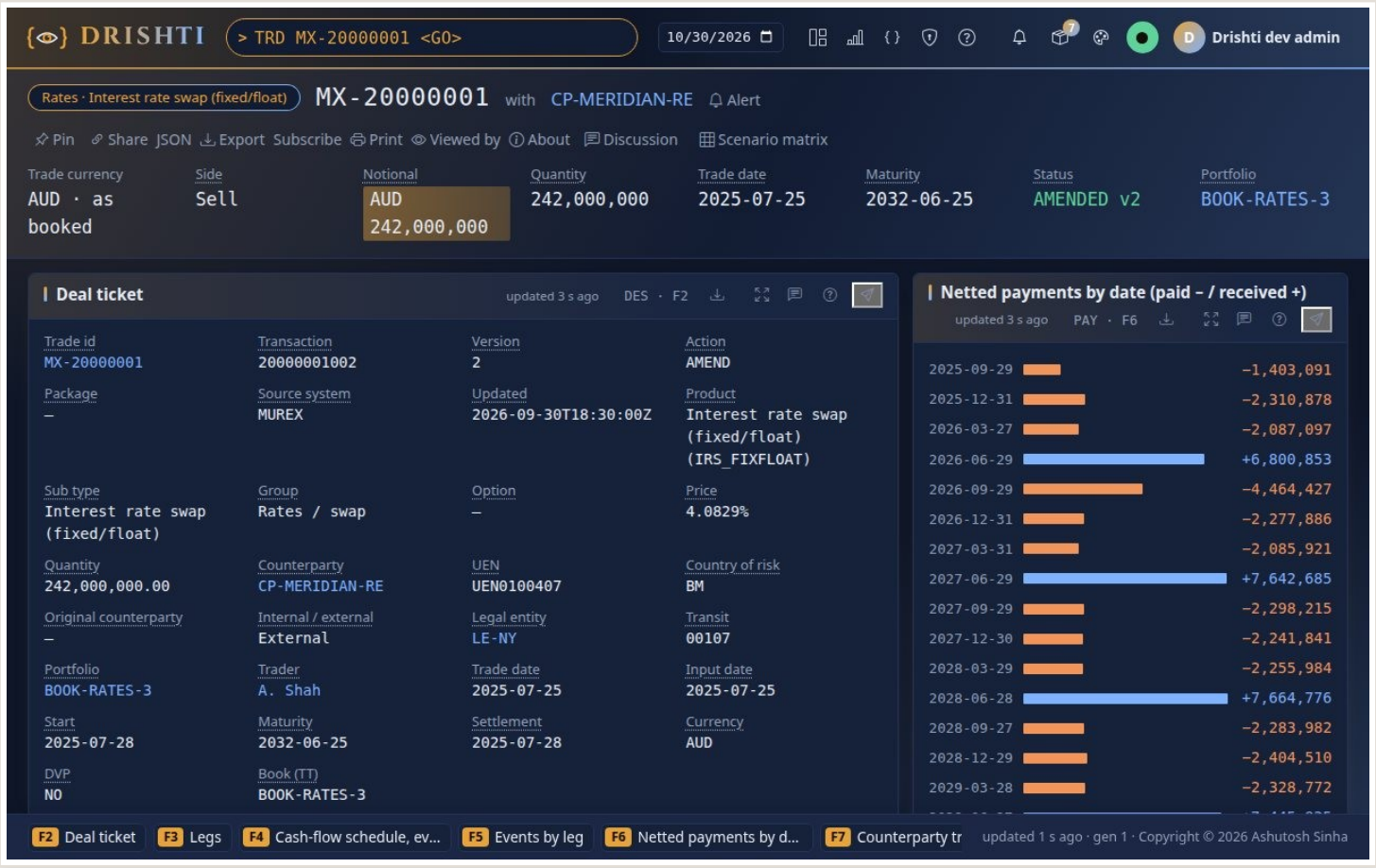
- **Live, not refreshed**
  When the source changes, the open screen changes: only the changed figures are sent, measured at p99 11 ms from tick to frame.

- **Linked**
  Every id on the screen is a link: a trade's counterparty, its netting set, its curve — one keystroke away, across systems.

- **Explained**
  Every view says where its data came from, how fresh it is, and why it is laid out the way it is.

Key insight **The terminal is the standard for dense, live screens — Drishti makes one for any record you have, not only the vendor's.**

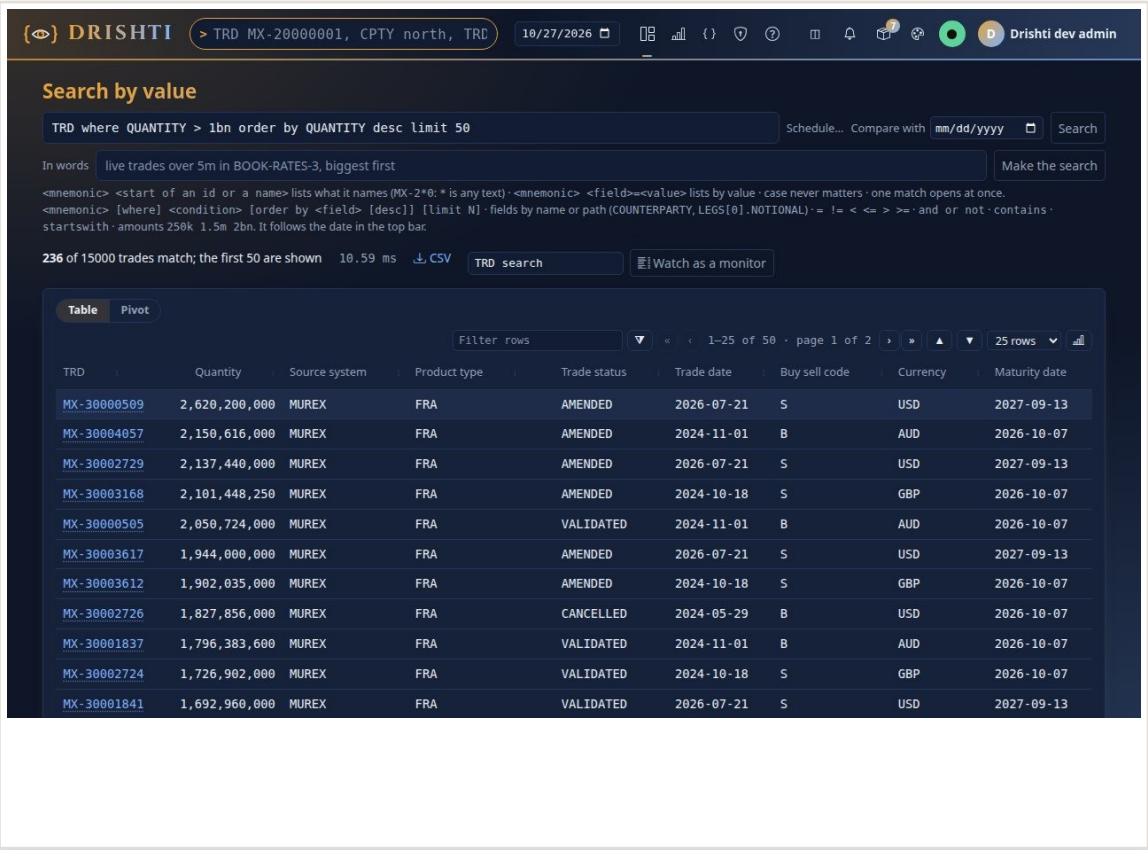
One trade, linked and explained



A MUREX interest rate swap, UPPER_SNAKE as production data is; captured on the demo lakes (tutorial 1).

- **Strip**
  Source, side, notional, quantity, dates, status and portfolio.
- **Panels by key**
  F2 deal ticket, F3 legs, F4 cash flows, F6 netted payments; Alt+L to rearrange.
- **Linked**
  Counterparty, portfolio, legal entity and trader open in one click.
- **Any day**
  The business date in the top bar; Compare shows what changed between two days.

Ask in a grammar or in plain words; pivot what comes back




```
TRD where QUANTITY > 1bn order by QUANTITY desc
amended trades over 5m in BOOK-RATES-3
FORM trading <GO>
```

134 ms

Search all 1M trades a day, 2,017 matches

29 ms

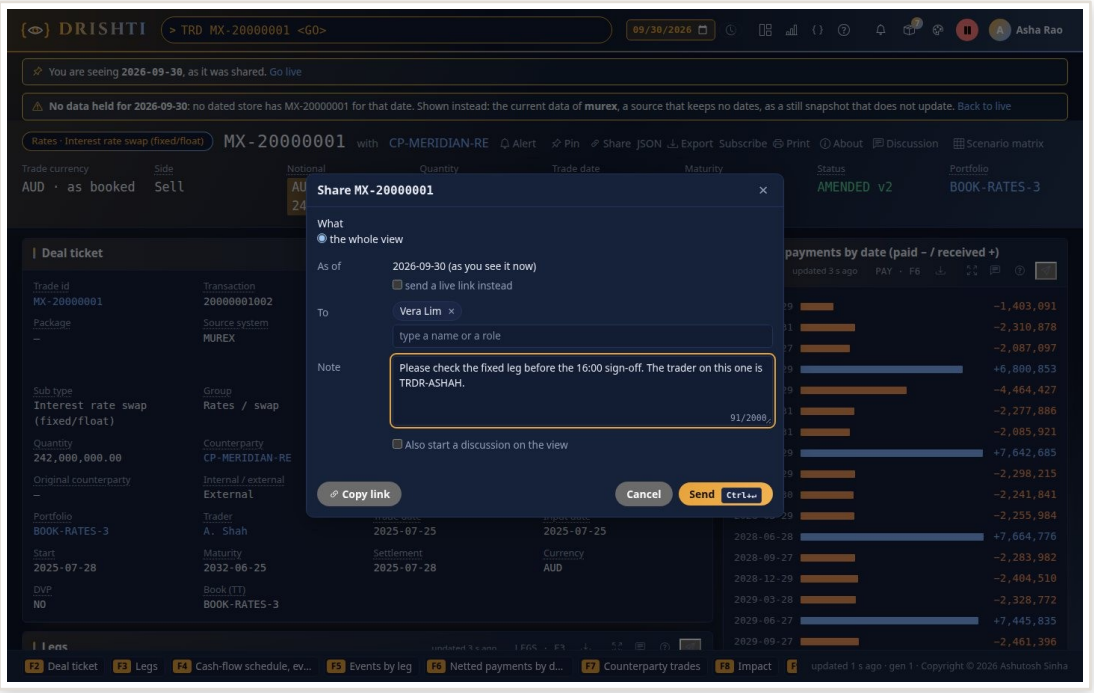
Type-ahead at 1M trades a day

Measured on a 24-core workstation, 1,000,000 trades a day for three days (5.2 GB).

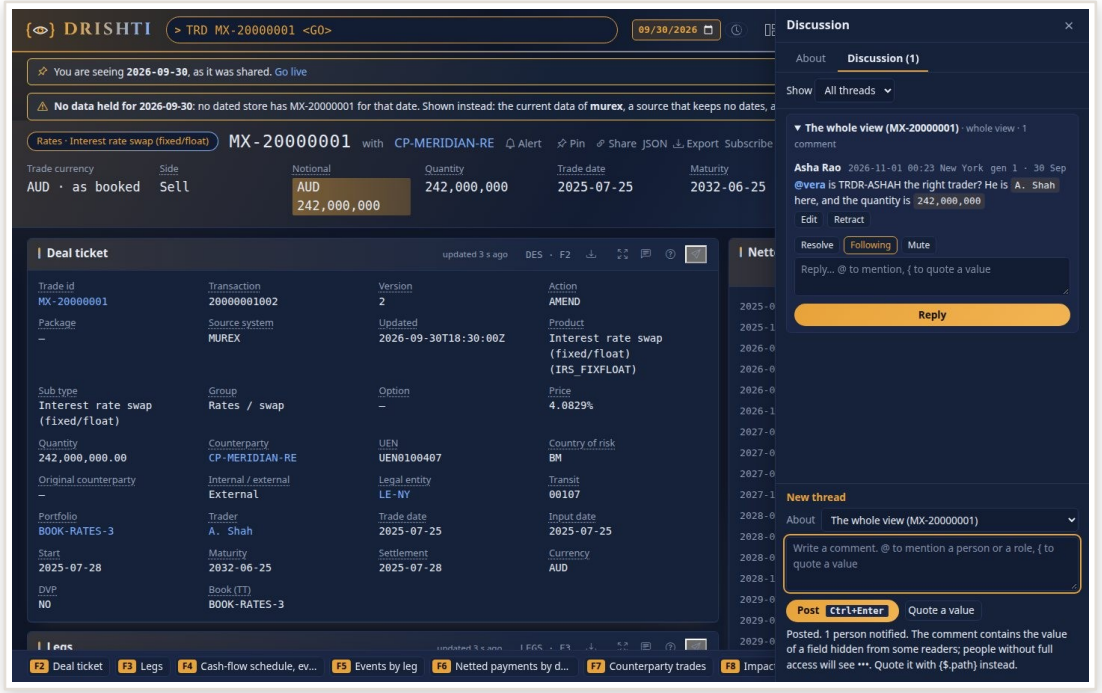
Key insight **Search is a question in the terminal's own grammar — and its answer is a view you can pivot, compare and share.**

COLLABORATION

Share what you see, discuss it, and prove what was said



Share with a note, to people or roles.



A discussion thread with @mentions and quoted fields.

Key insight **Every comment is hash-chained and sealable; retract and hide never delete — evidence, not just conversation.**

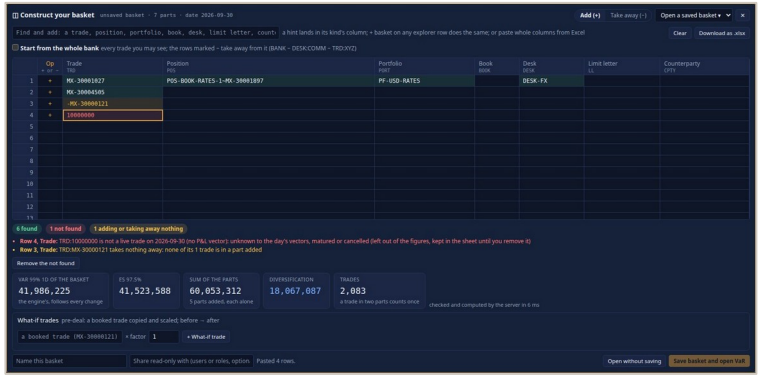
Explain the data, then let anyone build the portfolio they mean

Knowledge pages

- Market risk
- Counterparty credit risk
- Trades and products
- Entities and ownership
- Market data
- Data and contracts
- Economies
- Companies, funds and ETFs

Each with the plain idea, how the records connect, a worked example on the demo data and the questions to ask.

`WHAT <term>` 162 terms; 'What does this mean?' beside the VaR and exposure explorers, USEQ, FUND, HELD and TRACK.



Construct your basket

Pick portfolios, desks, books, legal entities, counterparties, positions, trades or a rule as you type; see VaR, ES and diversification live; save it. Alt+B.

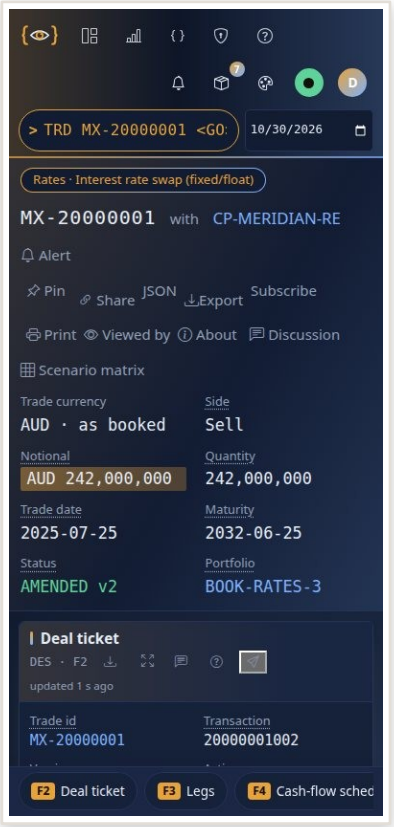
The pre-deal what-if

A hypothetical trade — a booked one copied and scaled, −1 its unwind — shows VaR before and after in the basket; the exposure explorer has it on the counterparty and the netting set. Exact methods run on the compute workers; trade pages link to the trade's P&L vector, VaR and exposure, or say why not.

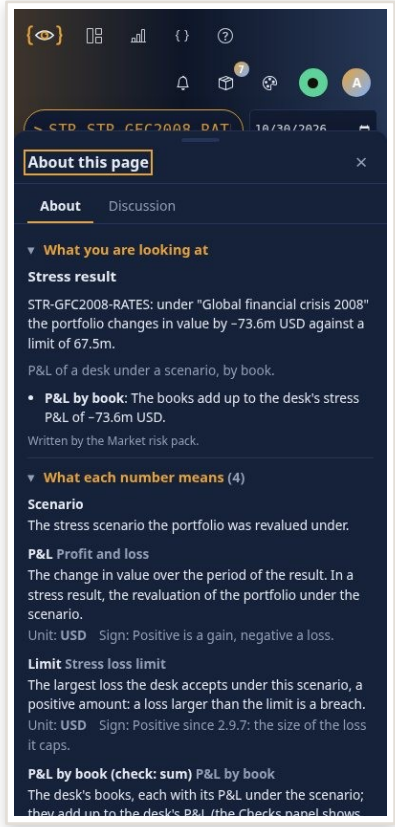
Key insight **An id that matches nothing, or a trade that is not live, is said in words — never shown as a VaR of 0.**

ON A PHONE

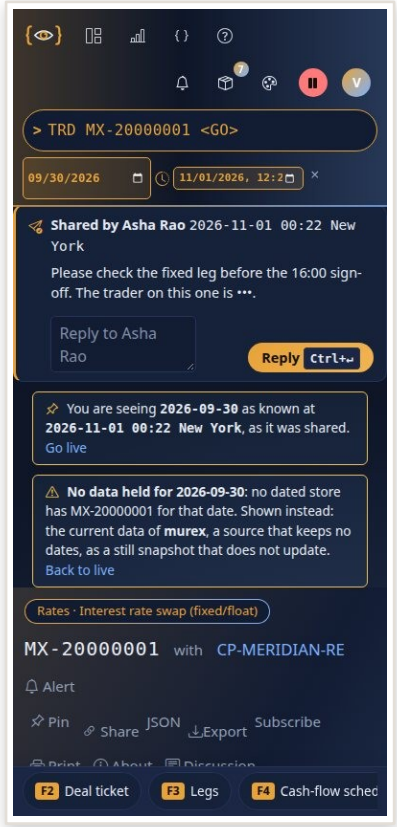
The same terminal in a pocket



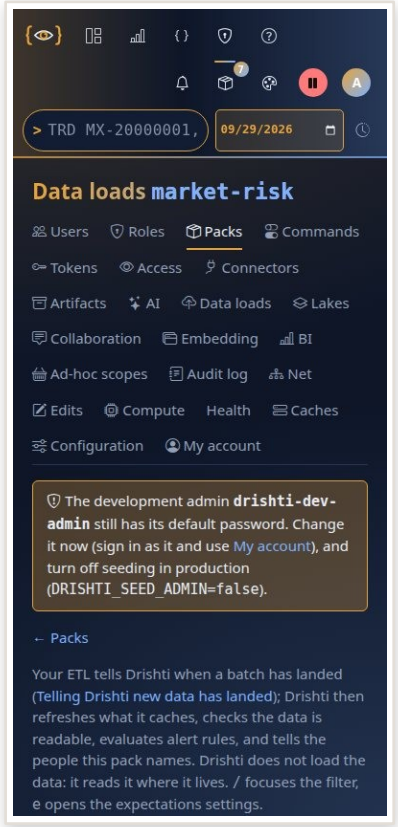
A live trade



About this page



A shared view



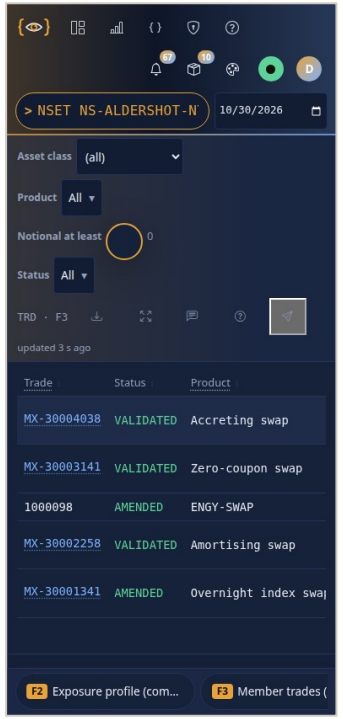
Data loads

- **44 px**
  Every control, on touch screens only
- **390 px**
  Every page tested at phone width
- **3 engines**
  Embedded views in Chromium, Firefox, WebKit

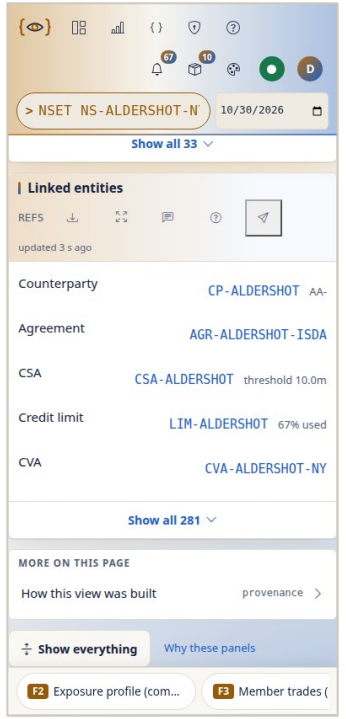
Must, glance, more and desk: a phone loads what matters first

must	**shown in full, loaded with the page, live** the header strip, key figures, status and breaches
glance	**a compact form: a few rows, a sparkline** a long table as its first five rows, with Show all
more	**listed under More on this page; loaded on tap** contributions, cash-flow ladders, provenance
desk	**not on a phone; Open on desktop says it exists** designers, wide pivots, editing forms

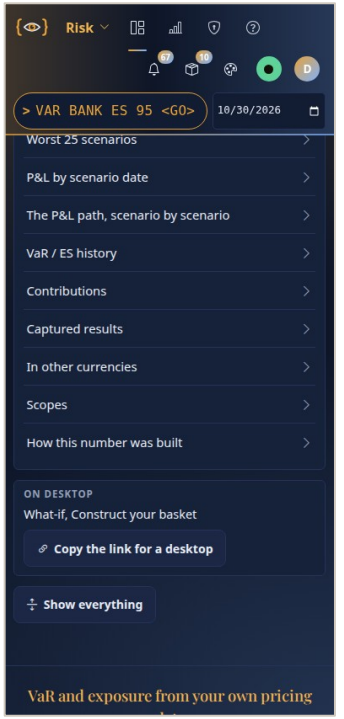
Set by the panel kind, the fragment, the Sutra, then the site, pack or edition (which can cap a heavy panel); the server binds only must and glance panels for a phone.



A netting set: trades at a glance



More on this page



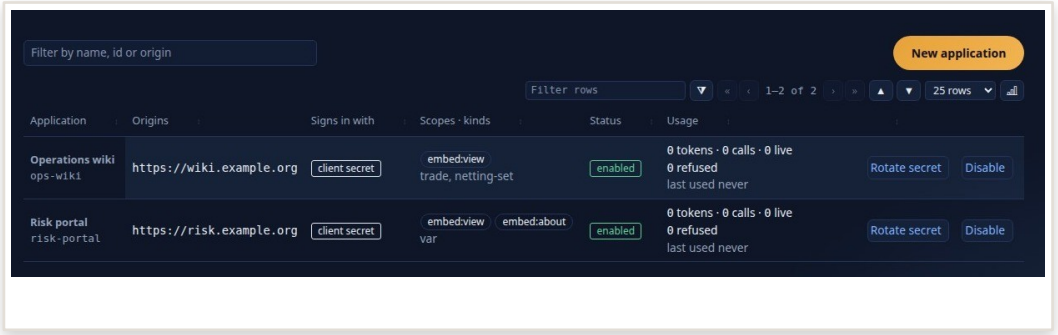
The VaR page on a phone

Key insight **Configured, never coded: masks and roles are unchanged, and desktop is untouched.**

The same live view, inside other web applications

```
<script type="module" src=
  "/embed/v1/elements/1.4.3/drishti-elements.js">
</script>

<drishti-view server="https://drishti.example"
  kind="trade" entity="MX-20000001"></drishti-view>
```



Admin → Embedding: host apps, origins, scopes and usage.

HTML

Over the wire: the console's own templates, sanitised, in a Shadow DOM

1 stream

Shared by every element on the host page

@container

Responsive to its own box: a sidebar gets the narrow layout

Masked

Always, whatever the user's roles; every call audited

Key insight Other teams get the view, not a copy of it — still live, still masked, still audited.

The top bar: five menus, one sitemap



Menu	Holds
Views	Terminal, Search and pick lists, Forms, Functions, Data map; Workspaces, Monitors, Alerts, Approvals (the one queue, with a count)
Domains	a column per menu a module brings: Blotter, Positions, Risk (a column per running engine), Counterparties, Markets…
Create	BI, report designer, datasets; Build home, New screen, New pack, Tables and models; Learn
Admin	People and access; Data (packs, connectors, loads, lakes, edits); Platform (modules, engines, cluster, health)
Help	Help centre, Knowledge, Install and run, What's new, About

One sitemap

- every menu, Admin tab and the BI rail drawn from one file, sitemap.yaml; a test holds every page to an entry
- what you see follows your role and your lens

Every entry is a command

- each shows its command and runs it as if typed: a menu never holds what the command line does not
- the logo always leads to the landing page, with a way back

One live engine behind every list, FORM result and blotter

Snapshot
paged from the lake: Delta, Iceberg or files, through the bindings

›

Live deltas
the kind's live: binding (Kafka, RabbitMQ, ActiveMQ), merged by key and version

›

One result set
per audience; a newer version replaces the row, an older one is dropped

›

One grid
Elements grid in live mode: ticks, Pause, Raw JSON, Pivot ›

What runs on it	Is
A blotter	a YAML declaration: columns, grouping, links and, optionally, edits
A FORM or the search page	the matches as a set: a blotter without edits
POS, TRD, NEWS	the kind's own list; a default declaration is made from its contract
A monitor, drill-through, a preview	the same set, opened from another page
A SQL or BI table	the static case: the same grid without the live channel

Made to scale

- a lake-held kind is never read whole into memory: the set keeps an index of keys and the sort column
- pages of at most 5,000 rows; a viewer too far behind gets one resync, never an unbounded queue
- first page at a million records in about half a second; later pages a millisecond or two
- rows tick while the business date is the live one; the chip says LIVE or STATIC

One engine, not one per screen: a FORM result, a blotter and a live table are the same thing with and without edits.

The trade blotter: edits, four eyes, write-back, and the next blotters

Edit
only if the site configuration and the person's rights both allow; checked on the server

›

Version guard
basedOn the version seen; a moved record is refused with who changed it

›

Outbox
recorded with the change, sent once, resent with the same change id

›

Four eyes
nobody approves their own change; audited

›

Write-back
to the system the record came from (its SOURCE_SYSTEM)

The page

- the grid with the record's panel beside it: Record, History and Raw JSON (the stored record, masks kept)
- sort, filter, group; Pivot › sends the same rows, filters and grouping to the Pivot
- row rules per person: a record outside a person's scope is never sent, live changes included

Never free for all

- a person without edit power sees no edit controls
- a crafted write is refused (403) and audited
- a system without a write path for the command refuses the change, with the reason

The next blotters are declarations, not projects

A blotter is blotters/<id>.blotter.yaml in a module: the kind, columns, editable fields, grouping, links, retention, approval rule. Risk, exposure and inventory blotters are new declarations on the same engine. The trade blotter is the first.

A blotter sends no orders: Drishti is the analytics layer next to the trading systems.

Drishti Elements

04

Every screen drawn from one set of JavaScript libraries

Library	What it holds
drishti-core	formats, the theme, the API client, dialogs, the filter model
drishti-inputs	date and range pickers on the bank's calendars, select, multi-select, combobox, forms from a schema, the code editor
drishti-grid	the grid with pivot, cell editing and write-back; an AG Grid-style adapter
drishti-chart	every chart kind; maps from the world to every country's first-level regions
drishti-data	data sources, live and Arrow clients
drishti-tree · drishti-graph	hierarchies; networks of linked records
drishti-insight	Explain, search, Approvals
drishti-designer	the designer kit: canvas, palette, inspector, outline, live YAML, the designer shell
drishti-renderer	Sutras drawn anywhere, inside the console or another application
drishti-risk	the banking module's views; served only where the module is licensed

One release train

Eleven libraries, each with its own version, released together; the gallery (/elements) shows every component live, and the API reference is generated from the sources.

Every screen, none bespoke

Every page of the console is drawn with them. Guard tests fail the build when a page draws its own select, form, chart or designer: each guard stands at zero.

Fewer lines, more product

The screens' own code fell from 81,053 to 79,764 lines while every new screen of the train was added; the libraries hold 41,295 lines in 186 ES modules.

Eighteen parallel implementations folded into one, the old ones deleted

Search and keys
one search (Alt+K) over entities, commands, screens and Help; one keymap, ? lists the keys

Approvals
every four-eyes decision in one queue: Sutras, record edits, BI, compute, functions, sign-offs, settings

Alerts
rules on one entity and on measures: one engine, one stream

Explain
hover, click or E on any figure: formula, inputs and lineage, why it moved, the workbook

Build home
everything people build in one rail, with what each item uses and what uses it

Sitemap
one file draws the menus, the Admin tabs, the BI rail, Help's sections and the footer

Charts
one chart engine; Vega-Lite visuals translated, Vega retired

Grid, tree, graph, form, select
one of each, from the libraries

The exposure what-if
computed in Java; the Python engine retired

Limits and pre-deal
one limit rule; one pre-deal service

Public data
the per-pack scripts folded into drishti-feeds

Editions
folded into lenses

Why it matters. Two ways to do one job means two places to fix, two things to learn and two answers that can disagree. Each fold deleted the old way in the same step.

The designer kit, and the libraries inside the bank, never from a CDN

The designer kit (drishti-designer)

The layout canvas, palette, inspector, outline and a YAML pane kept in step with the model; the designer shell: Visual / YAML / Both, versions, validate, preview, history, impact, propose for review.

The Sutra designer, the BI report designer, the theme designer and the table and model editors are all built from it.

The same controls everywhere

A business-date picker that reads T-1, ME or 6 Nov 2026 and names the bank's holidays; multi-select with chips and search; labels above fields, problems in words, numbers read loosely (1.5m, 250k); 44 px targets on a phone.

Why it matters. A bank's own intranet pages can show the same grid, chart, Explain or live view as Drishti, from files the bank keeps, so nothing reaches the public internet and every byte can be checked.

Form	How
Two files for one page	drishti.py elements pack --libs core,grid,chart
Release archive	drishti.py elements dist: SHA-384 integrity hashes, npm tarballs for the bank's own registry
WebJar	./mvnw -P elements -pl drishti-elements package, for the bank's Maven repository
From the server	the Drishti server you run serves them
Banking views	drishti-risk only with --include-domain banking

- Nothing is published anywhere public; there is no CDN to point at.
- The same sources make the same bytes: a build can be reproduced and compared.
- Pages that embed the earlier elements keep working, byte for byte.

Aggregate: any scope, any hierarchy

05

From a scope written as text to totals that add up, on any date

Scope as text
BANK - DESK:CREDIT, a node, a portfolio, a rule, a basket, a list of trades

→

Resolve
on the date, to a set of trade ordinals; set algebra, so a trade counts once

→

Entitle
intersected with what the viewer may see; the rest counted as hidden

→

Runs
a position is one run of ordinals; a node holds a few runs, not every trade

→

Sum
each scenario's P&L summed in float64 over the runs; parts summed once

→

Answer
VaR, ES, contributions that add up, provenance; cached by data version

What any scope can be

Word	Example	Resolves to
node	DESK:RATES, LE:NY, LL:LL-0003	the node's trades
portfolio	PORT:PF-USD-RATES	its stored definition
rule	WHERE:CURRENCY == 'EUR'	whole positions that match
list	TRD:..., POS:..., CPTY:...	those trades or positions
algebra	A + B, A - B, A & B	union, difference, both

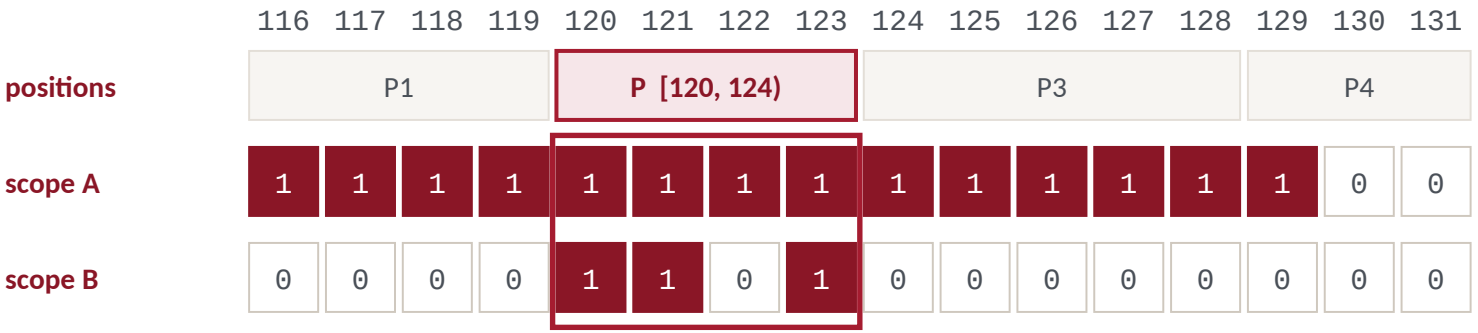
Why it scales

- **Membership is ordinals and runs:** a scope is a set of positions in the date's order, each one contiguous run of trades
- **Parts are summed once:** a desk's parts and their total come from the same pass, so the components add up exactly
- **The display limit is only for display:** a table shows the largest parts; the total is over every part
- **Shards add up:** in a cluster each member sums its shards and the sums are added in shard order

Key insight **Any set of trades is a sentence; the engine turns it into a number in milliseconds, never a stored copy.**

A scope is a set of ordinals; a position is one run

A business date's trades, numbered 0 ... N − 1, sorted by hierarchy cell and position



Is P whole in the scope? `nextClearBit(from) ≥ to`

A: the first ordinal from 120 not in A is 130; 130 ≥ 124, so A holds P whole.

B: the first ordinal from 120 not in B is 122 < 124: B holds P in part, its trades the runs [120, 122) and [123, 124).

A scan of the position's own bits, (to − from) ÷ 64 words, with nothing allocated: the cost of a position is its own trades, whatever the size of the date.

Ordinals

a date's trades numbered in hierarchy and position order: a position's trades are one contiguous run [from, to); an intraday batch's trades follow, listed with their position

A scope is bits[1]

one bit a trade, N ÷ 8 bytes: 25 KB at 200,000 trades, 290 KB at 2.3M; union, difference and intersection word by word; a node is one run

Whole or in part

whole: the position's cached standalone figures; in part: only its runs inside the scope

Cost[2]

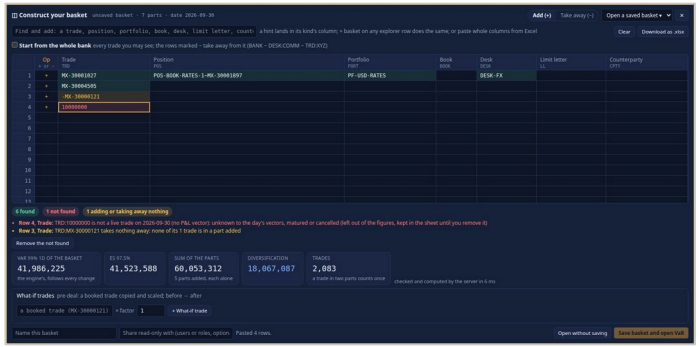
per position: its trades ÷ 64 words; per scope: its positions. Nothing grows with the date

[1–2] How these figures were derived: A-1, in the appendix (How it scales: the algorithms)

Any portfolio a person means, written as a sentence, on 2026-09-30

Scope (as typed)	Kind	VaR 99%, USD	Trades
BANK	node	112,041,646	9,572
BANK - DESK:CREDIT	ad hoc	76,605,857	8,629
DESK:RATES	node	74,633,680	—
DESK:RATES - TRD:MX-30000121	ad hoc	74,720,890	—
WHERE:CURRENCY == 'EUR' and DIRECTION == 'SHORT'	rule	134,918,298	700
NODE:MGMT~DESK-RATES + PORT:PF-USD-RATES - NODE:MGMT~BOOK-RATES-1	basket	live in the tray	—

- **One swap hedges a desk:** the rates desk without MX-30000121 has a VaR 87,210 higher; its incremental VaR is −87,210
- **A rule over positions:** 596 positions, standalone sum 231,439,042, diversification 96,520,744
- **A name that matches nothing** turns red in the tray and says why, never a VaR of 0



Construct your basket: parts added and taken away

- Save it as BASKET:owner/name; it follows its owner across a cluster
- Share it read-only; each reader sees it through their own entitlements
- Nodes and rules are dynamic, lists of ids static; every save audited

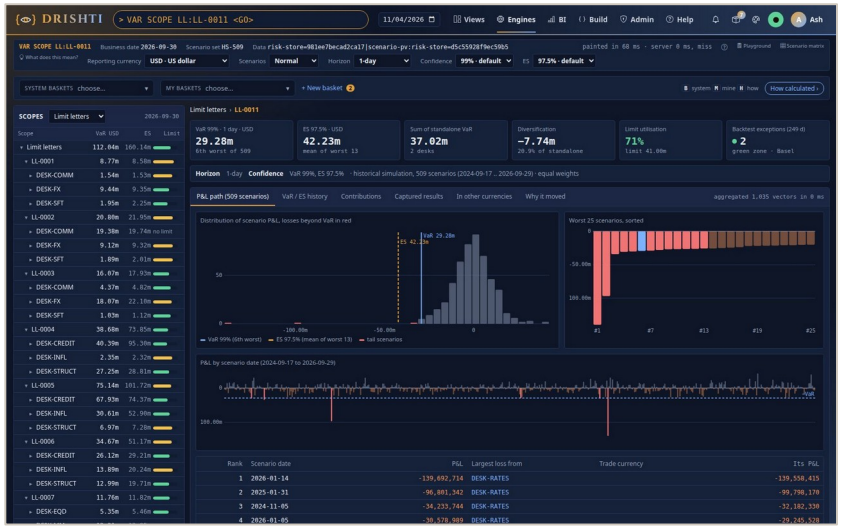
Key insight **A desk without a desk, a rule, a hand-picked basket: each is just another scope, computed when asked.**

Any hierarchy: from the delivered rows, or from the trades' columns

the market-risk pack: hierarchies

```
hierarchies:
  MGMT:  { source: risk-hierarchy,
           limits: var-limit }
  LEGAL: { source: risk-hierarchy }
  LL:    { title: Limit letters,
           levels: "LIMIT_LETTER > DESK",
           word: LL, limits: var-limit }
```

- **Rows or columns:** MGMT and LEGAL from the delivered rows; LL built from every P&L and scenario-PV row's columns
- **A word per hierarchy:** LL:LL-0003 is a scope like DESK:RATES, with its VaR limit from var-limit
- **Proved equal:** MGMT rebuilt from DIVISION > DESK > TT_BOOK gives the delivered tree exactly, to the cent at every node



The limit letters as a tree beside MGMT and LEGAL

Members are rows, not books

a letter may hold part of a book; a position under two nodes is split, and the provenance says how many

A missing column skips one tree

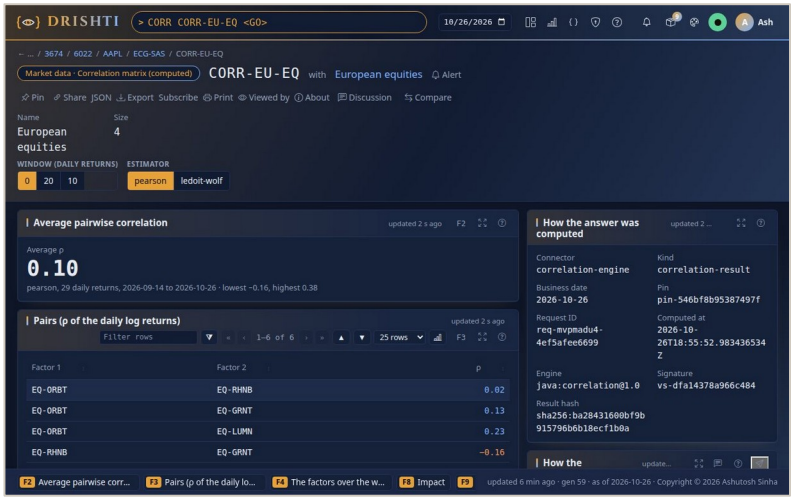
the date still loads; a warning names the column, the kind and the fix

Key insight A new way to cut the bank is configuration: a line of YAML, not a new mart or a new load.

Groupings and analytics computed from their members, not stored

Command	Computed from	Engine
LEIG · LEIC	GLEIF relationships: ownership groups (no member cap), entities per country	gleif-groupings
SIC	the sector's companies: count, revenue, net income	us-equities-groupings
ECG	member economies: GDP summed, growth GDP-weighted, inflation the median	macro-groupings
CORR	price history: Pearson or Ledoit-Wolf over a chosen window	CorrelationEngine
EQB	an equity's beta against its index, from both histories	BetaEngine
USEQ · IDX	ratios, yearly change of a fact, index totals: worked out by the pages	the pages

Every id opens the computed page; a control that feeds a computed panel (CORR's estimator and window) recomputes it at once.



CORR CORR-EU-EQ: estimator and window on the page (tutorial 6).

Every figure can be re-derived

Correlations come from the price history over the chosen window: about 0.1 on the demo, whose price series are independent. The guide reconciles each figure.

Key insight **Retire the stored copy, keep the id: users follow the same links and get a figure they can re-derive.**

Totals over every position; the row limit is for display only

A scope of P positions: one pass, in ordinal order

Each whole position
standalone VaR read from the cache: computed once per date and lens, the first time a scope holds the position, then kept

Each position held in part
the VaR of its trades inside the scope, summed on the fly

Missing from the cache
256 or more: computed in parallel on the engine's sum threads

→

Totals, over every part

`Σ standalone`
the pass on the left

`Σ component = VaR`
no pass at all: the parts partition the scope, so their components add up to its own

`diversification`
`= Σ standalone − VaR`

The row limit: max-parts, 5,000

5,000 of 18,567 positions shown (the largest by gross size); totals are over all 18,567

Position	Standalone	Component
POS-…-0001	4.21m	2.97m
POS-…-0002	3.88m	2.41m
… 4,997 more rows		
Totals, all 18,567	Σ 231.4m	Σ = VaR

- The rows: the largest by gross size, chosen with a heap of max-parts entries, O(P log max-parts), kept in ordinal order
- No total depends on the limit; the answer says truncated, total and shown

Worked: a basket of three positions

Standalone VaRs 100, 80 and 50; the basket's VaR 170.

Σ standalone 230 · Σ component 170 · diversification 60.

With a limit of 2 rows the table lists the two largest by gross size and says 2 of 3 shown; the three totals stay 230, 170 and 60.

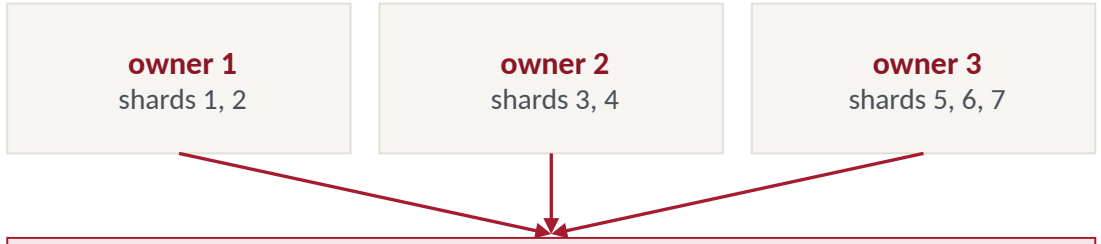
Memory and the k worst[1,2]

8 bytes a position and lens: 1.6 MB at 200,000 positions, 18 MB at 2.3M, in the one cache budget, released with its date. A standalone VaR keeps only the k worst values (k = 6 at 99% of 509) while reading the vector once.

[1–2] How these figures were derived: A-1, in the appendix (How it scales: the algorithms) Figures in the table: illustrative

Sums added shard by shard; a date's files decoded on several threads

In a sharded cluster: partial sums, added in shard order



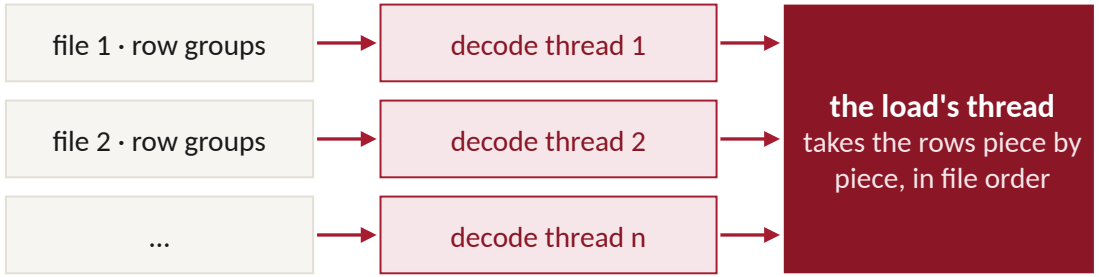
The member answering
adds each shard's partial vector in shard order, then takes VaR and ES once: the same bits as one server

- Exactly one answer a shard; no live owner of a needed shard: refused in words, never a partial VaR
- Every node's aggregate is summed bottom-up as on one server: the bank's VaR is the same bits on every member
- The totals over the parts: computed by the member answering from each position's vector, cached as on one server

Per-shard Σ standalone

each owner returns its shard's partial Σ standalone over the scope's whole positions from its own cache; the sums are additive across shards and, added in shard order, the same on every member

A date's first open: decoded on several threads, in file order

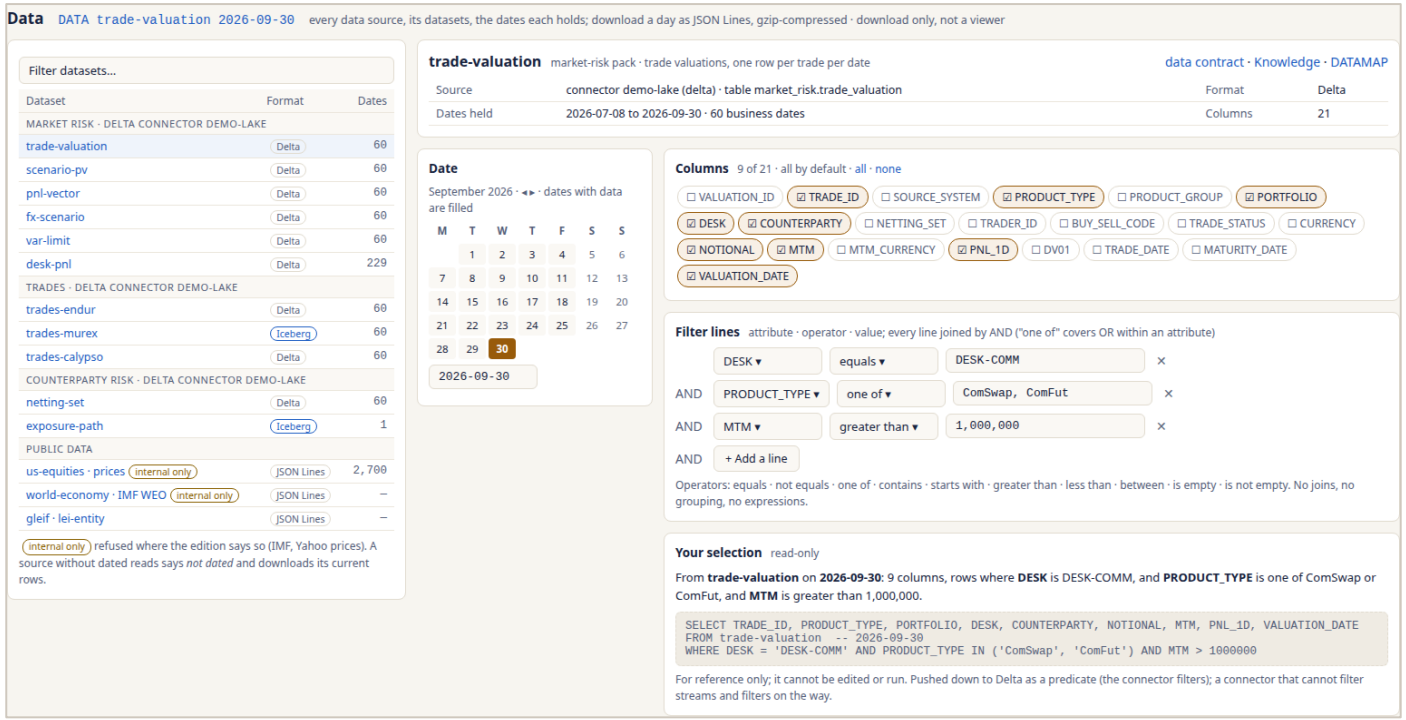


- Only the columns the load uses are read: the id, the vector, the trade's text columns
- The intake, sort, sums and hot file see exactly the rows one thread would give, in the same order: the same book to the bit
- Bounded: drishti.var.read.threads (half the cores, at most 8) and read.buffer-mb (64 MB, at most 1/32 of the heap)
- A piece that cannot be read fails the load with its error, as one thread would

Measured[1]	First open	The read
50,000 positions, one server	1.8 s	0.8 s
200,000 positions, one server	4.5 s	70–73% of CPU
1,000,000 positions, one server	19–21 s	65–69% of CPU
200,000 positions, cluster of 3	10.8 s	by the owners

[1] Measured on one workstation, 24 hardware threads: 50,000 positions at -Xmx4g (VAR_GUIDE 27a.6); 200,000 and 1,000,000: BENCHMARK.md section 4

Any dataset, any date, a few filter lines, streamed down as JSON Lines



Design mock-up: the datasets by source, a date, columns, filter lines

- **Every stored kind a connector serves:** its source, format (Delta, Iceberg, JSON Lines, a database), the dates held, the columns from its data contract
- **A selection, not SQL:** columns and up to 50 filter lines joined by AND: equals, one of, contains, between, is empty
- **Read back twice:** in plain words and as a SQL-like line, for reference
- **Pushed down and streamed:** comparisons go to the lake's column read; rows stream to the file, never held whole
- **Entitled and audited:** masked columns left out, internal-only data refused where the edition says so, each download one audit record
- **Three doors:** EXPORT <GO> (DATA still works), GET /api/v1/data/{kind}/rows, drishti.py data get

Key insight **Taking data out is a governed, streamed download, not a mart built for one request.**

An aggregate kept as a copy drifts; one computed from its members cannot

Overlapping portfolios cannot be added

28 stored portfolios overlap on purpose; a union is computed as a set first, so a position in both counts once. A table of overlapping portfolios never shows a total row.

VaR is not additive

the bank's VaR is not the sum of its desks'. A stored desk figure cannot give the bank minus a desk; the members can.

Stored copies drifted

stored ownership groups were capped at 50 members, stored countries listed 400 entities, stored correlations did not match the prices in the same lake. All are now computed.

Any date, any hierarchy

a stored aggregate exists only for the cuts someone loaded; computed, it exists for every scope on every date held.

Security stays exact

an aggregate is computed over what the viewer may see; a stored total would leak what a mask hides.

Data is not copied

the only stored data is disposable: caches and rollups. Deleting them changes no answer, only the time to it.

Key insight Fundamental data is kept as it is; whatever can be derived from it is computed when asked.

Analyse: BI and analytics

06

Business intelligence on the same data, rules and masks

Models rooted in packs	Datasets and data models built from the packs' kinds, with the relationships the packs suggest.
Three languages, one plan	A Python-like expression language, read-only SQL and Python meet in one plan, your row rules and masks applied.
Servers compute	The Java server and the console's sandbox answer every number; the browser draws; standing queries push live.
Governed	Versions and diffs, four-eyes review, certification, lineage past the connector, bundles with rollback.
Everywhere it is needed	Subscriptions and alerts per reader, PDF, PNG, Excel and CSV, and <drishti-report> in other apps.

Key insight **A report reader sees exactly the rows and masks they see on the terminal — BI cannot leak around them.**

Eleven principles hold the BI design; six matter most here

Four languages only

YAML for structure, Rachana for measures and rules, read-only Drishti SQL over models, and Python; all compile to one logical plan

The servers compute, the browser draws

every device shows the same answer; no totals, sorts or pivots in the browser

Security before arithmetic

row rules and masks are injected into the plan before optimisation; no language construct removes them

Correct by construction

each measure at its own grain; joins that repeat rows refused; currencies never summed without a conversion

Streaming by nature

time is a parameter of every plan: now, as of a business date, the last hour; any query can be subscribed

Governed like code

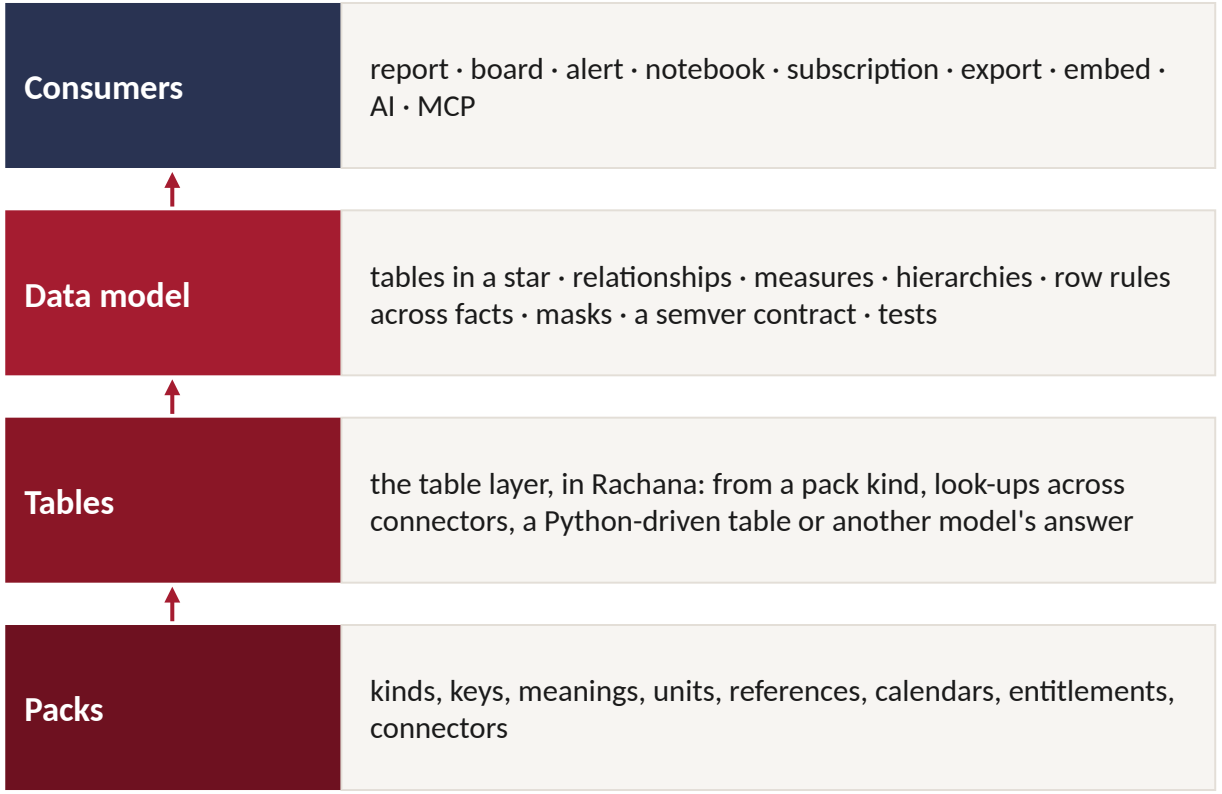
versions and diffs, four-eyes review, certification, lineage, bundles between environments

Deliberately not built

A Power Query clone (Python Transforms instead) · DAX and MDX (Rachana with a closed filter algebra, a published DAX mapping) · computation in the browser · raw SQL to sources · publish to the web (embedding with tokens) · a second storage engine (DuckDB over Parquet is enough)

Key insight **Where Power BI and Drishti differ, it is on correctness, security, live data and governance.**

Packs, tables, one model, many consumers: a mini database



Arrows: each layer reads the one below by reference, never by copy

	Decision
D1	a table is one governed table; a model is tables, relationships and measures
D2	one model per report, board, notebook, alert, subscription and feed
D3	every pack publishes a table for each kind, governed with the pack
D4	the model architecture first, before calculation and authoring depth
D5	one model query language for the UI, the API, AI and MCP
D6	every table has a time mode: live, snapshot or history

Key insight **A model is the unit of query, security, contract, cache and audit: reports reference it, never own it.**

BI on one table layer, written in one grammar, edited with the YAML in view

Legs, Notional by LEGS.CURRENCY rows LEGS.CURRENCY · subtotals below · lazy: closed nodes are not computed · exact

LEGS.CURRENCY	Legs	Notional
+ AUD	44	5.64 bn
+ CAD	59	8.03 bn
+ CHF	51	6.11 bn
+ EUR	128	15.57 bn
+ GBP	107	13.71 bn
+ JPY	122	61.79 bn
+ USD	437	50.66 bn
Grand total	**948**	**161.50 bn**

A virtual table of trade legs in the Pivot: the same table BI's models read (tutorial 18)

Piece	As built
Tables and models	config/tables/<domain>/, in Rachana; BI, the Pivot, Sutras and Flight read the same ones
Row rules across facts	a rule on one fact limits the related rows of another: nobody sees more
Pivot → Report	one analysis path; the risk cubes are Pivot presets; Export data takes any kind out
Proved before the switch	every recorded BI answer reproduced value for value; the old code deleted

Build › Tables & models

One editor for tables, models and Python-driven tables: a visual side with the live YAML below it, either editing the other; a preview read on the server as you; history, impact and four-eyes review.

The Notebook

SQL cells in Drishti SQL, run by the server as the reader; Python cells; a profile; a cell promoted to a report visual, a measure or a table.

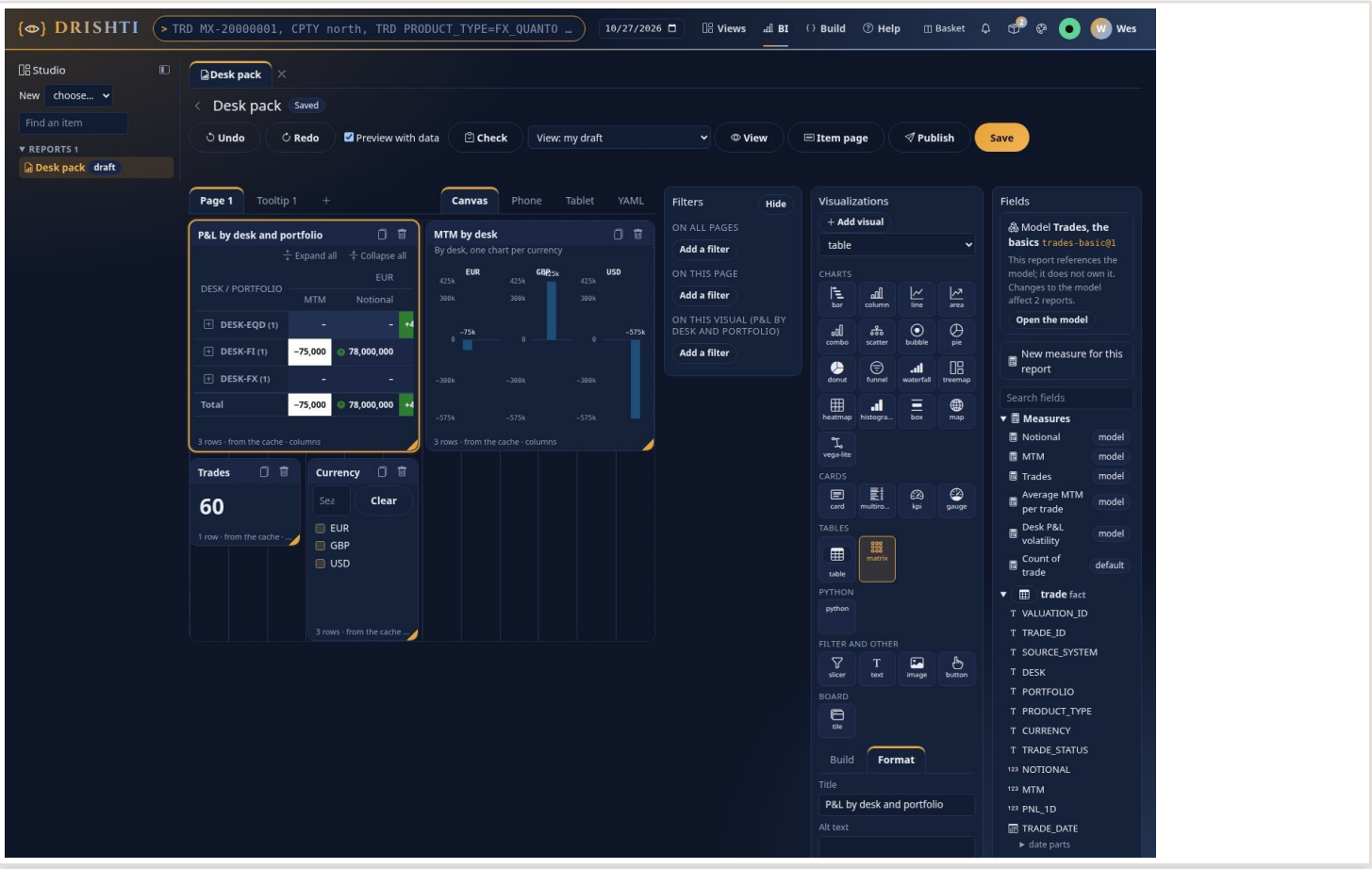
One Python runtime

Every Python run on the server's compute pool, in a strict sandbox on Linux (no network, confined files, limits), against pinned libraries, each run audited and checked against its table's contract.

Resizable everywhere

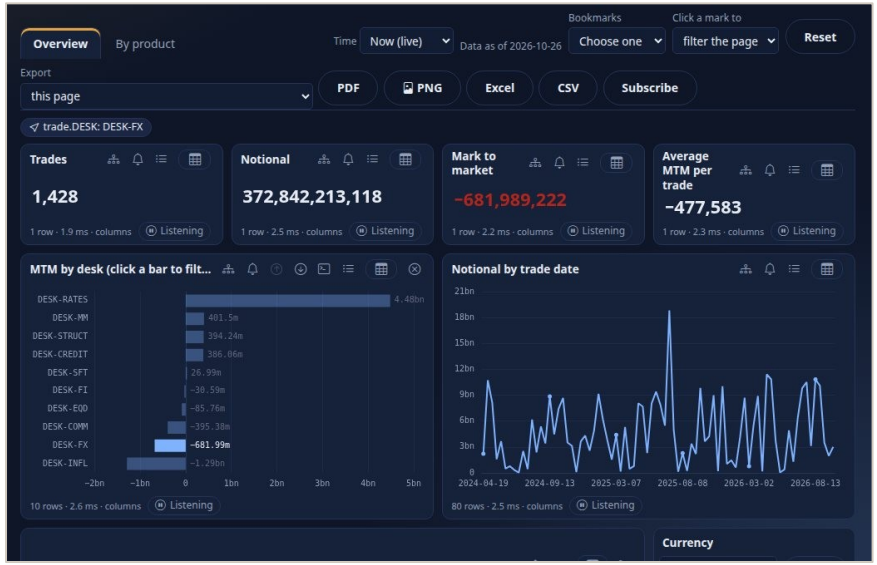
Panels and columns on every screen drag wider, kept per person; a double-click resets.

Reports, models, boards and themes in one studio

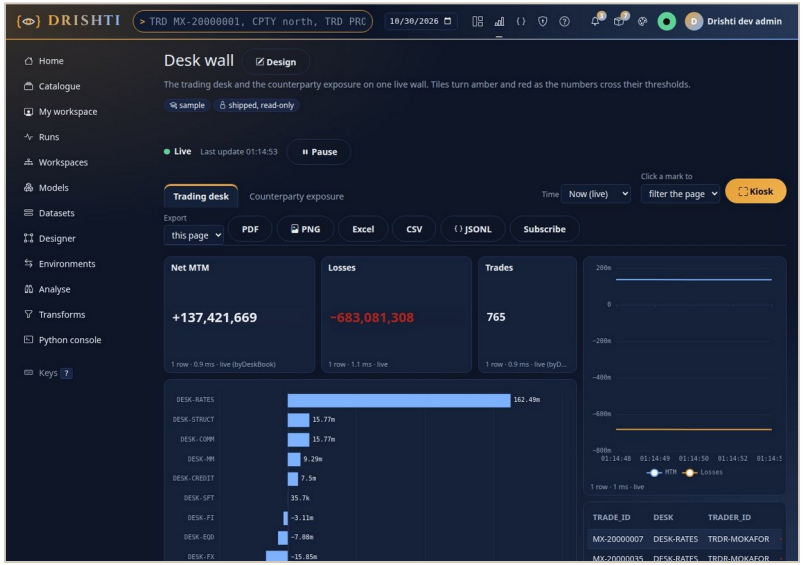


- **Explorer and tabs**
  Every item you may edit, by kind, with New; open items as tabs.
- **Problems and Query**
  The server checks the draft a second after each change; each visual's query and SQL.
- **A generated Format pane**
  Typed formats, conditional formatting, themes, tooltip pages, small multiples, the matrix.
- **Boards are reports**
  Board pages of live tiles beside the report's own visuals, with kiosk mode and alerts.

A report in the designer, cross-filtered, then a live wall for the desk



A click on the map cross-filters the page



A board of live tiles, for a wall in kiosk mode

Model

World economics: seven measures the model owns; GDP per capita is a ratio of sums, not an average of averages

Report

cards, a map drilled from the world to a region, a bar with a top-10 filter, a slicer; publish to a workspace

Live

reports follow the ticks; live tables keep their history; boards of tiles run in kiosk mode

Distribute

subscriptions as PDF, PNG, Excel or CSV, each reader on their own masks; alerts on a tile

Key insight Analysts get the BI tool they know, on the platform's own data and rules.

Every heavy question is sized before it runs, then becomes a visible run

A question
visual, MQ, SQL or statement; Prefer: respond-async

→

The planner
sizes it from metadata only: Delta row counts, Iceberg manifests, built layers

- **Cheap** answered at once, as before
- **Heavy** a run: Runs tab, bell when it ends, share, export, schedule
- **Very heavy** asks to confirm first
- **Impossible** refused before anything runs (DRS-9029)



Chart any table

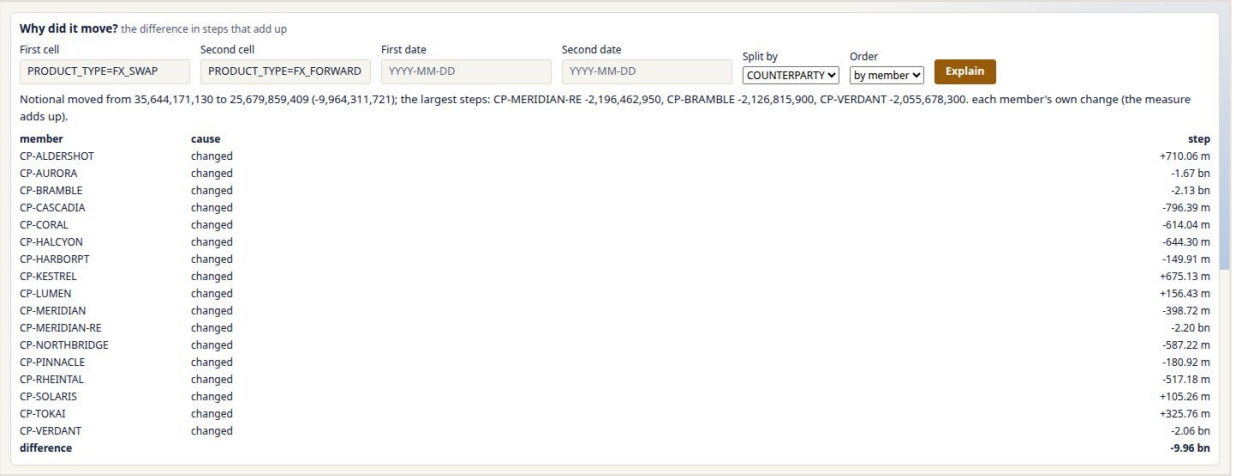
K on view tables, searches, pick lists, admin lists, history and BI tables: six types, Top N, stacking, two axes; PNG, SVG, CSV, Excel. Masked columns are never plotted.

0 connector reads to size a question (RunPlannerTest)

- **Fewer duplicate runs**
  An identical question joins the run already going; a finished answer is reused while the data is unchanged.
- **Masks hold**
  A run computes as its runner; a shared run is the same, re-masked or recomputed as the recipient, decided at every read.

Key insight **Refusing a year of raw trades costs a log read, not a year of builds — and an interactive slot is never held.**

One function catalogue; every difference split into steps that add up



Why did it move? the difference in steps that add up

First cell: PRODUCT_TYPE=FX_SWAP | Second cell: PRODUCT_TYPE=FX_FORWARD | First date: YYYY-MM-DD | Second date: YYYY-MM-DD | Split by: COUNTERPARTY | Order: by member | Explain

Notional moved from 35,644,171,130 to 25,679,859,409 (-9,964,311,721); the largest steps: CP-MERIDIAN-RE -2,196,462,950, CP-BRAMBLE -2,126,815,900, CP-VERDANT -2,055,678,300. each member's own change (the measure adds up).

member	cause	step
CP-ALDERSHOT	changed	+710.06 m
CP-AURORA	changed	-1.67 bn
CP-BRAMBLE	changed	-2.13 bn
CP-CASCADIA	changed	-796.39 m
CP-CORAL	changed	-614.04 m
CP-HALCYON	changed	-644.30 m
CP-HARBORPT	changed	-149.91 m
CP-KESTREL	changed	+675.13 m
CP-LUMEN	changed	+156.43 m
CP-MERIDIAN	changed	-398.72 m
CP-MERIDIAN-RE	changed	-2.20 bn
CP-NORTHBRIDGE	changed	-587.22 m
CP-PINNACLE	changed	-180.92 m
CP-RHEINTAL	changed	-517.18 m
CP-SOLARIS	changed	+105.26 m
CP-TOKAI	changed	+325.76 m
CP-VERDANT	changed	-2.06 bn
difference		**-9.96 bn**

Why (W): notional of FX swaps against FX forwards, split by counterparty; the steps add up to the difference (demo data)

The catalogue	What it holds
Statistics, distributions	norm_dist … weibull_dist, percentiles, var_samp, stddev
Finance on given flows	npv, xnpv, irr, xirr, pmt, rate, durations, convexity
Text, dates, information	round, text, date, datedif, shift_date on bank calendars
Excel, SAS, MATLAB	each family mapped: covered, added, or not with the reason

One grammar, one engine

Rachana's functions are the pivot's and BI's: one name for each, the same figure on the screen, in the API and in an export. The browser computes nothing.

Why did it move (W)

Two cells, or one cell on two dates, split by any field. Sums split by member; ratios telescope; VaR shows each member's change and one step for diversification.

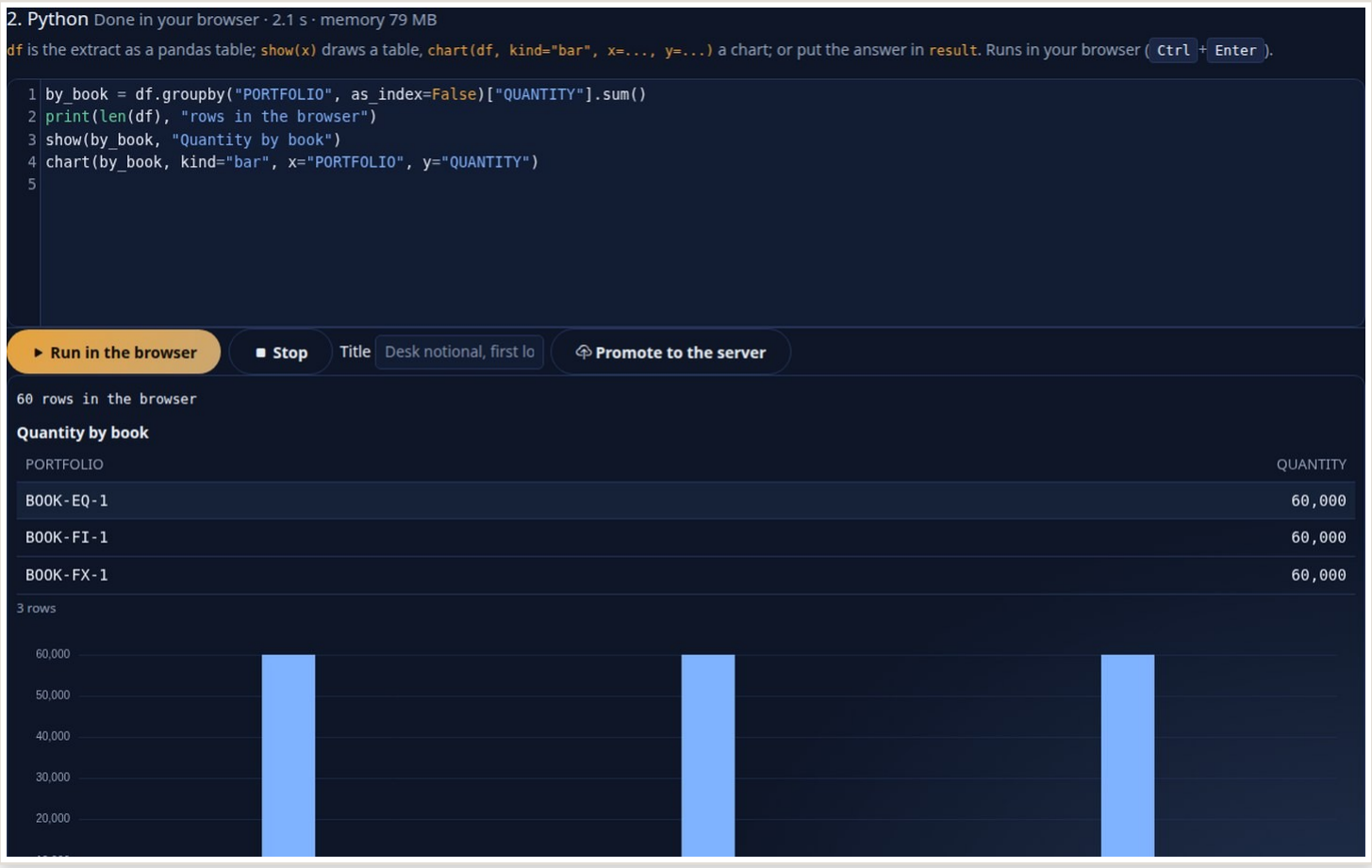
What-if (I)

Leave members out, scale or set a field where a condition holds, add rows: the overlay beside the original, nothing written to the lake.

Functions (U)

Write your own (formula, SQL macro; Python and Java plug-ins off as shipped); a harness proves it before it is saved; approved by someone other than its author.

Python where it belongs: on the server, or on your own rows



- **Transforms, step by step**
  Inputs read through a pack's connector as you; 21 tested snippets; a profile after each step.
- **Sandboxed, warm**
  Measures, visuals and Transforms run in a fork-server child with limits.
- **Your own Python**
  BI → Python console: pandas on a masked extract, in your browser.
- **Promote to the server**
  The same code run again on the backend, compared, kept as a notebook draft.

BI, VaR and exposure read the same data under the same entitlements

BI and analytics

tables and models, reports, boards, the Notebook's SQL and Python run on the server

VaR and ES

any scope, any hierarchy, any currency; the scenario matrix; the backtest

One read of the data

the same connectors, business date, data version, roles, masks and hierarchies

- BI's market-risk-var-official dataset reads the official VaR records, LIMIT_USE per node and date
- a person masked from COUNTERPARTY is masked in BI, in VaR's CPTY: scopes and in Export data

Exposure

EE, PFE, EPE and EEPE from the paths, netting and collateral path by path

The terminal

the trade, the desk, the netting set: every number linked to its source

Key insight Analyses never disagree about who may see what, because there is only one place that decides it.

Arrow everywhere: DuckDB in the server, Polars in Python, Flight between

Layer	Choice	Why
In-memory and wire format	Arrow everywhere	Arrow IPC to the browser, Arrow C Data between engines in the JVM; Arrow Flight and Flight SQL out to any client, as built
Storage	Parquet in Delta or Iceberg	unchanged: every engine reads it
Ad-hoc analytics and BI	DuckDB, kept as the one embedded engine	Arrow export removes the row-by-row JDBC read; direct Delta and Iceberg scans remove a copy
Python tools, loaders, compute packs	Polars, with deltalake and pyarrow	multi-threaded JSON Lines and Parquet reads; lazy plans push filters into the scan
VaR and exposure	the Java engines, in the same role	bit-level guarantees, sharding and hot files no general engine has
Anything new and native	a sidecar over Arrow Flight	a crash or a leak costs the sidecar, not the server

Watched, not chosen: DataFusion (its Java binding is young), Velox (no embedding story), chDB (overlaps DuckDB at these sizes), Lance (a candidate for AI embeddings only).

Key insight The aim: an ad-hoc question over millions of rows answers in tens of milliseconds.

Compute: risk measures on demand

07

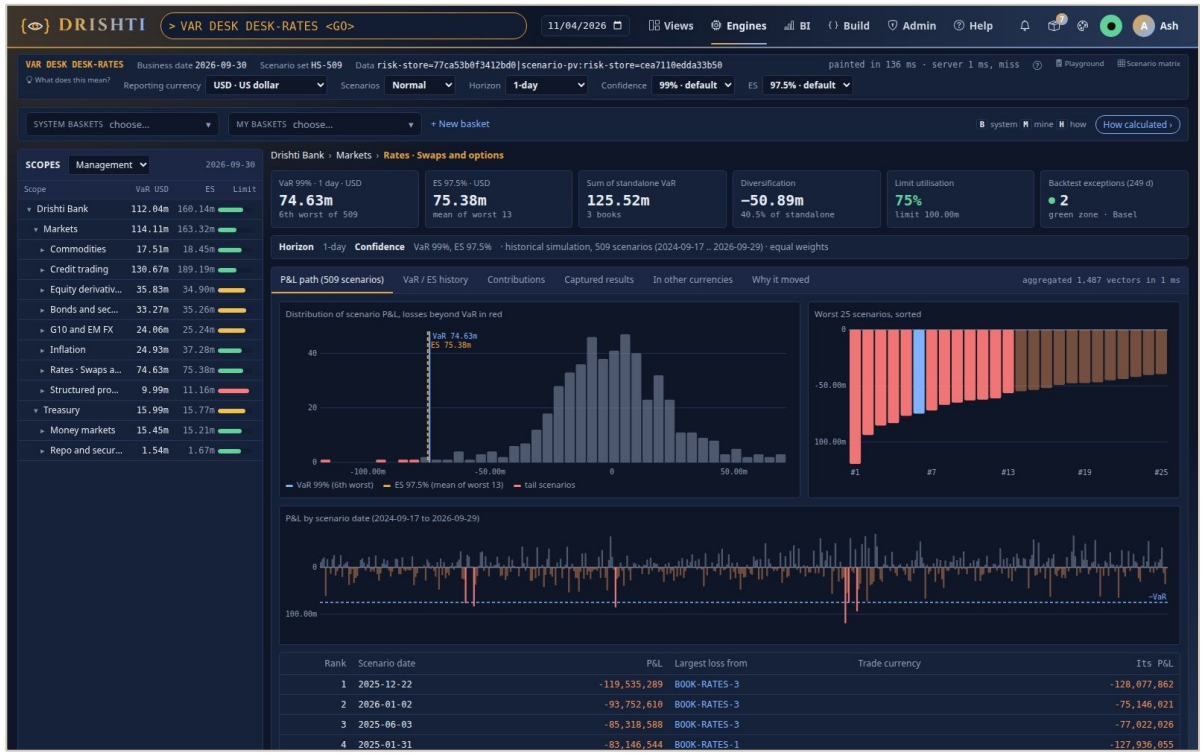
Risk measures are analytics too: computed from inputs, when asked

Measure	Computed from	For	Explained by
VaR and ES, 1-day and 10-day	P&L vectors or scenario PVs per trade or position	any scope, hierarchy, currency	the P&L path, SMAT, the Playground
Stressed VaR	today's positions under a stress period's moves	any scope	the stress window selection (SWIN)
Backtest and period statistics	the official records and the desks' actual and hypothetical P&L	any node, any period	the zone, the exceptions, the lineage
Exposure: EE, PFE, EPE, EEPE	the simulator's paths, netting and collateral path by path	the bank to one trade	the path fan, the collateral tab
Computed attributes	trade-grain inputs on the netting set and counterparty pages	every page	checks on the page, reconciled
Groupings and analytics	GLEIF entities, companies, economies, prices	groups, countries, sectors	LEIG, LEIC, SIC, ECG, CORR, PXH

What they share: inputs kept as delivered; nothing finished is stored; any scope on any date; the viewer's entitlements; an official record when a figure must not change; and, metric by metric, a playground that shows the working.

Key insight **A risk measure is one more computed analytic on the bank's data, not a separate system with its own store.**

VaR is a computation over the trades, never a stored number



VAR DESK: the scope tree with limits, six tiles, the P&L path and its tabs (VAR_GUIDE §3).

112.04m
bank VaR 99%, 1 day, USD, 2026-09-30

160.14m
ES 97.5%: the mean of the 13 worst

509
historical scenario days, equal weights

9,572
live trades in 7,984 derived positions

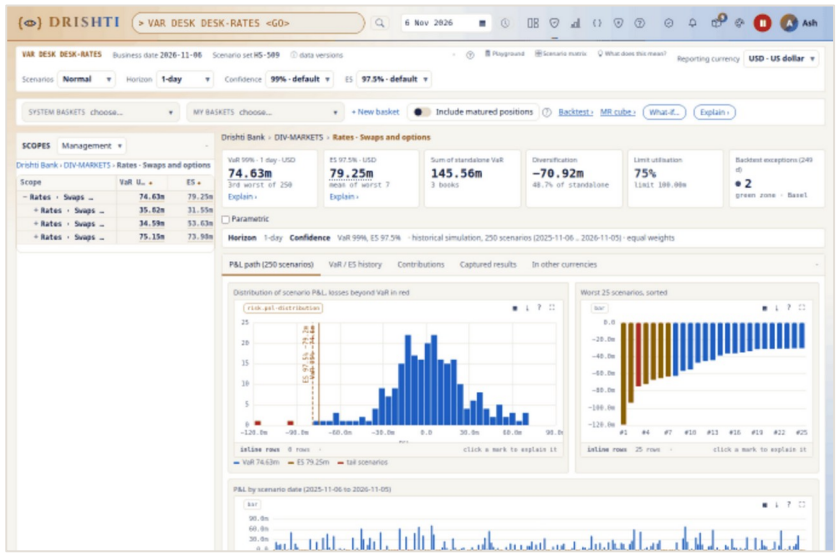
131 · 28
nodes in 3 trees · portfolios

< 1 ms
a node's VaR from its aggregate

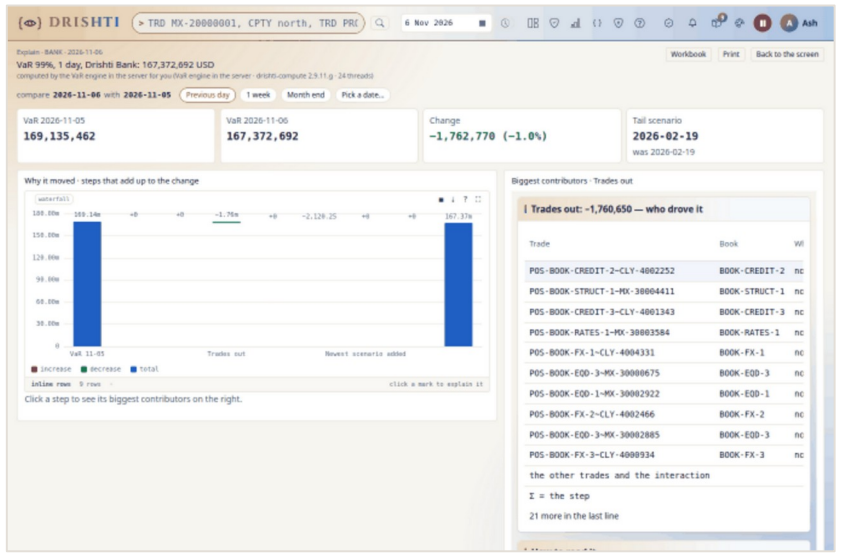
Any scope: bank, legal entity, desk, book, limit letter, position, trade, a portfolio, a rule or a basket, joined with + − &. Stored VAR retired; official records of every node of every tree, kept 7 years.

See also **In depth: the VaR engine deck — method, data flow, inputs, scale and governance.**

Scoped tree, Explain on every figure, and Why it moved against any date



The VaR page of a desk: the tree shows the chosen scope only



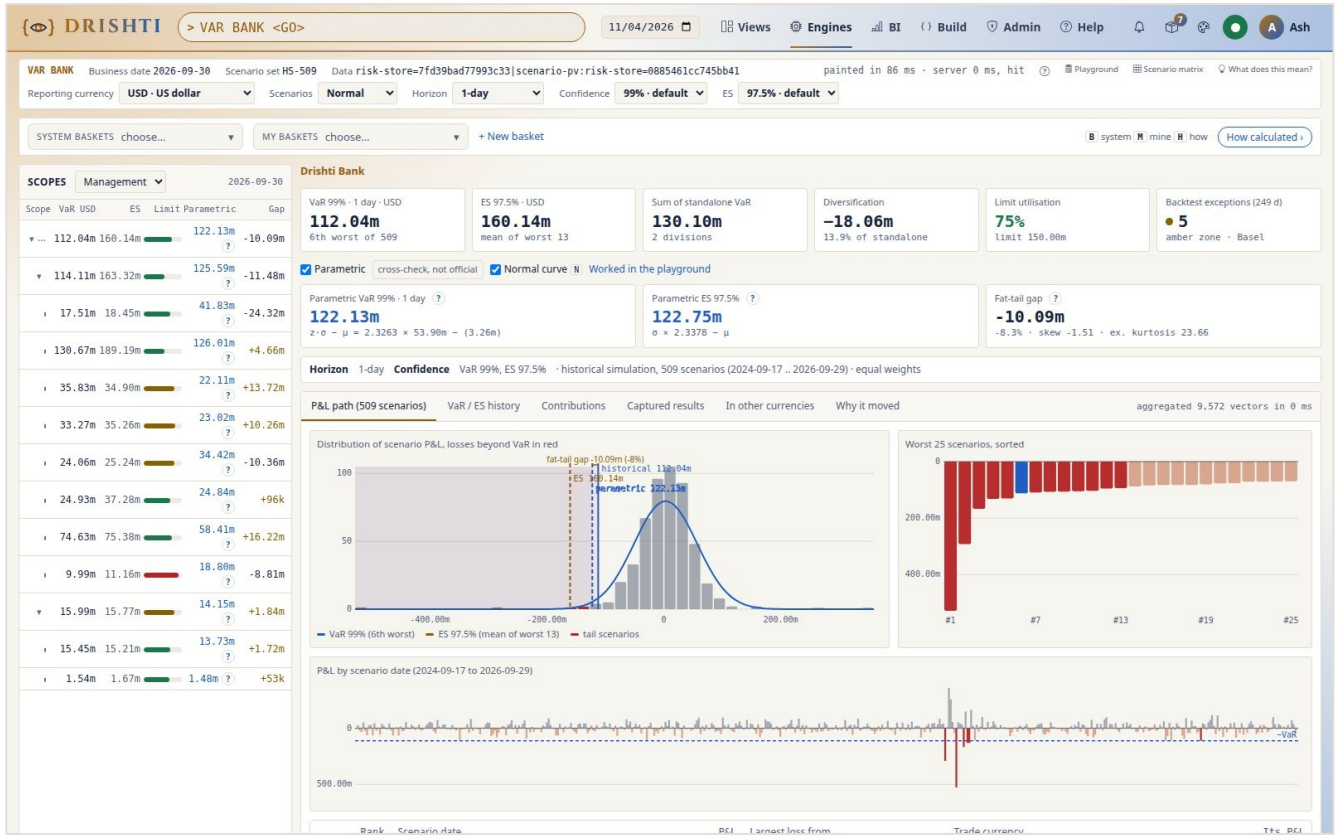
Why it moved: steps that add up to the change

- Scoped tree: the bank, a node and below, or a basket's own positions by any hierarchy; each part's VaR from the engine
- Explain on every row and tile, on every page that shows VaR or ES
- Why it moved: compare with the previous day, a week, month end or a picked date
- four figures, the waterfall first and wide; click a step for its biggest contributors

Why the steps add up exactly

Drishti moves one thing at a time from yesterday's state to today's (restatements, hierarchy, trades out, the oldest scenario dropped, the market, the newest scenario added, trades in) and recomputes VaR at each stage; each step is the difference between two stages. The pictures are the demo data.

Parametric VaR beside historical: a cross-check, never the official figure



VAR BANK with Parametric on: the tiles, the tree's Parametric and Gap columns, the normal curve over the P&L histogram (demo data, 2026-09-30)

Reading it. Historical VaR 112.04m against parametric 122.13m at 99%, one day: a gap of −10.09m. The gap is the signal: a scope whose tail is fatter or thinner than the normal says so at once.

A toggle, not a second screen

Parametric on the VaR explorer adds parametric VaR and ES, the fat-tail gap, the tree's columns and the curve (key N). PVAR opens the explorer with it on.

The method, shown

A normal fitted to the scope's own scenario P&L: VaR = z·σ − μ; ES = σ·φ(z)/(1 − c) − μ; equal weights or EWMA (λ 0.94). Every figure has a ? with the server's own working.

By hand, and in Excel

The VaR Playground's Parametric tab: each step a formula, each position's component adding up exactly; the workbook recalculates to the engine's figure.

Never official

Limits, backtests and official records stay historical simulation. pvar() also works in the pivot and the MR cube.

10-day VaR two ways, and any offered confidence, from the same vectors



VAR BANK 10D SUM: the Horizon, Confidence and ES pickers beside the reporting currency; the tiles and the method strip follow the lens.

Method	How	Scenarios
√10, the default	every 1-day P&L × √10: VaR and ES × 3.1623	the same 509
10-day sum	each window the sum of 10 consecutive days	509 − 9 = 500

Demo bank, 99%	VaR	Rank
1-day	112.0m	6th of 509
10-day √10	354.3m	× √10
10-day sum	274.5m	5th of 500

The confidence chosen

`k = ⌈N × (1 − α)⌉`: 99% of 509 is the 6th worst, 95% the 26th, 99.5% the 3rd: another rank of the same sorted vector, nothing stored

- VaR from 95%, 97.5%, 99%, 99.5%; ES from 95%, 97.5%, 99% (configurable)
- Limit use only like for like: a 1-day 99% limit is not compared with a 10-day figure
- Backtesting stays 1-day; the official records stay 1-day at the configured confidence
- Bit-equal whatever the shards and threads that summed the 1-day vector

Figures: the demo bank on 2026-09-30, USD, the committed synthetic test data (VAR_GUIDE 18.6, VarTestDataPrecisionTest)

Stressed VaR: today's positions under a stress period's moves

The same historical simulation, on a second scenario set

Today's positions
today's trades, sizes and sensitivities

→

A stress period's moves
a fixed 12 months, e.g. 2008-07 to 2009-06: about 250 days

→

Stressed P&L vectors
the pricing system revalues today's trades under each stress day

- Not the positions as they were then: a bond bought last month has a stressed P&L for 2008-10-10
- A second set of the business date, through the same feed and contracts: SET_TYPE STRESS (or the HS-STRESS- prefix)
- Its own book and hot file, loaded on the first stressed question; never written to the lake
- VaR at any offered confidence, ES, components, contributions; 1-day and 10-day (√10 or the 241 windows)

On every official record: SVAR, SES and the stress set's id, beside VaR and ES

The 12-week average of a node's official stressed VaR, for the capital rule's max(latest, multiplier × average); official dates without one are named, never counted as zero

The demo's stress set is synthetic. 2008-like moves made by the test-data generator, not market history: the set says so and so does every stressed answer (the method strip reads **synthetic**).

Normal or Stressed, beside the other pickers

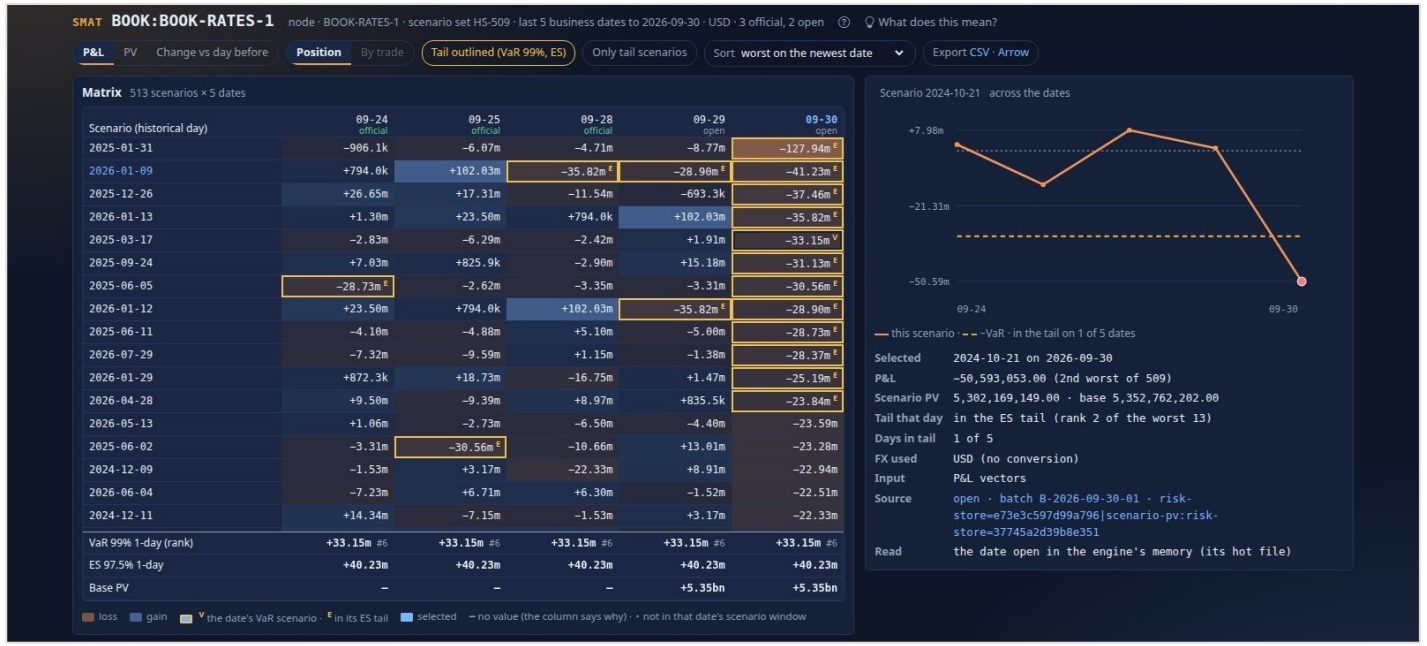


Bank, 99%	Normal	Stressed
1-day	112.0m	213.6m
10-day √10	354.3m	675.6m
10-day sum	274.5m	293.3m

- Reported in USD, the stress set's own currency
- No limit use: VaR limits are set against normal VaR
- Not backtested; intraday batches update the normal set
- Commands: VAR BANK SVAR 10D SUM; the API's set=stressed; the CLI's --stressed

Figures: the demo bank on 2026-09-30, USD; stressed on the synthetic stress set HS-STRESS-2008, 250 days (VAR_GUIDE 18.8, VarComputeFirstTest)

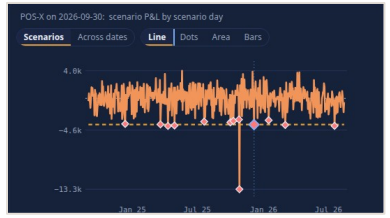
SMAT: one trade, position or node, scenario by scenario, across dates



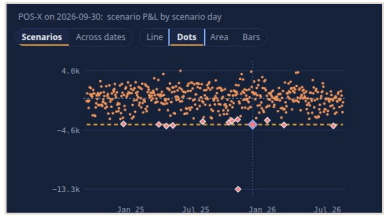
SMAT BOOK:BOOK-RATES-1 5: scenario days down, dates across; V the date's VaR scenario, E its ES tail

- Answers: is this loss scenario new, or in the tail all month? Did VaR move because the book changed or the window rolled?
- P&L, PV or the change against the day before; only the tail scenarios; By trade for a position; CSV or Arrow export
- Each date's numbers name their source: open in memory, read from the lake by id, the official record, or the engine

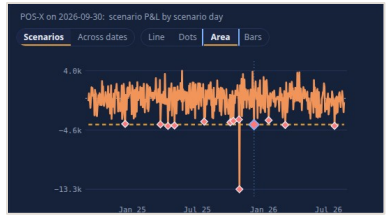
The chart: Scenarios or Across dates



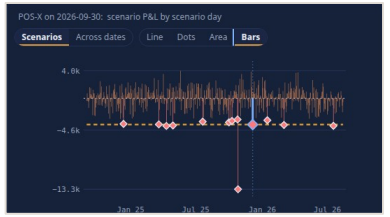
Line



Dots



Area



Bars

- One click switches the type, for both views; remembered in the browser
- The reference line: −VaR for P&L, PV at VaR for PV; the ES tail as diamonds
- The legend names exactly what is drawn

Scenario PVs or P&L, by trade or by position, from files or Kafka

Each system sends	By trade	By position
Scenario PVs in the trade's own currency	yes	yes
P&L vectors any currency	yes	yes

Mix and switch: systems mix on one date and may switch day to day; one form and grain per system per date (DRS-5210); a wrong record is refused and named (DRS-5208), never guessed; old files: drishti.py convert --set.

Intraday: only the changed trades, applied as new − old up the tree

Morning run
FULL: every live trade; the official records

→

Changed trades
files, or Kafka windows of 5 min (500 early)

→

Lake append
one intraday batch, latest COMPUTED_AT wins

→

Apply
book, every node above, every currency

One JSON Lines record, four forms

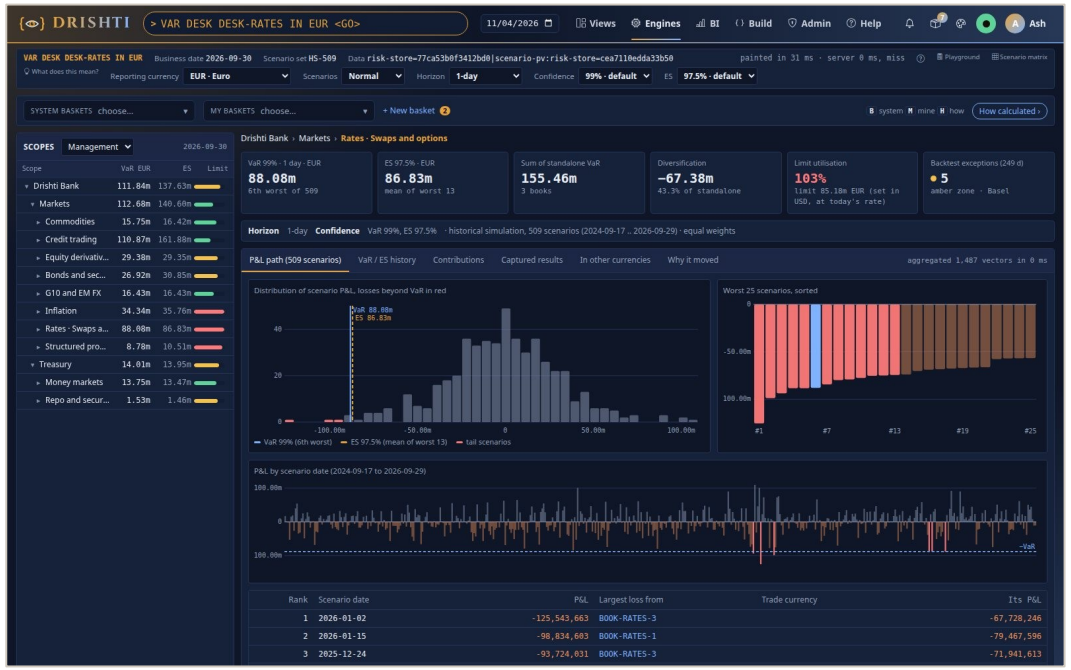
```
{"VALUE_TYPE":"PV","GRAIN":"TRADE",
 "TRADE_ID":"1000279","CURRENCY":"EUR",
 "BASE_PV":…,"PVS":[…509…],
 "COMPUTED_AT":"…","SOURCE_SYSTEM":"…"}
```

The guarantee
Incremental equals the same data loaded as one run, to the cent, on every node; replaying the Kafka topic after a restart writes nothing twice.

Cost of a batch
∝ trades changed × nodes above × currencies built — never the size of the book. Official numbers come only from full runs.

Key insight Source systems keep their own formats; Drishti turns every input into one internal form, a P&L vector per row.

The same trades, a different VaR per currency — exact to the cent

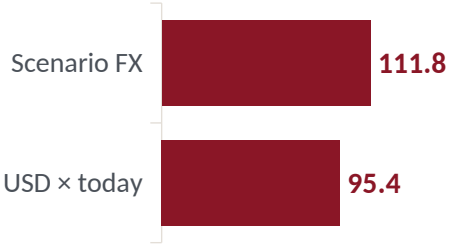


VAR DESK DESK-RATES IN EUR: every panel in EUR, the FX named.

P&L(s) = PV(s)·FX(s) − PV(base)·FX(base), every cross via USD; each node keeps its PVs summed per trade currency.

Ccy	Bank VaR 99%	Bank ES 97.5%
USD	112,041,646	160,143,809
EUR	111,837,045	137,631,546
CAD	189,801,404	230,680,413
JPY	20,883,226,318	23,578,418,999
GBP	104,139,554	125,595,958

Bank VaR, EUR m



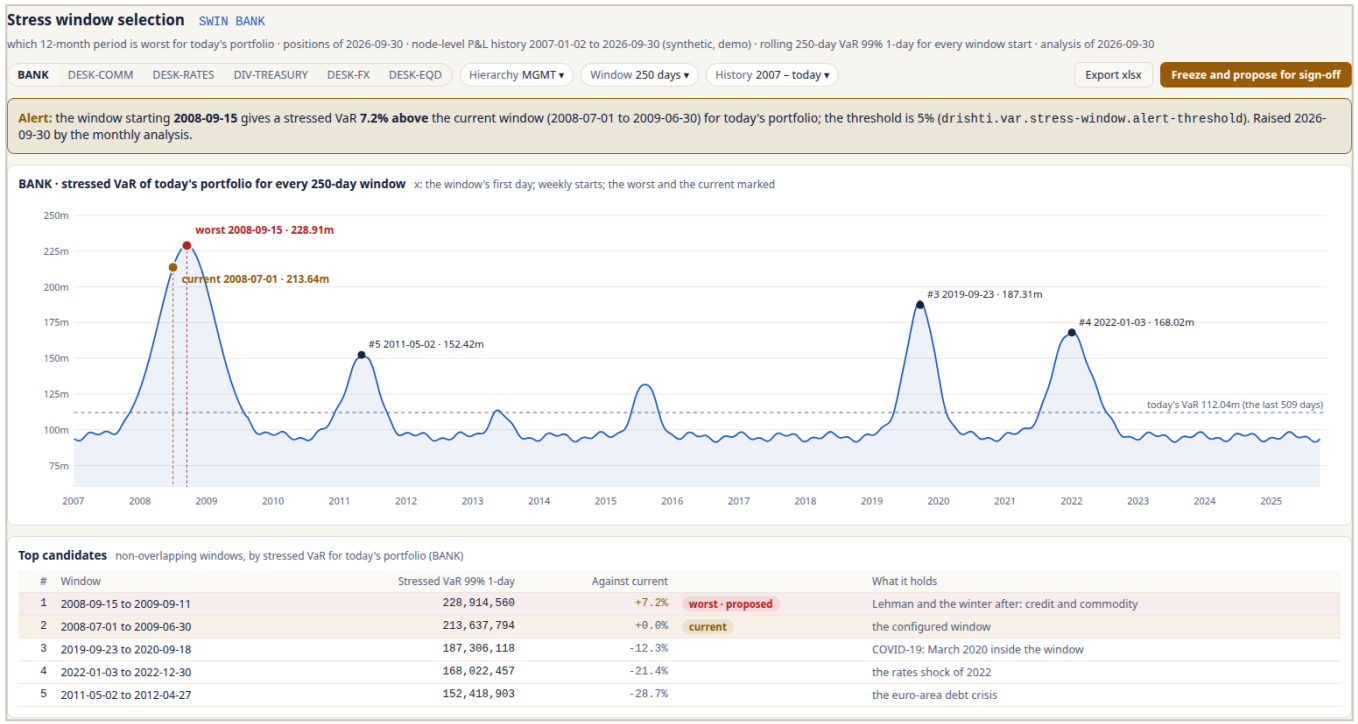
A net 3.65bn USD[1] of value moves against the euro on the scenario days.

Setting	What it does
`drishti.var.scenario-window`	all (509) or the newest N; window=250 per request, labelled, never official
`DRISHTI_VAR_CONFIDENCE`	VaR 99% by default, 95% by setting; ES 97.5%; every label follows
`drishti.var.currencies`	USD, EUR, CAD, JPY, GBP built ahead; any other currency on first ask

Key insight A euro-based owner carries a risk a dollar-based one does not: EUR VaR is not USD VaR times today's rate.

[1] the demo bank's net value on 2026-09-30, from its positions (VAR_GUIDE.md section 15); it is why EUR VaR (111.84m) differs from USD VaR at today's rate (95.4m EUR)

Every 12-month window since 2007, valued for today's portfolio



Design mock-up: the alert, the history, the candidates (illustrative figures)

The rule	As configured
window length	250 business days
VaR of a window	the 3rd worst at 99%
history	2007 to the day before
alert threshold	5% over the current

- **Rolling:** every window valued as a stressed VaR of today's portfolio
- **Apart:** candidates at least 250 days from each other, so each is a different period
- **Alert:** when the worst window beats the current one by more than the threshold
- **By node:** each node's own worst window; one is chosen for the bank
- **Frozen and approved:** the choice is frozen with its inputs and approved by a second person

Key insight **The stress period is chosen by computation and approved by people, not carried forward by habit.**

250 days of VaR against actual and hypothetical P&L, by node



Design mock-up: 250 days, the zone card, the desks (illustrative figures)

green · amber · red

0–4, 5–9 and 10 or more exceptions in 250 days at 99%, as configured

the multiplier

from the configured table: 3.00 to 4.00 in the US; the Canadian row set by the bank

the desks that drove it

each exception names the desks with the largest actual losses that day

- **Three P&Ls:** actual and hypothetical from the bank's desk P&L feed; risk-theoretical from the official record
- **The rule as configured:** green 0–4, amber 5–9, red 10+; the multiplier table, the count read (US: actual; OSFI: the higher); Kupiec and Christoffersen on the actual P&L; the desks that drove each exception

Key insight VBT BANK <GO>: the backtest is computed from the records when opened, for any node.

Period statistics, a report filled cell by cell, checked, frozen and signed



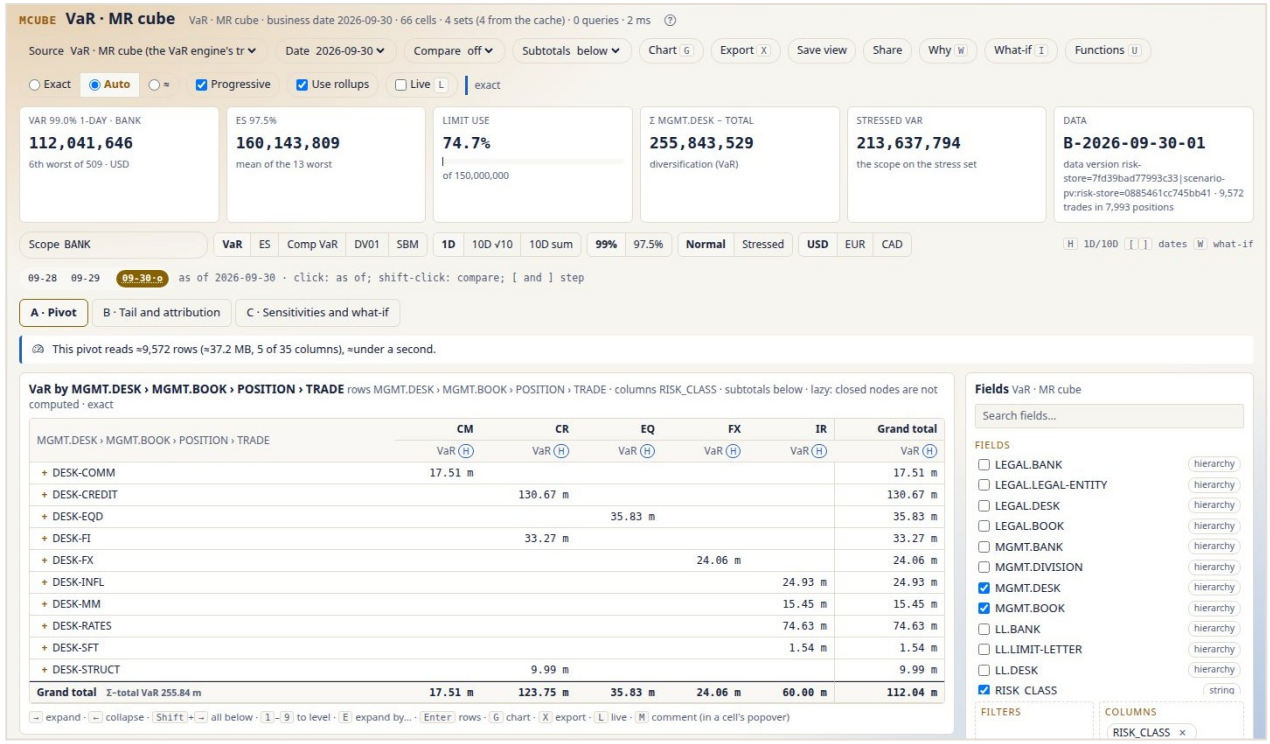
VAR BANK STATS FY2026 on the live demo stack, 30 October 2026: a preview until the year closes

- **Period statistics from the official records only:** period end, average, high and low with their dates, the 60-day average for the capital formula, the stressed VaR's 12-week average
- **Missing records are named, never skipped:** here the records start on 2026-10-26, and the page says so

Definition	→	Run	→	Checks	→	Sign-off	→	Files
one YAML file a report		every cell filled, with lineage		a failure blocks sign-off		maker, checker, approver		xlsx, csv, PDF

Key insight Reports are produced automatically and signed by people; a restatement makes a new run, never an edit.

The MR cube, FRTB-SA and SIMM, computed when asked



MCUBE: VaR by desk × risk class; each cell the VaR engine's figure for its own scope (demo data, 2026-09-30)

In every cell: click for its provenance (scope, members, data version, method); Why splits a move into steps that add up; What-if sets an unwind beside the original.

The MR cube (MCUBE)

VaR, ES and sensitivities pivoted by any dimension on the pivot engine. A cell is never a sum: it is the VaR engine's answer for that cell's trades; the gap shows as diversification.

VaR for any scope

Any node, portfolio, basket, ad-hoc expression or trade column (by trader, say): one engine, the same figure on every screen.

FRTB-SA (FRSA), computed

From position sensitivities: delta, vega and curvature with the three correlation scenarios, the DRC and the RRAO; Basel MAR21–23 parameters versioned and hashed into every answer; PLAY FRTB.

SIMM, per netting set

Collect and post, against the counterparty's call; PLAY SIMM. ISDA's calibrated numbers are licensed: the parameter file ships empty and the bank installs its own.

Exposure computed for any scope, down to a trade, collateral path by path

Paths
every trade's MtM on every path and date (exposure-path)

→

Net
per netting set, path by path

→

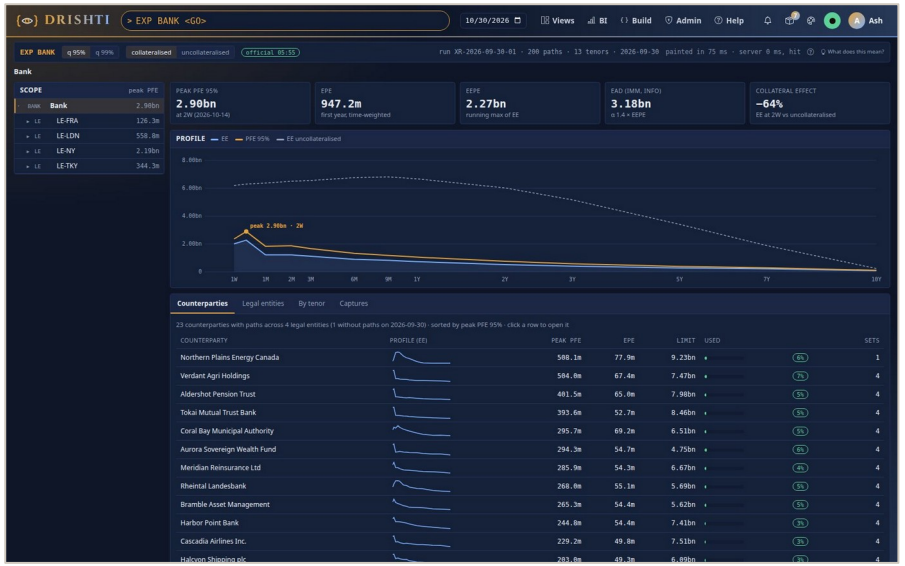
Collateral
each CSA: threshold, MTA, IA, rounding, margin period of risk

→

Measures
EE, ENE, effective EE, PFE 95/99, EPE, EEPE, peak PFE, EAD

→

Any scope
NSET, CPTY, GROUP, LE, DESK, BOOK, TRD, BANK; + − &



74
netting sets on the demo, 8,022 live trades

85.1m
peak PFE, NS-ALDERSHOT-LDN

61 ms
the bank of 9,901 sets (measured)

0.06 ms
one netting set (measured)

Explorer, pre-deal what-if (exact on the run's own paths, or a marked proxy), captures, official results, tiered retention: trade paths 20 days and month-ends; statistics 7 years.

Key insight **The stored peaks summed to 92.6bn; computed with the CSAs on the same sets, 7.6bn — the stored profile is retired.**

Derived on demand from trade-grain inputs, never stored

Trade valuations
live status, a netting set, maturity not before the date

→

Derived kinds
built once per date and data version, shared

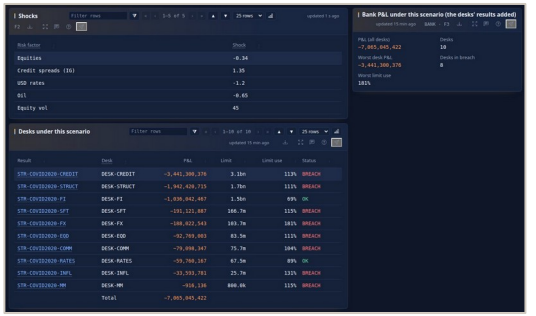
→

Beside the stored
computed, stored and their difference on the page

Command	One page per	What it computes
NSMTM	netting set	net MtM, trades, gross ±, netting benefit, net-to-gross, uncollateralised CE, by product group
CPMTM	counterparty	net MtM, its netting sets, Σ max(set net, 0) as uncollateralised CE
SSPNL	stress scenario	bank P&L (Σ desks), worst desk, breaches, limit use per desk

Live, one rule everywhere: a live-status trade past its maturity is in no total; it is counted and listed as a data-quality warning. Every total says its sum currency (USD).





Key insight **A figure on the page is a computation over the inputs, shown beside the stored copy it will replace.**

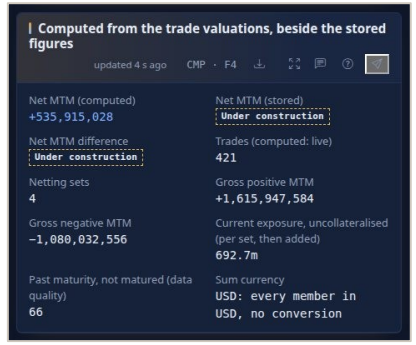
Checks on the pages; exposure on its real sources



Collateral balance: held, posted and net after haircut, from the positions



Credit limit: stored usage, so headroom and utilisation say Under construction



Counterparty: computed from the trade valuations; the stored copy Under construction

The exposure engine reads	From
A netting set's members and MtM	trade-valuation, live status
Collateral held today	the positions' value after haircut
Holdings of the 76 balances	6,229.6m after haircut (6,304.4m market value)

BANK, 2026-09-30, USD	Computed
Peak PFE 95%	2,897.6m
EPE	947.2m
CE	1,586.8m

Key insight Exposure reads its fundamental sources: live trades' valuations and collateral after its haircut.

Pre-deal and one limit rule

08

Does the new trade fit the limit? Approve, refer or decline, with the reasons

```
PREDEAL CP-ALDERSHOT LIKE 1000001 250M
PREDEAL CP-ALDERSHOT EQUITY 768495000 2031-10-20 MTM 9227785
```

Decision	When
approve	exact data; the limit and every tenor bucket within their limits after the trade
decline	exact data; the limit or a tenor bucket over its limit after the trade
refer	anything less than exact, each reason said: a proxy answer, no limit on record, a stale run, netting sets the person does not see, no exposure on the date

How it is computed

A booked trade's simulated paths copied and scaled (LIKE), or the trade described with a conservative proxy; added to its netting set path by path; held to the counterparty's credit limit and each tenor bucket (0-1Y … 10Y+).

The answer

The decision and its reasons; the limit and each bucket before and after; the headroom; EPE and EAD before and after. A proxy never approves and never declines.

Computed as the person asking

Their netting sets only; nothing kept between requests. Every check audited, and explained by the one Explain.

One service

Counterparties → Pre-deal check, POST /api/v1/risk/exposure/predeal, and the Trading landing's new-trade check all ask the same service.

Under construction: SA-CCR's incremental EAD in the answer, said so on the screen.

Within, near or breach: said in the answer, coloured the same everywhere

Within
below 80% of the limit

Near
from 80% up to 100%: still within, said in amber

Breach
over 100% of the limit

Where	Held to the rule
VaR	every scope and hierarchy node with a limit; the VaR what-if
The official record	the end-of-day figures and their limits
Pre-deal	the counterparty's credit limit and each tenor bucket
Sensitivities	the sensitivity ladder's limits

One setting

drishti.risk.limit-near (0.8 as shipped), in the banking module's VaR engine file.

One colour per status

Every bar, tile, pill and heat-map cell takes its colour from the server's status: the screens' own thresholds are gone.

Why it matters. The same use of a limit can no longer be amber on one screen and red on another. The status comes with the number, so a report, an export and a screen agree, and Near never changes a decision: it is the warning before the limit.

Explain and trust

09

Every figure as real formulas that equal the engine to the cent

1 Open
PLAY VAR <scope> or Playground on the VaR page: the same date, currency, horizon, confidence

2 Grid
positions across, scenarios down; the formula bar shows each cell's formula

change a PV in the sheet and every figure moves; the check cells say whether it still equals the engine

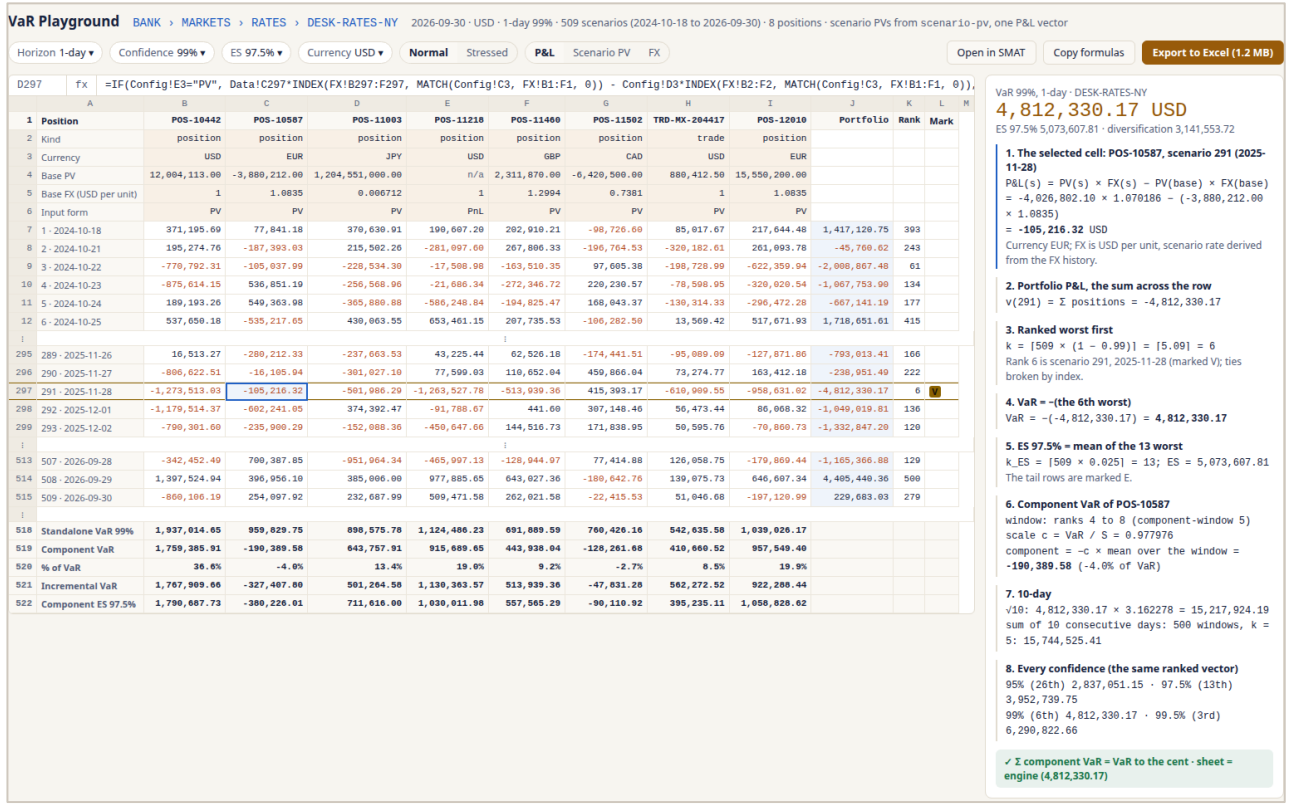
The engine's figure
the sheet's VaR equals the page's, to the cent

3 Explain
the steps in words with the real numbers: PV × FX, the sum, the rank, k, VaR, ES

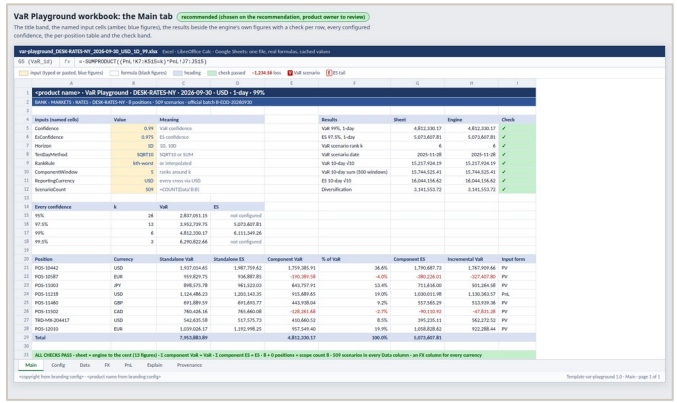
4 Workbook
an .xlsx of real formulas: Main, Config, Data, FX, PnL, Explain, Provenance; check cells

Key insight Trust us becomes check it yourself: offline, in Excel, LibreOffice or Google Sheets.

The grid with its explanation on screen; the same calculation as a workbook



Design mock-up: grid, formula bar and the explanation beside it



Design mock-up: the workbook's Main tab

- **Up to 500 positions** on screen; the rest of the scope in one column, so the sheet's VaR is the page's
- **Export 100, 500, 1,000, 10,000 or All,** sized before the download; positions as rows beyond 16,384 columns
- **Every confidence and ES,** components, incremental VaR, both 10-day methods

Key insight **The workbook is built from the engine's own vectors, read as the person, and proved equal in every drill.**

Every view explains itself, in five layers



- **What you are looking at**
  From the pack's own help text and glossary.
- **Where the data came from**
  Source, health, generation and freshness.
- **Why it looks like this**
  Which Sutra matched, which lost, what inference added.
- **Masked for you**
  Says which fields your role cannot see.

One contract per command, and everything else is generated from it

config/packs/<pack>/contracts/<kind>.contract.yaml

```
kind · mnemonic · grain · key
columns: name, type, unit,
  meaning, source, example
delivery · validation
entitlements · retention
kind of data: fundamental,
  engine input, own, computed
```

drishti.py contract list, show, check, docs, bootstrap; kept current by tests

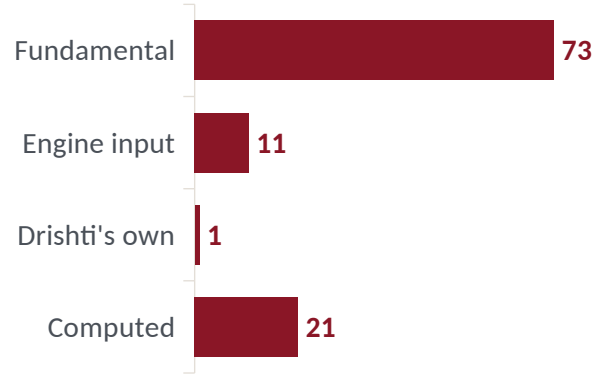
Generated pages docs/contracts and the COMMAND_CATALOG: every command, what it does, an example

Every pack's guide ends with its contracts and where each kind's data comes from

DATAMAP joins between kinds drawn from the contracts and checked against the data

FUNC Views → Functions: every command you may use, by pack; FUNC <words> filters

The 106 commands, by kind of data



A computed figure is never stored in a fundamental or input table: a guard test fails the build.

Admin → Functions
per command

Switch off
any command, for everyone, with a reason

→

Stored and gossiped
the Drishti database, every server

→

'Not active' everywhere
page, API, CLI, search, exports, BI, AI: 403 DRS-1016

Key insight **A command's meaning lives in one file; the catalog, the guides, the map and the switches read it.**

Official results other systems can read — and edits that never overwrite

Official end-of-day results: VaR and exposure

Morning full run
every node of every hierarchy, as the server

→

Captured, append-only
a restatement adds a result version; kept 7 years

→

risk-results
var-official, exposure-official, -cpty: Delta or Iceberg, a folder or S3; JSONL beside

→

BI reads them
read-only kinds VOFF and XOFC; the Market risk VaR model and report

Write-back: forms save through an edit log, with four eyes (off by default)

Submit...
the draft and a review sheet: what you saw, what is there now, your change

→

Approve
a change request; the approver is never you

→

Edit log
appended to its record's lane, acknowledged after commit; 64 lanes, epoch-fenced

→

Apply
overlay, owned value or the source through an outbox; rights re-checked

→

Recompute
reactive recomputes and live updates after each change

Conflicts are named, never overwritten: a change whose fields moved since you saw them is stopped with the three-way difference in words; DRS-9301 to 9306; Review → Changes; tutorial 8.

Key insight **Engines never read the published results (a guard test): the record is for reporting, the lake stays the input.**

RECONCILED AND GUARDED

55 of 76 netting sets agree at once; the other 21 are explained to the dollar

55 / 76
netting sets: derived net MtM equals the stored, at once

827
live-status trades past maturity in the other 21 (MtM 16.7m)

76 / 76
agree to the dollar with those trades added back

112,041,646
VAR BANK, USD, 2026-09-30: unchanged

Retired field	Why	Where it lives
`netting-set.UTILISATION`	a stored ratio the page can compute	dropped; BI's card shows trades, largest net MtM
`risk-hierarchy.LIMIT_VAR`	VaR limits kept in two places	only in var-limit (same values)
`portfolio.LIMIT_VAR`	the same	var-limit, hierarchy PORTFOLIO

The guard

`tools/contracts.policy.yaml` removed: fails the build if a retired field returns.

No VaR figure moved

Trades, valuations, vectors, scenario PVs and paths are untouched by the computed attributes.

Retiring the stored netting-set and counterparty figures (NET_MTM, TRADES[], COLLATERAL, LIMIT) with the netting-set engine.

Key insight Each difference has a named cause; a retired field cannot come back without failing the build.

A faster path must give today's answers, and prove it at start-up

Path	Rule	Why this rule
Today's code (the baseline)	bit-exact, run to run	the reference every other path is measured against
SIMD sums (the Vector API)	bit-exact against a plain-Java twin that adds in the same tile and lane order	same order, same bits; the baseline moves to that order once, measured
BLAS (OpenBLAS)	each scenario total within a relative 1e-12; the same scenario selected as the k-th worst	on a total of 100 million, a ten-thousandth of a cent; VaR's date and every component unchanged
Any breach	the path switches off, back to today's code, with the reason in Admin → Health → Native	found by the start-up canary or the release drill's parity suite

Why sums can differ at all

floating-point addition is not associative; any change of order can move the last bits

Tiles are more accurate

pairwise or tiled sums err about log2(n) × 1.1e-16; one by one, up to n × 1.1e-16

The selected scenario matters most

a near tie is broken by the lower scenario index on both paths

Key insight Speed is never bought with a different number: a path that cannot prove parity does not run.

Nothing unbuilt looks built: one registry, one mechanism

One registry

config/under-construction.yaml lists everything that depends on unbuilt work: what it is, where it shows, and whether it is hidden or marked.

The server rewrites a page's Sutra when it matches, so pages, phone lines, pivots and the client renderer read the same answer; the console reads the same file for menus and Help.

What a person sees

“Under construction” in place of the figure, or nothing at all. The data stays in the lake; only what is displayed changes.

Checked

A generated document (UNDER_CONSTRUCTION.md), checked by a test on both sides: every place it names exists.

Registered today

SA-CCR figures
EAD, replacement cost, add-ons, multiplier, RWA: stored, not computed

CVA figures
CVA, DVA, FVA, KVA, CS01 and their parts: stored, not computed

Credit-limit usage
the used amount and utilisation as stored, not measured by the exposure engine

Counterparty net MTM, stored
the computed one, from the trade valuations, stays beside it

Pre-deal: SA-CCR and EAD limits
SA-CCR's incremental EAD, soft limits, the maximum size (the credit-limit decision is built)

P&L explain waterfall
the stored steps; the computed explain is built

The CCR cube
a Pivot preset, until the counterparty engines compute its figures

Wrong-way risk, collateral, margin calls
their contracts, planned

Arrow Flight on three nodes, end to end
not measured yet; single-node reads and streams are run

Modules, lenses and access

10



The core knows no industry: each industry is a module

Licence which modules a customer may run → **Module** installed, licensed and enabled → **Lens** the experience a person works in → **Function** a command word, the finest unit of access → **Permission** roles, row rules and masks inside

Module	What it brings
core	connectors, lakes, Sutras, search, the Pivot, BI, tables and models, approvals, alerts, Explain, Admin; GLEIF's legal entities
banking	VaR and ES, exposure and the pre-deal check, sensitivities, FRTB-SA, SIMM, official results; drishti-risk
capital markets	US equities (EDGAR filings and facts, indices), funds and ETFs, the market data's functions
economy	economies, forecasts, governments, elections, news, and the public sector

Off is gone

A module that is off loads no engine, job, pack or API; its pages, menus, commands, type-ahead, Help pages and Build items are gone for everyone. Switched in config/modules/<module>.yaml or the admin API, audited.

One trust point

Every layer asks the server, which asks the registry: the console and the browser never decide on their own.

Nothing changes on upgrade

Without an installed licence every module is on, so an existing installation keeps everything it had.

From JSON Lines files to a deployed module, in one workspace

1 Data
drop .jsonl files; every record is read on the server, none sampled

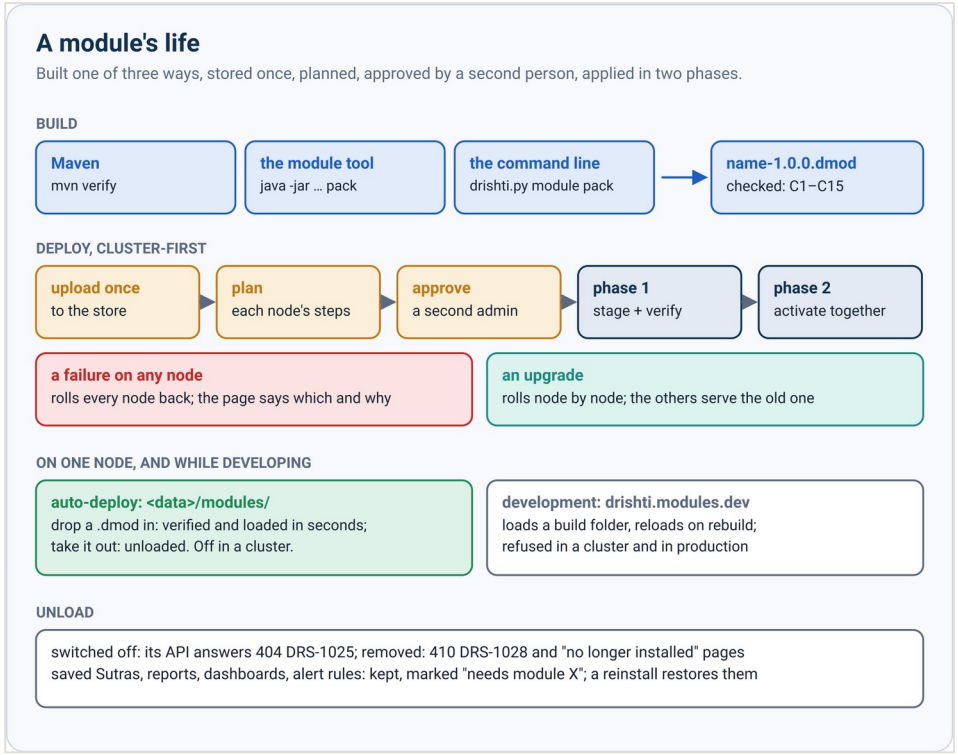
2 Kind and keys
discriminators give one screen per combination; the ID is checked across every file

3 Source
name, version, connector and one data binding per kind

4 Design
each Sutra's YAML beside its live screen on real rows

5 Finalize
the pack linted and tested, packed as a .dmod, fifteen checks

6 Deploy
one module deploy: planned, approved by a second administrator, on every placed node



- Build > New pack at /build/pack/designer, with the same engine as the command line (drishti.py pack new)
- three ways to build a module, one packer: Maven archetype, the standalone module tool, or Python and Java only
- a failure rolls every node back; an upgrade goes node by node; on one node it installs at once
- a second administrator approves every deploy: audited, no restart

The Pack designer reads every record, then builds a kind or maps a source

Whole-file analysis, nothing sampled

- drop .jsonl, .jsonl.gz, .ndjson or a JSON array: every record of every file is read on the server, with progress and cancel
- discriminators in any combination give one Sutra each; the ID is checked across every whole file
- measured: 66 MB (5,000 deals) in under a second; 2 GB (500,000 rows) in about 20 s

Anonymise samples

- on by default for data files: names and ids replaced the same way everywhere, amounts scaled
- dates and short vocabularies kept, so the design is the same
- turn it off only for data you may share

Mode 1: a new kind

- kind and keys, child tables for arrays, one Sutra per combination
- name, connector, data binding; each Sutra's YAML beside its live screen
- finalize: linted and tested, packed as a .dmod; deploy approved by a second administrator

Mode 2: a source for an existing kind

- pack new --source-for KIND --system SYSTEM: a source module from a sample file
- each native field is scored against the kind's contract: name, type, values, keys; confidence and reasons in words
- required fields left unmapped and unused native fields are listed; the result is the connector's map file

Same engine in the console (Create > New pack) and on the command line (drishti.py pack new).

SOURCE SYSTEMS

One module per booking system, one canonical trade

source-endur	source-calypso	source-murex	source-wss	source-swapone	source-ici
own table; canonical schema	own table; canonical schema	own table; canonical schema	treasury, FX, money markets	OTC swaps	generic cross-asset

▼ each maps its native fields (UPPER_SNAKE names) to the canonical ones ▼

One canonical trade kind: every row carries SOURCE_SYSTEM

What a source module ships

- connectors: read from Delta, Iceberg and files; a Kafka live stream whose writes: is the write-back
- a field map from the system's native names to the canonical ones
- inbox routing rules and a short guide
- the module requires the trading module and switches on and off alone

Why this shape

- ownership and operations follow the bank's real systems
- risk stays cross-system: one trade kind, one grammar
- a new system is a new module, never a split of the trade kind
- the data of WSS, SwapOne and ICI is mocked, with no positions: the VaR does not move

Endur, Calypso, Murex, WSS, SwapOne and ICI are the systems the modules are written for; the allowed SOURCE_SYSTEM values are the installed source modules, never a fixed list.

Wiring a source: where, set, map, row rules, and an off switch

Option	Meaning
lake-tables	the table read: the system's own, or one several systems share
where	a predicate selecting this system's rows of a shared table (pushed down to the store)
set	constants stamped into the rows read and into the rows the inbox stores (SOURCE_SYSTEM)
map	the native-to-canonical field map: dotted paths, first-of, scale, cast, as, values, value

Shared or separate

- shared table: one connector per system, each with its own where; the binding lists them with union: true
- separate tables: one connector per table with set: SOURCE_SYSTEM
- several live connectors per kind

Row rules

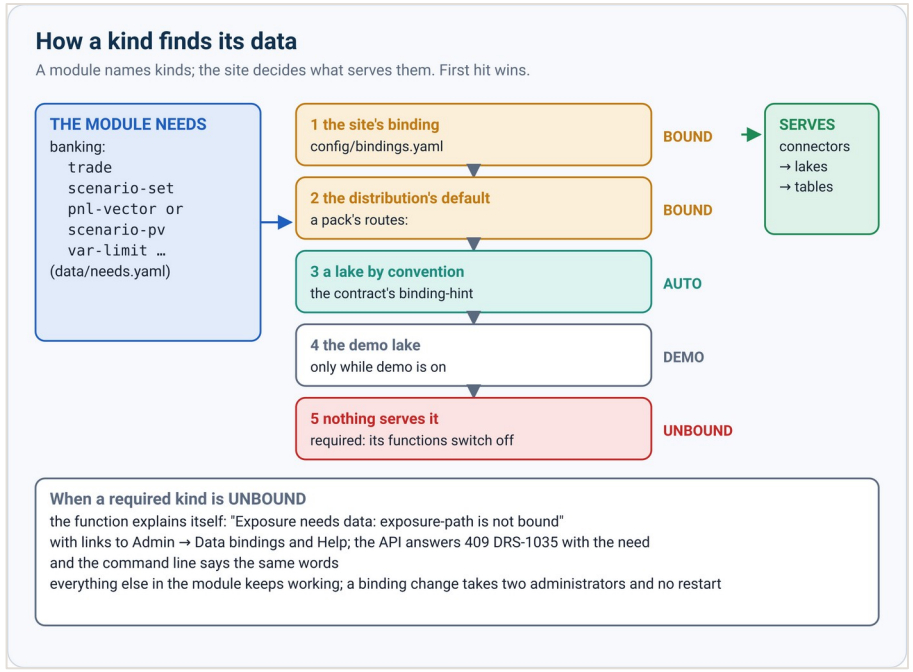
- who sees which rows is a role's row rule on the canonical kind, by person attributes: SOURCE_SYSTEM, DESK, ASSET_CLASS
- applied per request: lists, blotters, search, and each live stream
- edit rights follow the record's source

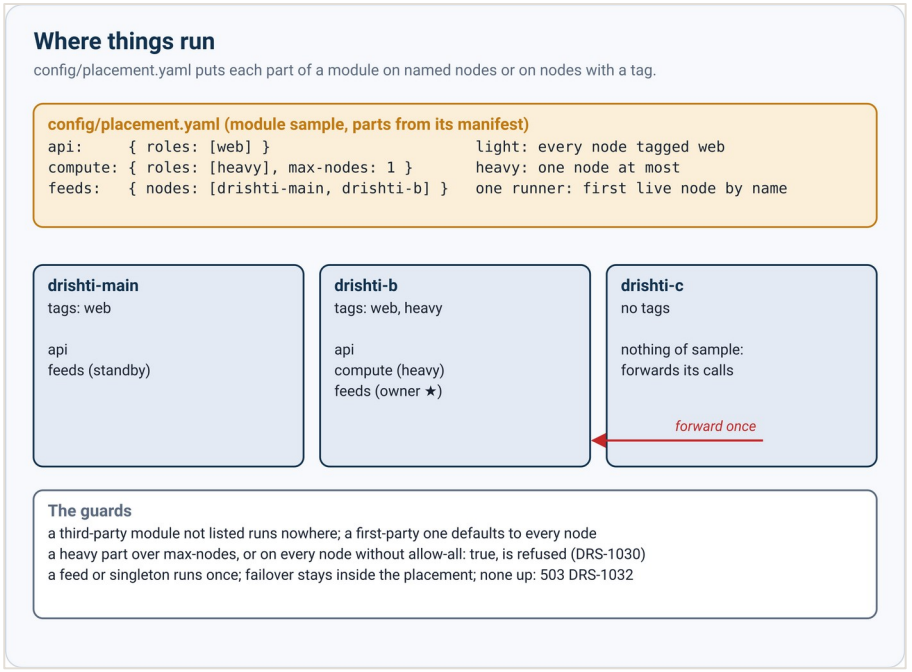
Switch a source off, VaR untouched

- Admin > Modules, at runtime, no restart: its trades leave lists, blotters, search and BI; its live feed stops
- pages of the kind say 'excludes ENDUR (switched off)'; switched on, all comes back
- VaR is driven by scenario PVs at position grain: a source switch leaves it unchanged

Order of work on a row: where on the stored row, then map, then set. Raw JSON shows the stored record.

Bindings say where data comes from; placement says where it runs





Bindings (config/bindings.yaml)

- one binding per kind: connectors to read, union, live source, where a produced kind is written
- first hit wins: site binding, pack default, lake by convention, demo, else unbound
- an unbound kind switches off only its functions, and they say why

Placement (config/placement.yaml)

- each module part runs by node name or tag: light, heavy, feed, singleton
- a singleton has one owner and fails over inside the placement; heavy work is capped
- a node that does not hold a part forwards the call once, for the same person

A lens is the view a person works in, and only that

Module	Lenses	For
banking	Risk · Trading · Market · CRO · Market-risk cube · Counterparty-risk cube	risk managers, traders, the CRO's desk, the two risk cubes
economy	World · Public sector	economists, public-sector analysts
core	CIO · Oversight	the technology owner, audit and oversight

A file per lens

config/lenses/<module>/<lens>.yaml: the modules it needs, the functions it bundles, its menus, Home cards and chips. Offered only when every module it needs is on.

Functions, one by one

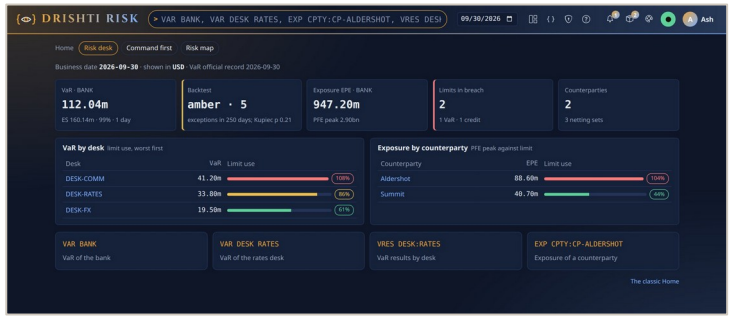
Each command word belongs to one module; roles are granted or denied single functions on top of a lens. Typed anyway, a refused function answers in words, and the refusal is audited.

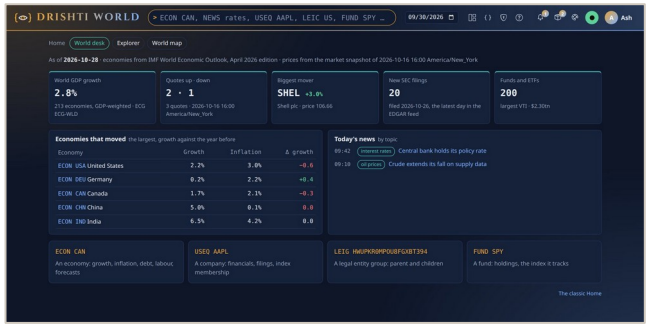
Editions are lenses

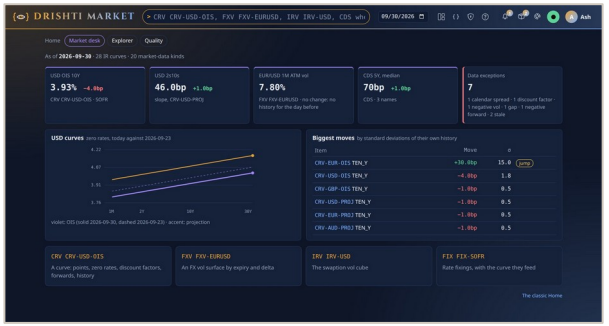
drishti.edition still runs a whole instance as one lens (for example risk); drishti.lenses offers several. A site's own edition file moves to config/lenses/<module>/.

Why it matters. Each person sees the work they do, and only what they may use: a trader's lens has no CRO board, a public-sector analyst never meets a VaR limit. Each person's modules, lenses and refused functions are one API call (GET /api/v1/modules).

Drishti Risk, World and Market, each with three landings of its own







Drishti Risk

VaR and exposure first

5 packs: core, market-data, trading, counterparty-risk, market-risk; 36 commands

Landings **desk · command · map**

Drishti World

Economies, markets, companies and funds

5 public-data packs: world-economy, news-and-markets, us-equities, gleif, funds; 32 commands

Landings **desk · explorer · map**

Drishti Market

The bank's curves, surfaces and fixings

market-data and core; 22 commands; no trades, no VaR; quality checks

Landings **desk · explorer · quality**

Role hooks

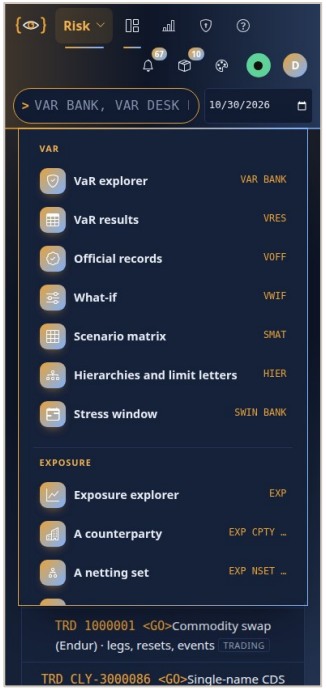
`home.landing-by-role` is read and wins over a person's choice, e.g. risk-manager: map, trader: command. `lens.roles` (who sees a lens) is documented, not wired.

Key insight **Three products from one engine: each new edition was one file and its landings, no engine change.**

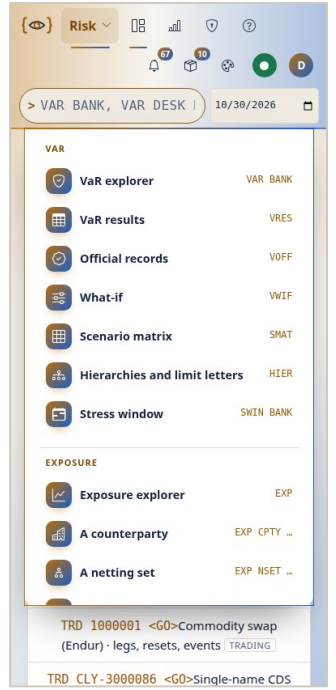
Each lens with its own subject menus in the bar

Lens	For	Subject menus in the bar
Drishti Risk	risk managers near the desks	VaR · Exposure
Drishti Trading	traders and desk heads	Blotter · Positions · Risk · Counterparties
Drishti Market	the market-data team	Curves · Volatility · FX · Credit
Drishti World	macro, research and context	Economies · Markets · Companies

- **Configured, not coded:** menus.top in the lens file names the menus and their order
- **Every item is a command:** an item appears only when its command is on for the lens, so the menus hold nothing the command line does not
- **On a phone:** the subject menus fold into one menu named after the lens



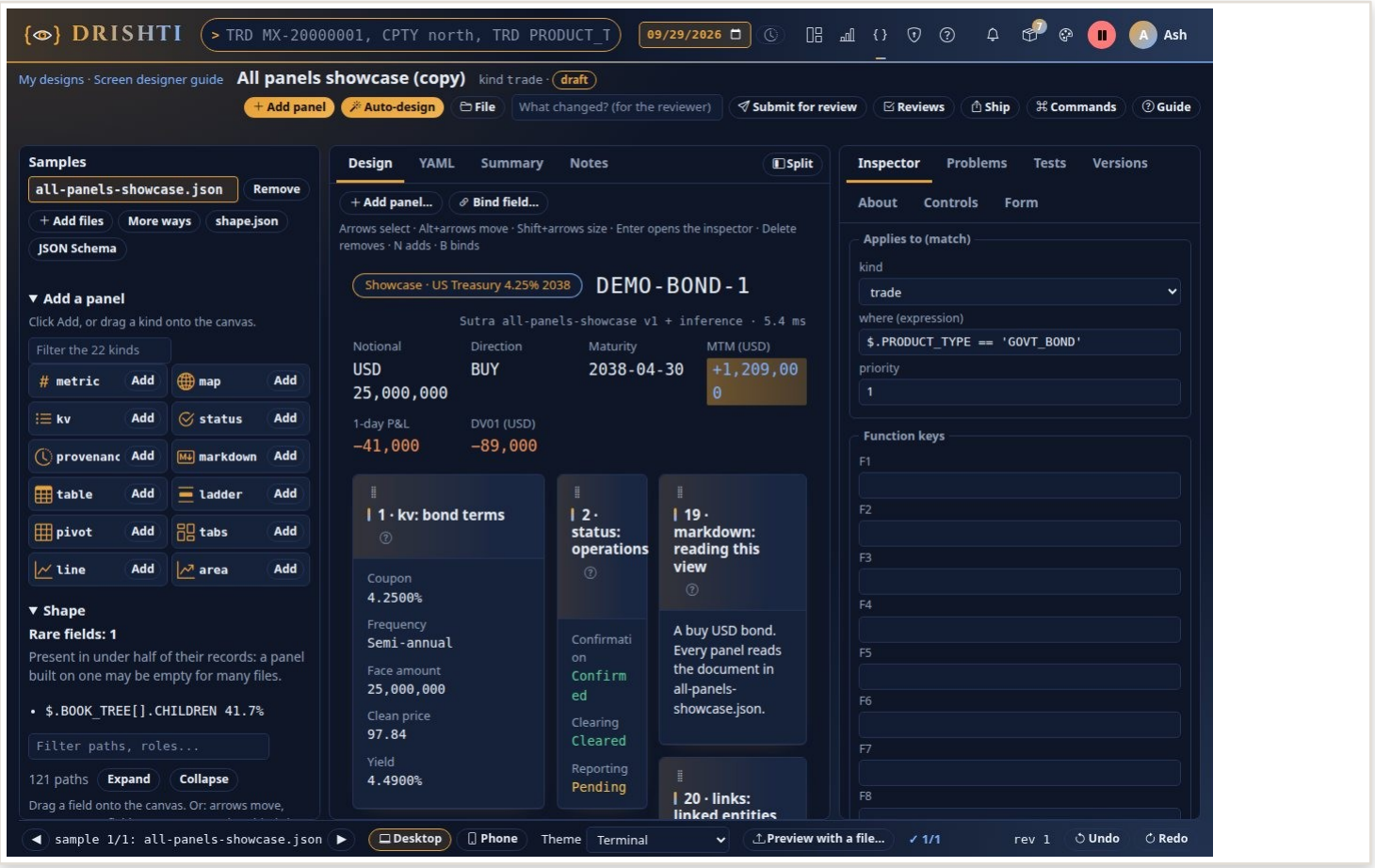
Risk: VaR, Exposure



The same menu, light theme

Key insight A lens hides; roles, entitlements and masks still decide what anyone may read.

Design a screen without writing code — and ship it through review



- **From data you have**
  Start from JSON files, a schema or stored entities.
- **Drag, or type**
  Palette and inspector, or the YAML tab with completion.
- **Tested before shipped**
  Sutra tests run on sample data in the tests tab.
- **Four eyes**
  An approver reviews the evidence and cannot approve their own.

Screens that respond, built from one UI kit

Events and children

A click on a row opens linked records under it, beside, below or in tabs, from any enabled pack (emits, @panel.event, @parent); masked ids stay hidden.

Every pack screen is built from fragments, each proven to draw the same as the screen it replaced.

Refresh and freshness

refresh: per panel or screen; pause, resume and interval per person; every panel says 'updated N s ago' and names a source that is not answering.

The Drishti UI kit

config/ui-kit: 17 reusable fragments (row-picker@1, field-list@1, cashflow-ladder@1 …), pack fragments, theme tokens and brand; every fragment and Sutra has a doc: docstring

Layout

Named areas on a 12-column grid, splits, tabs, sections, a sticky header, phone order; user-can-arrange per reader.

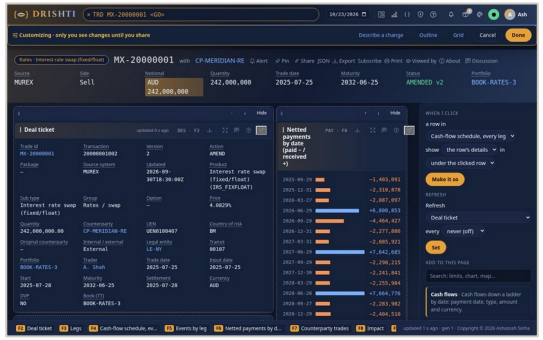
On a cluster, one server per screen; when a server goes away the page says 'your selections were reset' in words.

Forms

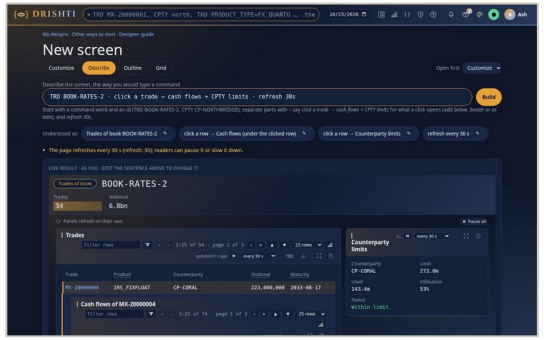
Edit beside a table, in its cells, or New in a guided drawer; linked-record fields, rules and masks; saved through write-back.

Key insight A screen is a paragraph of documented, reusable fragments that respond to the user.

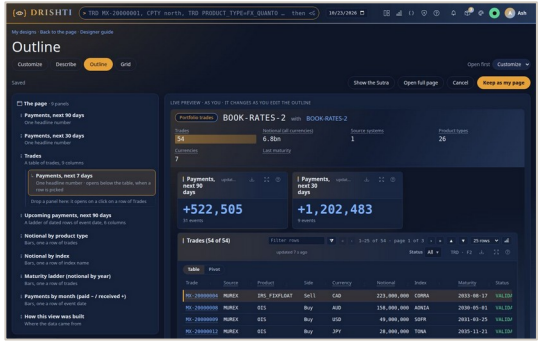
Four modes over one Sutra: customize, describe, outline, grid



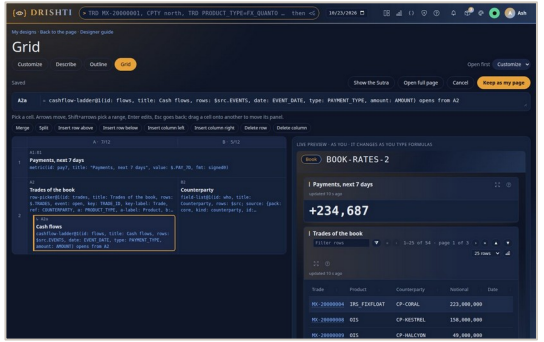
1 Customize the live page



2 Describe a screen in one line



3 Outline: the page as a tree



4 Grid, with formulas per cell

A cell is a fragment call

```
B2 = field-list@1(...;
     source: {... id: @A2.open.ref})
```

- **One Sutra, four views**
  Every mode edits the same file; switch at any time.
- **My version of this page**
  Each person keeps a customised copy; the shared page is untouched.
- **Two levels of click-opened cells; off by default**
  ui.designer; tutorial 19 builds a screen with it.

Key insight Designing a screen in plain words still produces a reviewed, versioned Sutra — not a one-off.

The product's name and the bank's names come from configuration

Branding

`ui.product · drishti.branding.product`

`ui.owner · ui.owner_email`

No template, script or server message writes the product's name; features follow it: <product> SQL, <product> Elements, <product>*net. The landing headline and tagline too.

Guard `test_branding_guard.py · BrandingLiteralTest`

The bank's names

`config/demo/names.yaml`

`lake demo --names FILE --dry-run`

The bank, its short name, the legal entities LE-NY, LE-LDN, LE-FRA, LE-TKY and branches, renamed as the demo loads, in every kind. Defaults change nothing; LEIs and BICs stay; committed test data is never rewritten.

Guard `tools/lake/test_names.py, in the drill`

The VaR tree and method

`DRISHTI_VAR_ROOT_NAME`

`scenario-window · confidence · currencies`

The top of every VaR tree named by a setting (per hierarchy too); the window, confidence and reporting currencies are settings, and every answer carries its labels.

Guard `VarRootNameTest`

Key insight A bank sees its own names in a demo by editing one file — and no code names the product or the bank.

Case studies: Drishti at work

11

Ten people, ten jobs, one platform

	Case	Who	What they open
1	Market risk: VaR from the bank to a trade	a market risk manager at month-end	VAR, SMAT, PLAY VAR
2	Regulatory VaR reporting	the reporting team and the CRO	VBT, VAR STATS, RPT
3	The Risk lens	risk managers near the desks	the Risk desk landing
4	The Trading lens	traders and desk heads	the Trader desk, VWIF, XWIF
5	The Market lens	the market-data team	CRV, FXV, the Quality landing
6	The World lens	economists and credit analysts	ECON, LEI, FUND, USEQ
7	Counterparty exposure	a counterparty credit officer	EXP, NSET, XWIF
8	BI and analytics	a business analyst	tables, a model, a report, a board
9	Data and operations	a data engineer	lake build, data landed, DATA, DATAMAP
10	Oversight (breaches designed)	the second line, for the CRO	VAR EXPLAIN; breaches

Key insight Every figure in these cases is the demo data's, quoted from the guides and tutorials.

Month-end: is the bank's VaR right, and who makes it?

Who they are

Business-aligned market risk manager
answers for the VaR of the businesses they cover: right, explained, within limits

Model validator and auditor
must check how a figure was worked out, independently of the engine

What they need

- the bank's VaR and ES on the official date, with limit use
- the desk that moves it, down to the trade
- what-if cuts: the bank without a desk, a desk without a hedge
- proof of a figure a validator can recompute

Their day

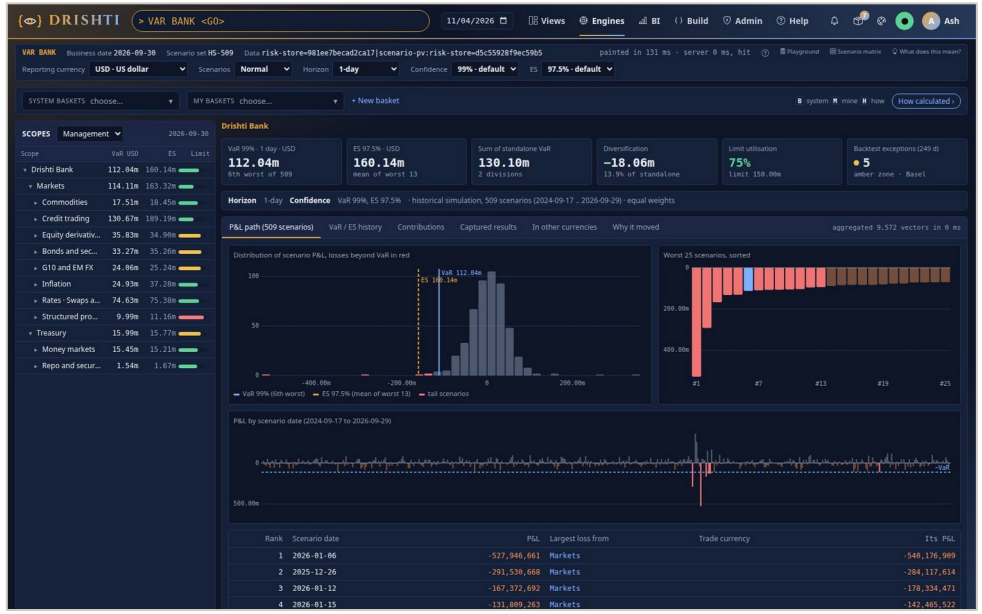
07:00	the official run: VaR by desk, limit use, breaches
09:00	the morning meeting: why it moved
Intraday	a scope someone asks about; a hedge to test
Month-end	backtest, limit review, stressed VaR
On request	the validator's workbook

What was painful before

- finished VaR figures stored by a batch, which nobody could re-derive
- a new cut of the bank meant a new hierarchy load, or a spreadsheet
- explaining a figure took a day of extracts and formulas

Key insight The manager asks in sentences; the engine answers from the trades, and shows its working.

From VAR BANK to the trade, on 2026-09-30



VAR BANK: the explorer on the bank, its hierarchy and its tabs

The screens and commands, in order

VAR BANK
VaR 99% 112,041,646 USD, ES 97.5% 160,143,809; limit 150m, use 74.7%; 9,572 trades

drill: LE-NY › Credit trading
144,262,258 › 130,670,864, each node with its limit use

VAR BANK - DESK:CREDIT
76,605,857: the bank without a desk, as one scope

VAR DESK RATES · parts by book
components 30,612,282 + 25,280,825 + 18,740,573 = 74,633,680

SMAT DESK RATES · PLAY VAR DESK RATES
scenario by scenario, then the workbook

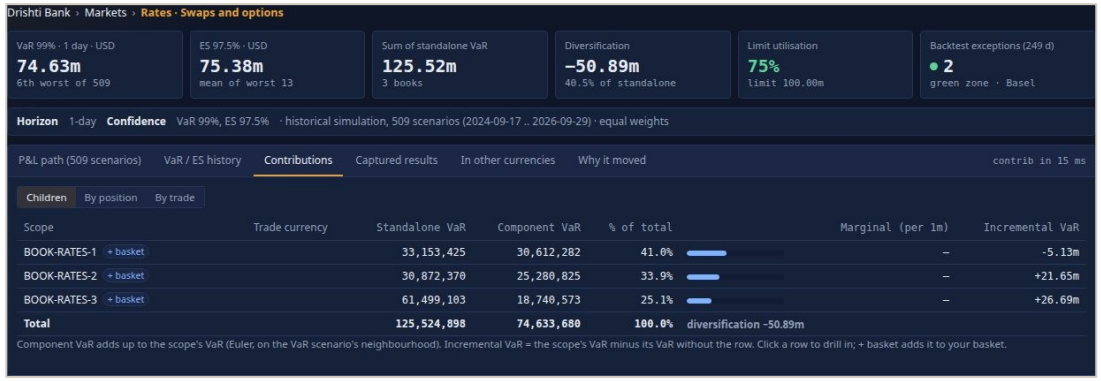
Computed and explained

the P&L path: VaR at 99% is the 6th worst of 509 days (509 × 1% = 5.09, rounded up); ES at 97.5% the mean of the 13 worst.

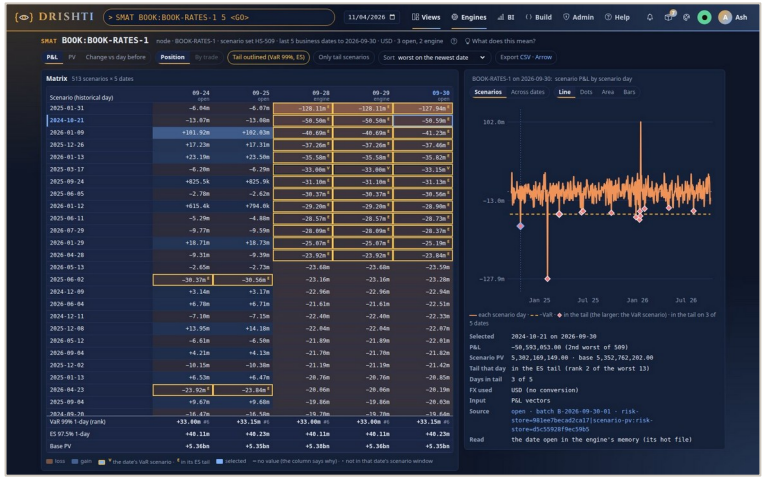
The result

- diversification across divisions 18,060,250
- the 6th worst of 509 scenario days, to the dollar
- every component sums to the desk's VaR exactly

Who makes the desk's VaR, scenario by scenario, and in a workbook



Contributions: the desk by book, adding up



SMAT: the desk scenario by scenario

DESK:RATES by book	Standalone VaR	Component VaR	% of VaR	Incremental VaR
BOOK-RATES-1	33,153,425	30,612,282	41.0	−5,131,638
BOOK-RATES-2	30,872,370	25,280,825	33.9	21,654,002
BOOK-RATES-3	61,499,103	18,740,573	25.1	26,690,191

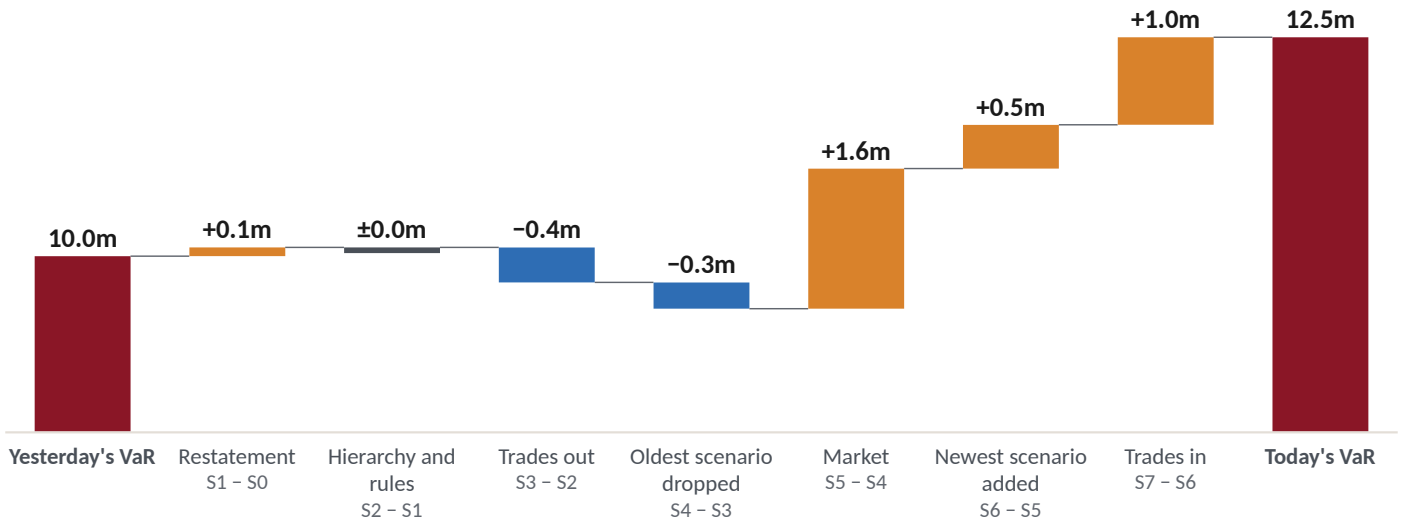
- the components add up to 74,633,680 to the dollar
- BOOK-RATES-1 lowers the desk's VaR: a hedge
- PLAY VAR DESK RATES exports the same calculation as formulas

Key insight A validator recomputes the desk's VaR on their own machine and gets the engine's figure to the cent.

Why VaR moved: one change at a time

The method works by moving one thing at a time from yesterday's state to today's and recomputing VaR at each stage, which is why the steps always add up exactly.

From yesterday's VaR to today's, a step for each cause (illustrative figures)[1]



A rise in VaR orange, a fall blue; the value axis starts at 8.0m. The steps add up to +2.5m exactly.

The states: each one a full VaR

State	What changes from the one before
S0	yesterday, as officially captured
S1	yesterday's data as it is now (restated)
S2	today's hierarchy and rules
S3	only the trades still live today
S4	only the scenario days common to both windows
S5	today's market: today's vectors
S6	today's scenario set: the newest days added
S7	today's trades: today's VaR

A step is one state's VaR less the one before: S1 − S0 … S7 − S6. Their sum is S7 − S0: today's VaR less yesterday's.

One answer, three views: Waterfall · Matrix (every node's bridge) · Sentences (the answer in words). **Download the working (.xlsx)** to check every state by hand. Each step names its drivers: trades by incremental VaR, desks by component VaR, the scenario days that left or entered. **`VAR EXPLAIN <scope> <GO>`**

[1] Illustrative figures, invented to show the shape, not from the demo data (VAR_GUIDE.md section 3.11)

The fiscal year closes: the VaR table, the backtest, three signatures

Who they are

Regulatory reporting team
prepares the annual report's VaR table and the backtesting chart for the period

The checker and the CRO
review, send back or approve; one person a step, four eyes

What they need

- period end, average, high and low of VaR for every row
- the backtest's exceptions, zone and multiplier
- every cell traceable to the record it came from
- a signed, frozen run; a restatement as a new run

Their day

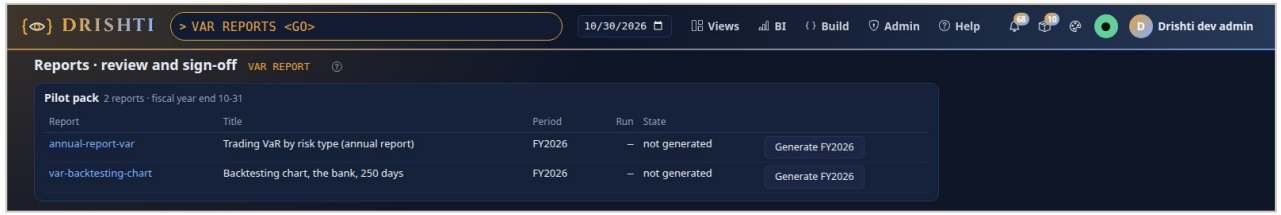
Daily	official records captured at the close
Period end	the run generated from the records
Checks	totals, rows, missing days, the backtest
Sign-off	maker, checker, then the CRO
Filing	xlsx, csv and PDF to the reporting team

What was painful before

- figures re-keyed from risk systems into spreadsheets each quarter
- no lineage from a filed cell back to the day and the data version
- a restated input silently changed an earlier figure

Key insight Drishti fills, checks and freezes; people write the narrative, approve and file.

From the official records to a signed run



VAR REPORTS on the live demo stack: the pilot pack before the period's run



VAR BANK STATS FY2026: three official records so far, a preview

The screens and commands, in order

`VBT BANK`
the backtest: 250 days of the official VaR of the day before against the desks' actual and hypothetical P&L

`VAR BANK STATS FY2026`
period end, average, high and low from the official records; missing days named

`Generate FY2026`
every cell filled from the records, the checks run, the run frozen with each cell's lineage

`VAR REPORT annual-report-var FY2026`
the run on screen; maker, checker, approver sign in turn; files released

Computed and explained
the backtest from the records of the day before and the desks' P&L; statistics from the records, never stored beyond them.

The result

- seven checks: a failure blocks sign-off, never generation
- a restatement makes the next run, never an edit
- one YAML definition per report

Risk managers near the desks: VaR and exposure first

Who they are

Market risk manager
the VaR of the businesses they cover: right, explained, within limits

Counterparty credit risk officer
exposure against limits, pre-deal approval, collateral and netting

Risk analyst
risk reporting: the figures and the evidence behind them

What they need

- the bank's VaR and exposure and the limits in breach, first thing
- a pre-deal answer while the trader waits
- the official close, kept for years
- the stress window reviewed once a year

Their day

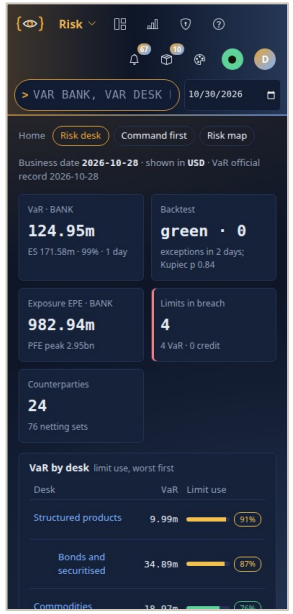
07:00–09:00	official run, VaR by desk, limit use, breaches
Intraday	VWIF and XWIF pre-deal, scopes, baskets
Close	official captures
Month-end	backtest; LIMIT_USE in BI; stressed VaR
Year	SWIN: the stress window review

What was painful before

- the morning figures came from an overnight batch, one fixed cut
- VaR and exposure lived in different systems with different trees
- a pre-deal check meant a request to another team

Key insight One lens, opened from a link: the risk manager's whole day without leaving it.

The Risk desk landing: numbers first, then who drives them



Drishti Risk's Home on a phone, 2026-10-28

The screens and commands, in order

VAR BANK · VAR DESK RATES
the explorer: any hierarchy, any scope, down to a trade

EXP BANK · EXP CPTY:CP-ALDERSHOT
EE, PFE and EPE from the paths, after netting and collateral

VWIF · XWIF
add or unwind trades before dealing: VaR and exposure

VRES · VOFF · HIER · SMAT
results, official records, hierarchies and limit letters, the scenario matrix

PLAY VAR · VBT · SWIN
the VaR menu's check and report items

Computed and explained

every tile is the engines' own answer for the date; a tile the engines do not serve is left out, never faked.

The result

- on 2026-10-28: VaR BANK 124.95m, ES 171.58m; EPE 982.94m; 24 counterparties, 76 netting sets
- landings: Risk desk, Command first, Risk map

The first line: my books, my limits, and does this trade fit?

Who they are

Trader
the P&L of their books, inside delegated limits; a fast pre-deal check

Desk head
the desk's P&L, limits and breaches, and each trader's limit use

Business manager
the desk's view across books and counterparties

What they need

- my books on one page, scoped to me
- VaR and limit use per book, near-limit warnings
- one answer for a new trade: fits, near or breaches
- today's blotter: new, amended, cancelled

Their day

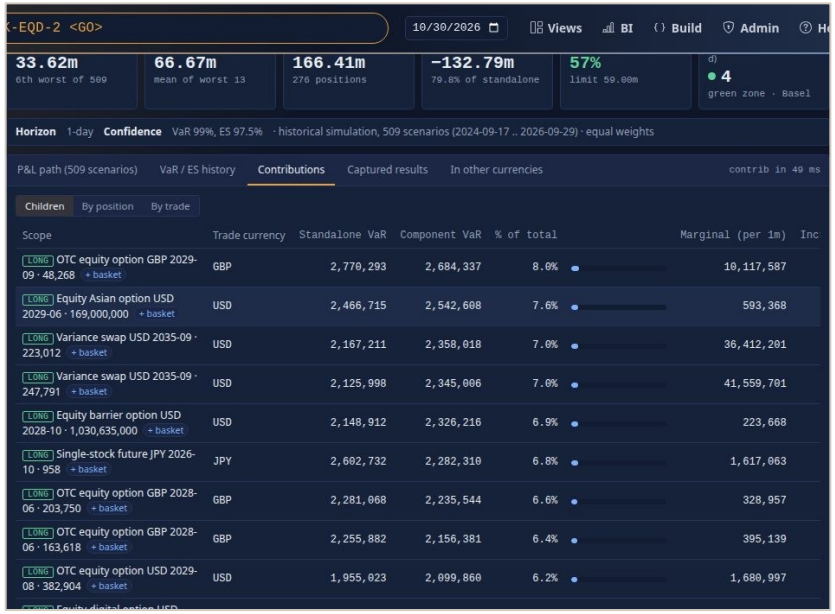
06:30–08:00	before the open: books, limits, overnight changes
Intraday	check a new trade; the blotter
16:00–19:00	the close: P&L against VaR
Month-end	limit use over the month
Year	the desk's limits renewed

What was painful before

- a pre-deal check meant asking risk, and waiting
- VaR and the counterparty limit came from two systems
- my books meant filtering a bank-wide report

Key insight **The trader asks one question, does it fit, and gets the VaR and the counterparty limit in one answer.**

A trader's book, and a hedge tested before dealing



A trader's book: its positions and their VaR, 2026-09-30

The screens and commands, in order

`TRD · CTRD · PTRD · LTRD`
the blotter: trades by counterparty, portfolio, legal entity

`BOOK · POS · VAR NODE MGMT~BOOK-RATES-1`
my book's positions, VaR, path and contributions

`VWIF DESK:RATES`
the VaR what-if of a hedge, before dealing

`XWIF`
the exposure pre-deal on the counterparty's paths

Computed and explained

Check a new trade runs one VaR what-if and one exposure pre-deal, and answers fits, near (0.8 of a limit) or breaches.

The result

- the hedge on DESK:RATES: VaR 74,633,680 → 47,599,220 (−27,034,460)
- ES 75,377,415 → 56,247,260; stressed VaR 85,318,588 → 59,556,766

The market-data team: are today's curves, surfaces and fixings right?

Who they are

Market data analyst
curves, surfaces and fixings in on time and clean

Valuation control (IPV)
independent checks of the prices behind valuations

Market risk methodology
the history behind VaR's 509 scenario days

What they need

- today's moves across curves, vol and FX at a glance
- stale data, jumps and gaps flagged, not found by accident
- one search for any curve, surface or fixing
- the impact of a bad point on VaR (designed)

Their day

17:00–20:00	the close and the batch window
Morning	exceptions before the desks open
Intraday	a curve someone questions
Month-end	IPV
Year	the history behind the scenarios

What was painful before

- checks lived in scripts and inboxes, not on the screen
- a jump was found after VaR moved, not before
- each data kind had its own viewer

Key insight **The data that feeds every valuation and every VaR gets a desk of its own, with its checks on it.**

Curves, surfaces and fixings, with their checks



CRV: a curve, live

The screens and commands, in order

`CRV CRV-USD-OIS`
a curve: points, zero rates, discount factors, forwards, history

`FXV EURUSD · IRV USD`
the FX vol surface; the swaption vol cube

`FIX SOFR · CDS · INFC · CMDC`
fixings, credit, inflation and commodity curves

`CORR · DATAMAP`
correlations computed from prices; the map of all data

`Quality landing`
stale, gaps and jumps by kind

Computed and explained

the checks are computed from the data each time the landing opens; VaR at stake and two-source checks are designed.

The result

- stale after 2 business days; biggest moves over 21 days in σ; a jump at 3σ or more
- on the committed demo data every check is clean

Context from public data: economies, markets, companies and funds

Who they are

Economist, macro strategist
growth, inflation, debt and forecasts, country by country

Country and credit analysts
sovereigns, companies, legal-entity groups, coverage

KYC and reference data
who owns whom: legal entities and their parents

What they need

- an economy, a company or a fund one command away
- linked: a company to its filings, its index, its legal entity
- groups and sectors computed from members
- licences respected: internal-only data marked

Their day

Morning	the world desk: indicators, news, filings
Intraday	a country or a company in question
Month-end	sectors and groups reviewed
April, October	the IMF's new vintages
Year	ten fiscal years of filings

What was painful before

- public data downloaded by hand into spreadsheets, then stale
- company, legal entity and fund data never linked
- group totals kept as copies that drifted

Key insight **Public data as packs: linked, offline, refreshed by script, and computed where it can be.**

One command line across economies, companies and funds



ECON: an economy from World Bank data

The screens and commands, in order

ECON CAN · MACRO · FCST
an economy: growth, inflation, debt, labour, forecasts

USEQ AAPL · FACT · FILING
a company: financials, filings, index membership

LEI · LEIG · LEIC
a legal entity, its ownership group, a country's entities

FUND SPY · HELD · TRACK
a fund's holdings and the index it tracks

NEWS · QUOTE
headlines and the offline market snapshot

Computed and explained

ownership groups, countries and sectors computed from their members when opened (LEIG, LEIC, SIC, ECG), never stored.

The result

- the 200 largest US funds and ETFs; ten fiscal years of XBRL
- World Bank data (CC BY 4.0); IMF and Yahoo data internal only

If this counterparty defaulted, how much would we lose?

Who they are

Counterparty credit risk officer
exposure against credit limits; pre-deal approval; collateral and netting

Credit committee
the largest counterparties and their netting sets

Their day

Morning	the bank's exposure board; limits
A request	a counterparty and its netting sets
Pre-deal	XWIF: the trade on the paths
Close	the official record

What they need

- exposure on an average path and on a bad one: EE, PFE, EPE, EEPE
- from the bank to one netting set and one trade
- the effect of collateral, path by path
- a pre-deal answer that is exact, not an add-on

What was painful before

- exposure stored per counterparty by an overnight run
- collateral applied as an add-on, not on each path
- pre-deal checks by a rough add-on table

Key insight Exposure is computed from the simulator's paths for any scope, with collateral on each path.

From the bank to a netting set, collateral path by path



A netting set's 200 paths and its collateral

The screens and commands, in order

`EXP BANK, 2026-09-30`
peak PFE 95% 2,897,620,439 at 2W; EPE 947,202,991; 74 netting sets, 23 counterparties

`EXP CPTY:CP-ALDERSHOT`
peak PFE 401,456,810; limit use 5.0%; four netting sets

`EXP NSET:NS-ALDERSHOT-LDN`
peak PFE 85,063,192 at 2M with collateral, 478,546,912 at 3Y without

`XWIF`
an unwind on the paths, exact: peak PFE −3,099,344, EPE −615,288

Computed and explained

the netting set's collateral applied on each of 200 paths; the trade's share of the set's EE shown; the official record kept.

The result

- 200 paths, a 24-date grid, 8,022 live trades
- CSA: 10-day margin period, 10m thresholds, 500k minimum transfer

A business analyst builds a report and a board, without a data team

Who they are

Business analyst
builds reports for their desk or function; knows Power BI

Report reader
opens the report or the board and sees only what they may

Their day

Start	pick a model, or a pack dataset
Build	cards, a map, a bar, a slicer
Publish	own workspace, shared, the catalogue
Run	subscriptions and alerts; a board in kiosk mode

What they need

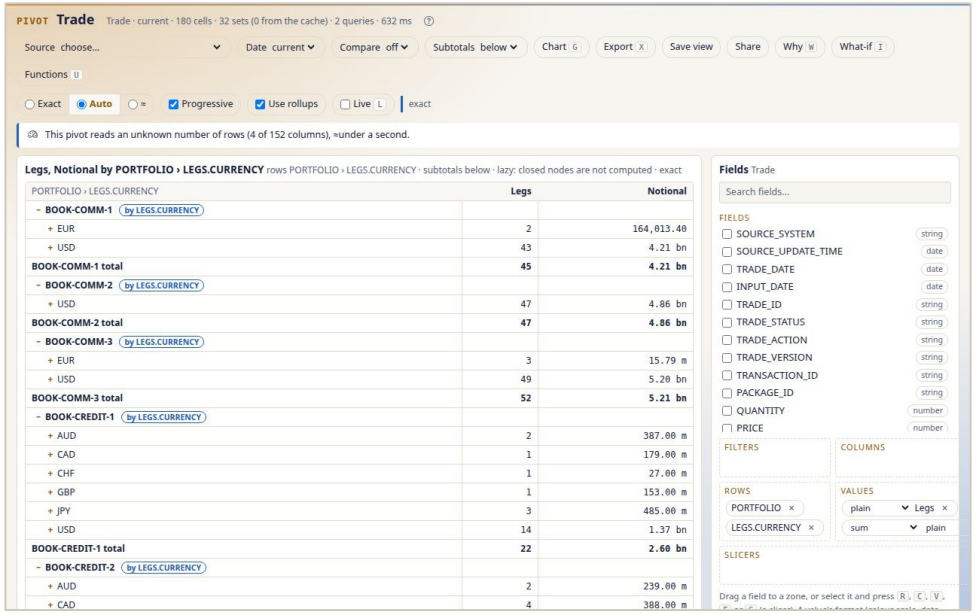
- a governed model to start from, not a raw table
- drag-and-drop visuals, slicers, drill-through
- two facts side by side, filtered by one slicer
- a live board on a wall; a subscription by email

What was painful before

- an extract requested from a data team, then a copy in a BI tool
- masks and row rules rebuilt in the BI tool, or forgotten
- the report and the terminal disagreed on the same figure

Key insight **The BI author starts from a model the packs already describe, under the reader's own masks.**

A model, a report, two facts in one model, published



A table of the layer in the Pivot: trade legs by book and currency

The screens and commands, in order

`BI <GO> · Models · World economics`
seven measures the model owns; GDP per capita a ratio of sums

`New report`
cards, a map drilled world › country › region, a bar with top 10, a slicer

`Two facts in one model`
trades and netting sets, each added up on its own, one slicer filters both

`Build › Tables & models`
a table from a pack, a computed column, a model, with its YAML in view

`Publish · Subscribe`
a workspace, the catalogue; PDF or Excel by email, each reader's masks

Computed and explained

every number by the server; the browser only draws; the reader's row rules and masks applied before any sum.

The result

- one slicer filters trades and netting sets together
- a join that would repeat rows is refused, not summed

A new feed lands every night: load it, announce it, prove it

Who they are

Data engineer
owns the nightly feeds from the pricing and trade systems into the lake

Operations
knows by 07:00 whether every expected load arrived

What they need

- one data standard for every feed: JSON Lines
- a loader into Delta, Iceberg or S3 Tables, one commit a date
- a signal that data has landed, and an alert when it has not
- a way to see and download what is there

Their day

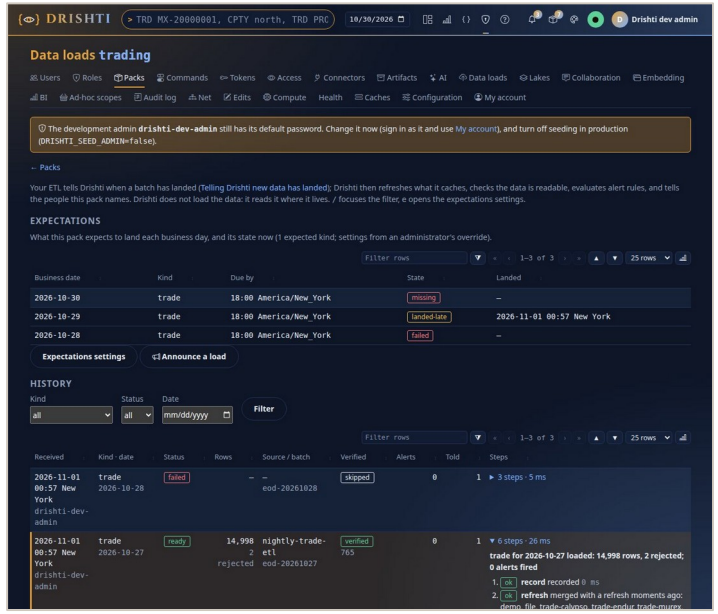
Night	the pricing systems write JSON Lines
Load	lake build; one commit per date
Signal	data landed: the pack acts on it
07:00	the loads page: expected, arrived, late
Any time	DATAMAP and Export data

What was painful before

- each feed in its own format, with its own loader script
- a missing day found by the first user of the morning
- no map of what data exists and how it joins

Key insight **One format in, one signal when it lands, and the platform shows what it holds.**

From a folder of JSON Lines to a dataset anyone can find



The Data loads page: every announced load, its batch and its state

The screens and commands, in order

`drishti.py lake build`
JSON Lines into Delta or Iceberg; --engine java for the parallel loader

`drishti.py data landed`
announce a load: ready or failed; the pack acts on it

`Admin → Data loads`
expected, arrived, late; an alert for a missing day

`DATAMAP · DATA <GO>`
what the packs hold and how it joins; any dataset downloaded

Computed and explained

every join on the data map is checked against the data; every download is streamed, masked and audited.

The result

- 10,000 positions × 510 scenarios loaded in 0.19 to 0.29 s
- the demo lakes: 60 business days, 85 tables, about 15 minutes

The second line: are we within appetite, and who approved what?

Who they are

Market risk oversight

the second line, reporting to the CRO: appetite, limits, independent challenge

The CRO's committee

escalations, temporary limit increases, the board pack

Their day

07:30–09:00	the morning meeting: why did VaR move?
Breach	explained, approved or remediated, closed
Escalation	desk, business, CRO
Committee	the board pack

What they need

- why VaR moved overnight, by cause, adding up exactly
- a breach as a record: status, commentary, approvals
- temporary limit increases with four eyes
- escalation from the desk to the business to the CRO

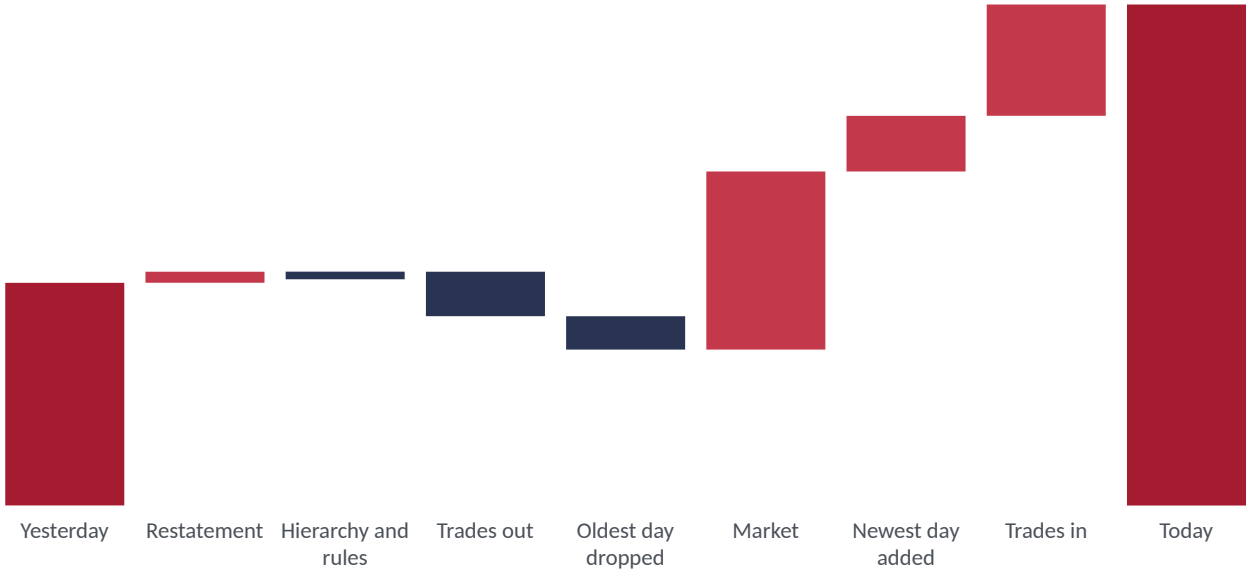
What was painful before

- the VaR change explained by hand, steps that never added up
- breaches tracked in email and spreadsheets
- approvals with no record of who and when

Key insight **Why did VaR move is the most-asked question of the morning meeting; the design answers it exactly.**

Why VaR moved, step by step; a breach from open to closed

RO1 · the VaR change: VAR EXPLAIN, in the product (case 1)



Illustrative: the guide's example of case 1 (10.0m to 12.5m). Each step changes one thing at a time, so the steps add up exactly; a Shapley view of the order is optional (at most 720 orders).

RO2 · a breach as a record

Open BR-<year>-<seq>, raised by the limit check

↓

Explained the desk's commentary

↓

Approved or remediated a temporary increase with four eyes, or the position cut

↓

Closed with its history

- escalated at any point: desk, business, CRO
- views: queue, board, timeline

Key insight **Designed on what is built: official records, captures, hierarchies and limits; all 131 nodes in well under a second.**

Data in

12



One JSON object a line: every generator writes it, every export offers it

Generators
every tool under tools/ writes JSONL; .jsonl.gz where the repository gzips

→

JSON Lines
one object a line, UTF-8; generating and loading are separate steps

→

Load
load_rows.py, lake build: the lake byte for byte the same as before

→

Delta or Iceberg
the lake every connector reads

Readers

- JSON Lines, JSON arrays and per-entity .json still read
- CSV deprecated: one warning a file, named in Admin → Health
- drishti.py convert FILE: CSV, a JSON array, gzip or a folder into JSONL

Exports

- Server: search, SQL, BI runs, /export/jsonl, statements, subscriptions, scheduled reports, VaR and exposure
- Console: every download and menu; CLI: --jsonl, --format jsonl
- CSV, Excel, PDF and PNG all stay

Records

- Official VaR and exposure results as JSONL beside their Delta tables
- Sutra test cases: one cases.jsonl per Sutra, 197 folders converted
- Sample catalogues, staged lake rows, seals: JSONL

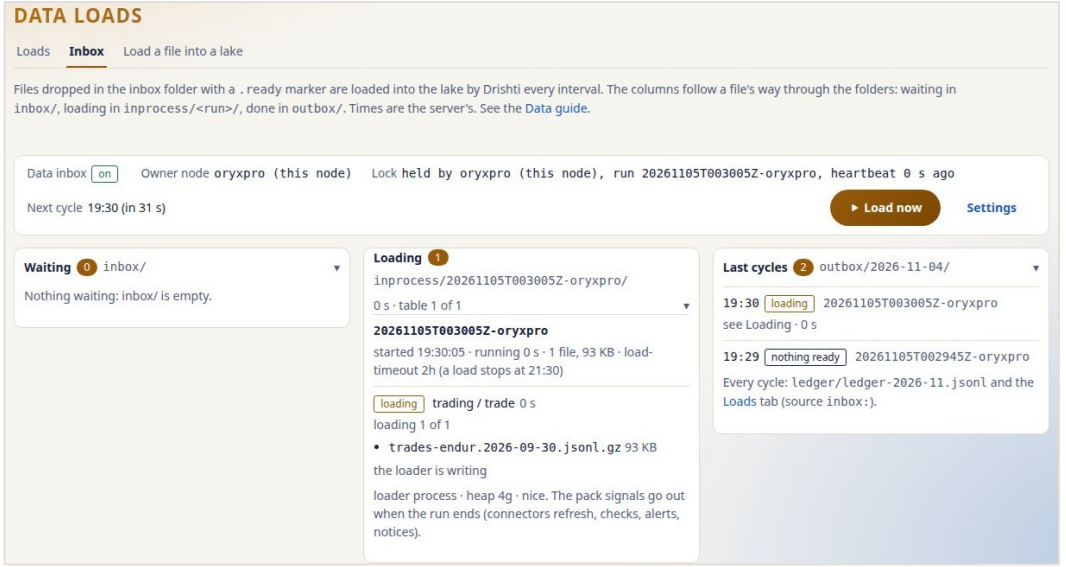
Guarded in the drill: `test_generators_write_jsonl.py` runs the generators and reads every data file; `test_exports_offer_jsonl.py` checks every export entry point.

Key insight A pipeline streams any Drishti input or output a line at a time; CSV, Excel, PDF and PNG stay for people.

Data in by clear rules: the file says its table and its date

Rule	As built
Table and date	from the name: <table>.<YYYY-MM-DD>.jsonl.gz, its .ready file, or the command line; never a row
Several at once	a tar of <table>/<date>/ folders: all of it loads, or none
Append by default	mode: replace replaces a date; every row carries its load id
Undo	drishti.py dataload undo <load id> removes one load, in its own lake
One file per lake	config/lakes/<lake>.yaml: partitions, load mode, retention, maintenance; lake check proves it
Everywhere the same	the Python and Java loaders, the inbox, every store loader, the demo and test data

Why it matters. A file loaded twice, a date guessed from a row or a half-loaded set are the usual sources of wrong numbers. Here the name decides, a set loads whole, and any load can be taken back.



Admin → Data loads → Inbox: a file waiting, loading, and the last cycles

The data inbox

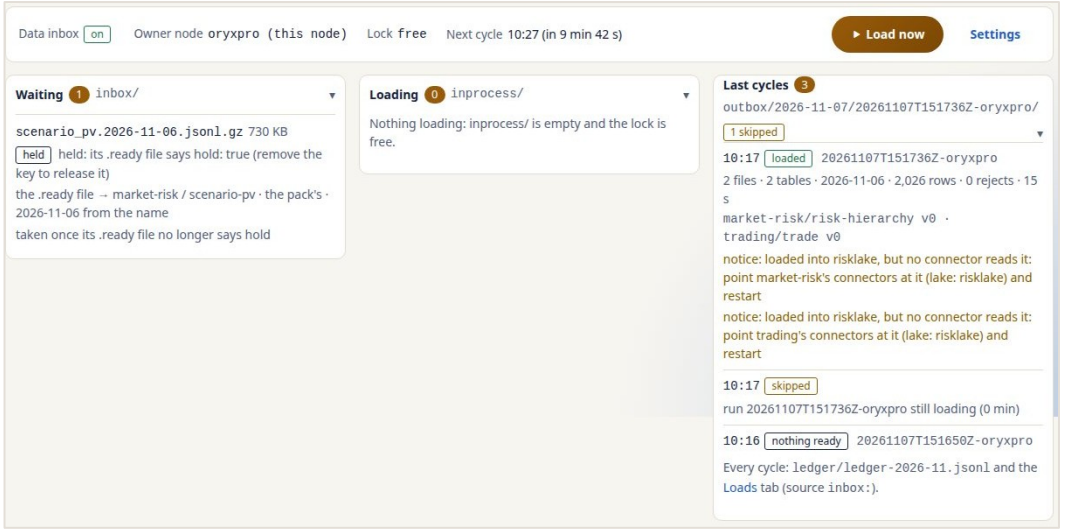
Drop files with a .ready marker: the Java loader loads them every 10 minutes, moves them to the outbox with a report and tells the cluster. Off as shipped.

The in-memory lake

Delta tables held only in memory, for feeds that by contract may never touch a disk: a hard cap, refilled by server jobs, proved by a marker scan. Off as shipped.

Every lake in its own folder: real data beside the demo, never over it

What	As built
A folder per lake	data/delta/<lake>/ by default; a lake file may name any folder, s3:// or mem://
The demo lake	data/delta/demo; an older demo lake moves there once at the first start, measured, rolled back on any difference, never deleting
Shipped empty	risklake, newslake, worldlake, regulators, capitalmarket: lake files with no tables; nothing shipped points at them
lake: in .ready	names the lake a file goes to; the first load creates the lake file
Said, not guessed	a load into a lake no connector reads says so, naming the connectors to point at it
Never mixed	once a kind reads a real lake, the demo samples stop answering for it; Health flags a pack whose connectors read two lakes



Admin → Data loads → Inbox: a day loaded into risklake, with the notice that no connector reads it yet

Why it matters. A pilot loads the bank's own files next to the demo, and a screen never answers from a mix of the two. One guide covers lakes and loading end to end; tutorial 20 runs it on a real day.

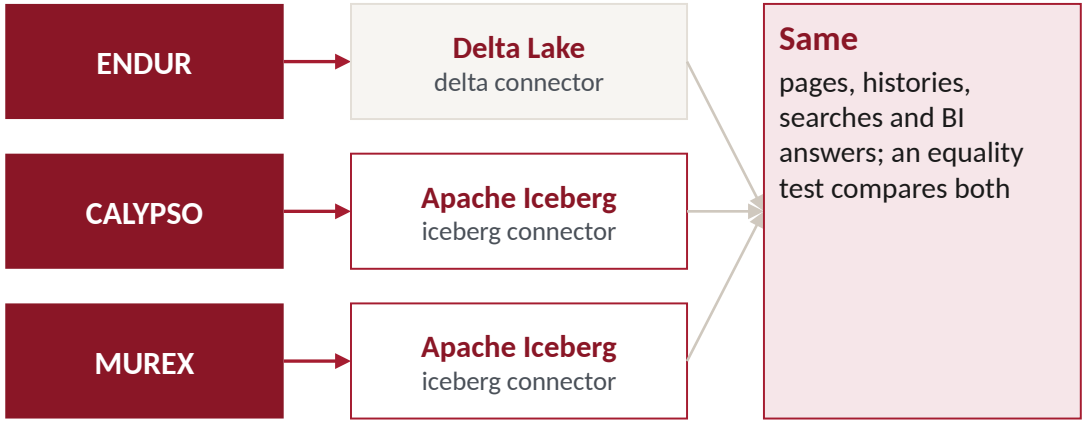
Undo in the load's own lake

Admin → Inbox → Undo a load and drishti.py dataload undo <load id> find the load's lake in the ledger and take back exactly that load.

Each system's trades in Delta or Iceberg — the same answers

One switch per source system

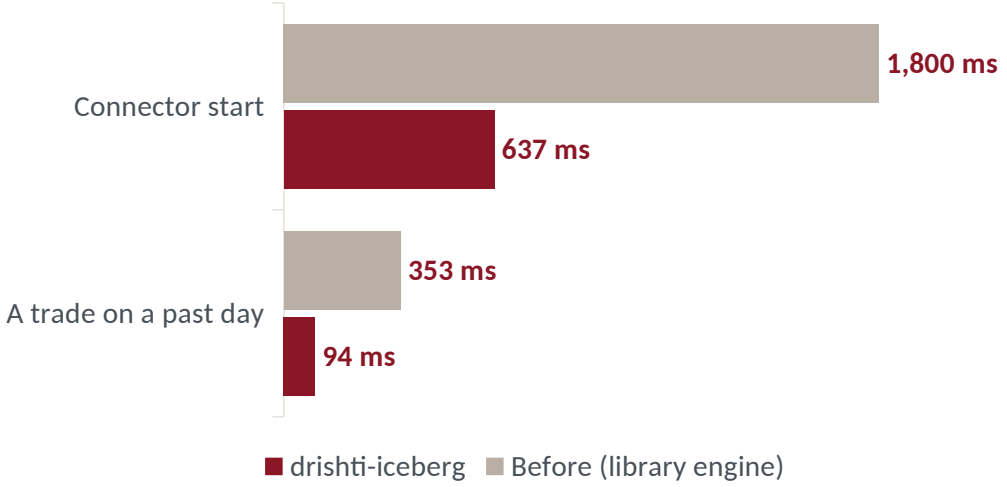
```
build_demo_lakes.py --store endur=delta \
    --store calypso=iceberg --store murex=iceberg
```



The iceberg-external connector also reads tables Drishti did not write: Spark, Flink, Trino, Snowflake, Glue.

drishti-iceberg: Iceberg without Hadoop

Drishti's own engine, the default — so Iceberg runs on Windows



2.8×[1]
faster connector start

3.8×[2]
faster read of a past day

Key insight **The store is an admin's decision per system, not a rewrite: the pages cannot tell which format answered.**

[1] 1,800 ms ÷ 637 ms = 2.8: connector start, library engine against drishti-iceberg, both measured (RELEASE_NOTES, DATA_ARCHITECTURE.md section 3) [2] 353 ms ÷ 94 ms = 3.8: a trade on a past day, both measured

Delta, Iceberg and files: the same features, one retention rule

Store	Where row retention runs	What it does
Memory (mem://)	the compaction	the rewrite leaves out the dropped rows
Delta	lake maintain, before compaction and vacuum	a date holding a dropped row is rewritten without it, one commit per date; vacuum removes the old files
Iceberg	the same run	a day holding a dropped row is replaced by one commit; snapshot expiry removes the old files
Files	the same run, for a files lake	a day file is rewritten through a temporary file moved into place

One block, the same in every store

retention: { ttl: 8h, on: UPDATED_AT, max-rows: 100000 }

- rows older than ttl go; of the rest the newest max-rows stay
- one rule, one class: a parity test loads the same rows into all four stores and compares what is left
- never a second scheduler: it runs on the store's own upkeep

Business-date retention and time travel

- retention: 7y keeps that many business dates; risk and risk results keep seven years whatever is set
- time-travel: vacuum and snapshot expiry never go below it: old versions stay readable that long
- on the same rows Delta and Iceberg give the same answers, VaR equal to the cent

AWS S3 Tables and Delta Lake on S3: one connector file and a script each

AWS S3 Tables (Iceberg REST, SigV4)

```
plugin: iceberg
kinds: [pnl-vector, scenario-pv, …]
settings:
  catalog: s3tables
  table-bucket-arn: ${DRISHTI_S3TABLES_ARN:}
  domain: risk
```

- **One setting:** the table bucket ARN gives the endpoint, region, signing name and prefix
- **AWS keeps them in shape:** S3 Tables compacts and expires; Drishti runs no maintenance there
- **Writing:** the loaders write the same tables; each date one commit

Delta Lake on S3 (native engine, no Hadoop)

```
plugin: delta
kinds: [pnl-vector, scenario-pv, …]
settings:
  root: s3://${DRISHTI_DELTA_S3_BUCKET:drishti-lake}/delta
  domain: risk
  s3.region: ${AWS_REGION:us-east-1}
```

- **Exactly as on local disk:** each table at s3://bucket/prefix/domain/table; VaR reads the same tables
- **No key in the file:** the AWS default chain (the ECS task role, a profile); least-privilege IAM listed
- **Commits:** S3's conditional writes; a check script proves a bucket honours them

Tool scripts `s3-tables.sh · delta-s3.sh  create | check | remove` Python with boto3, .sh and .ps1, no AWS CLI, --dry-run, no secret printed

Key insight Tested against recorded AWS signatures and stand-ins; the live run waits for an AWS sandbox account.

Arrow where it pays; our own formats where they are faster

Boundary	Format	Why
The lake	Parquet (Delta or Iceberg)	Arrow's partner on disk: compressed columns, statistics to skip files
VaR hot files	our own float32 rows, memory-mapped	summing runs near memory bandwidth, about 70 GB/s[1]; Arrow would not be faster
Shard partials	raw vectors of doubles	already minimal; Arrow would add framing
Server ↔ Python workers	Arrow IPC, memory-mapped	no copy, no parse
VaR contributions to the browser	**Arrow** on request; Arrow JS	fewer bytes, faster fetch: measured, right
BI queries	Arrow on request; JSON in the console	mostly text: hardly smaller after gzip, parsed slower
P&L paths, entity pages	JSON	small answers; Arrow is no smaller
Search results	HTML drawn on the server	no table reaches the browser

Where Arrow paid

VaR contributions	JSON	Arrow
by trade, 1,281 rows	631,873 B	244,304 B
by position, 1,173 rows	692,324 B	243,392 B
fetch, through the console	81–91 ms	59–61 ms[3]

61–65% fewer bytes[2], read with Arrow JS (vendored) in the VaR explorer; painted in the same time.

Kept as JSON, measured

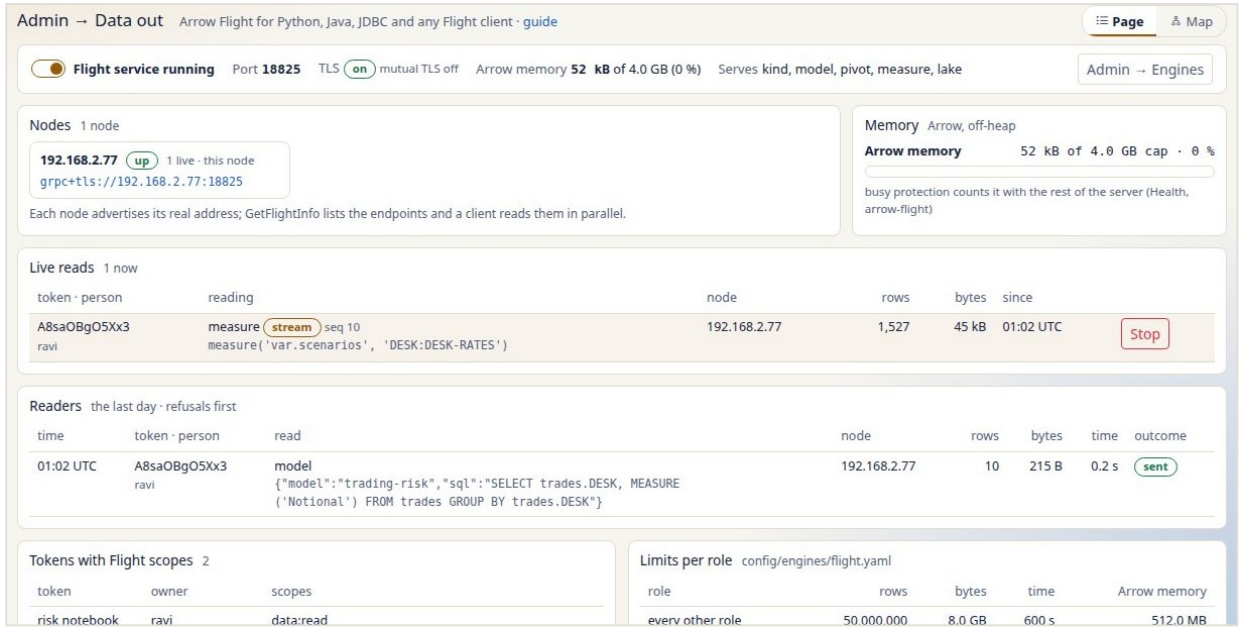
BI grids: 8–14%[4] smaller after gzip, parsed slower, painted no sooner; drawing the rows is the cost.
P&L paths: 509 numbers, no smaller.[5]

Next: a client-side renderer; search drawn in the browser with it, where Arrow can carry the rows.

Key insight Measured, Arrow paid in one place: the VaR contribution tables. Elsewhere JSON, Parquet or our own formats are as fast or faster.

[1] 1,000,000 trades × 509 × 8 B = 4.07 GB summed in 58 ms (VarScaleBenchmark, measured, VAR_GUIDE.md 27.2) = 70 GB/s [2] 1 − 244,304 ÷ 631,873 B = 61% (by trade) and 1 − 243,392 ÷ 692,324 B = 65% (by position): bytes as the console sends them; measured, DATA_GUIDE.md section 2 [3] fetch through the console, against 81–91 ms as JSON (Chromium, one machine); measured, DATA_GUIDE.md section 2 [4] gzip 122,953 → 112,656 B (5,000 trades) and 1,216,777 → 1,048,434 B (50,000 trades) = 8% and 14% smaller; measured, DATA_GUIDE.md section 2 [5] the P&L path: 21,607 B as JSON, 22,032 B as Arrow; measured, DATA_GUIDE.md section 2

Drishti's data to any client, as the person it belongs to



Admin → Data out: the service, its nodes, Arrow memory, a live VaR stream and the readers

Client	How it reads
Python	pyarrow: a token to a DataFrame, VaR scenario P&L, a live stream
Java	the Arrow Flight client: kinds, models, pivots, measures
JDBC and ODBC	Flight SQL: read-only SQL over the BI models
Excel	through Python; working examples in examples/clients

What it serves

The kinds of every pack, BI model queries, pivot views, VaR's scenario P&L vectors and the in-memory lake: Arrow record batches, no JSON step, no copy of the lake.

The same controls as the screens

Tokens with Flight scopes, entitlements and masks per reader, limits per role, Arrow memory counted by busy protection, every read audited. Off as shipped.

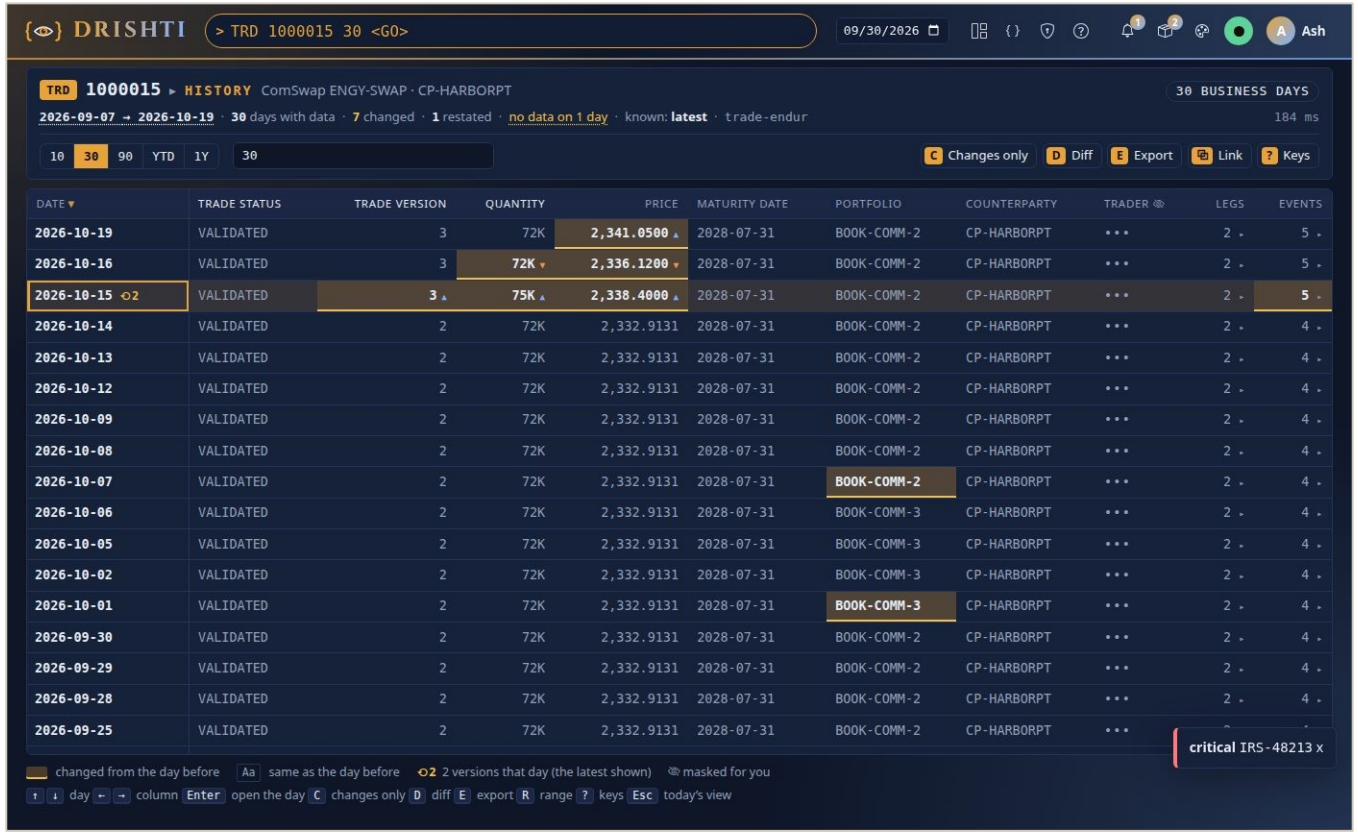
Every node an endpoint

Each node advertises its real address; a client reads the endpoints in parallel; a live stream resumes from its sequence number.

Under construction

Three nodes end to end (endpoints on every node, a node stopped mid-read, a VaR stream through an intraday batch) is not measured yet. The single-node reads, the stream, Stop and resume are run.

Type a range after an id and see the entity over time



TRD 1000015 30: one row per business day, changed cells marked ▲/▼, restated days flagged (tutorial 1).

One grammar, any range

```
TRD 1000015 30
TRD 1000015 3M DIFF
TRD 1000015 FROM 2026-09-01 TO 2026-09-30
trade 1000015 last month
```

13 ms
30 days of one trade among 10,000, cold

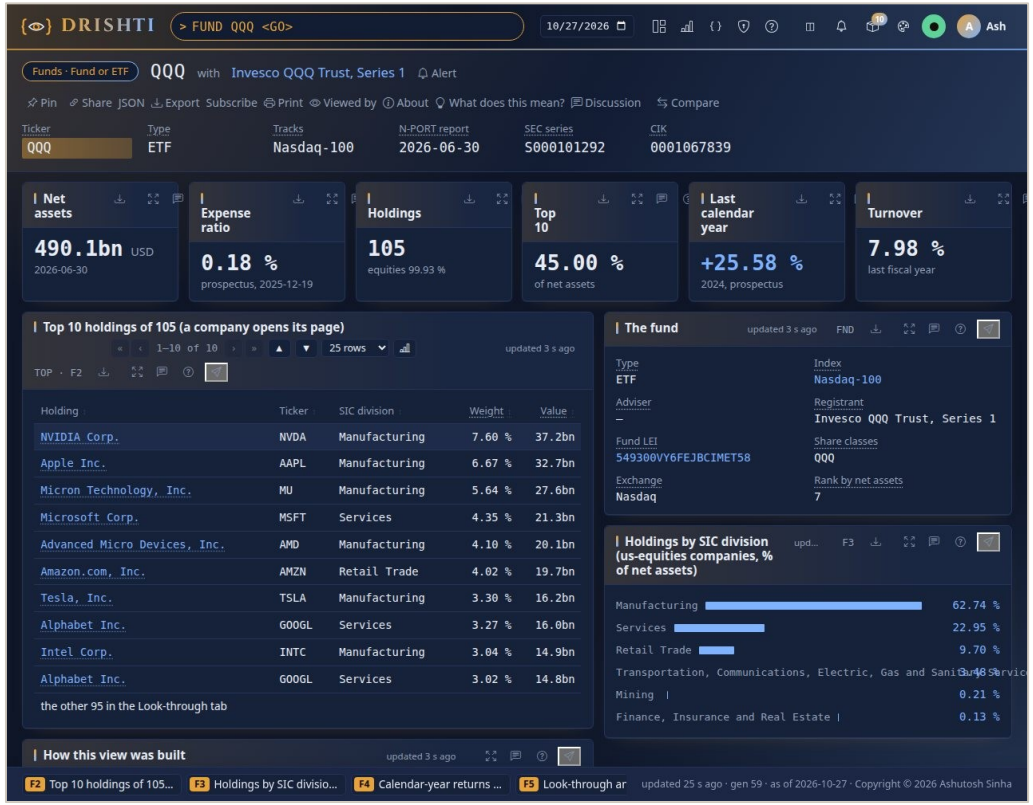
< 1 ms
the same, warm

0.3 ms
from the history cache

Generic: any kind with dated data, no code per kind; fast paths on Delta, Iceberg, Redis, PostgreSQL, DuckDB, MongoDB, Aerospike and Cassandra; masks hold on every day's row.

Key insight Across a cluster, a node reads an entity's days from the peer that holds them in one call, masked on both.

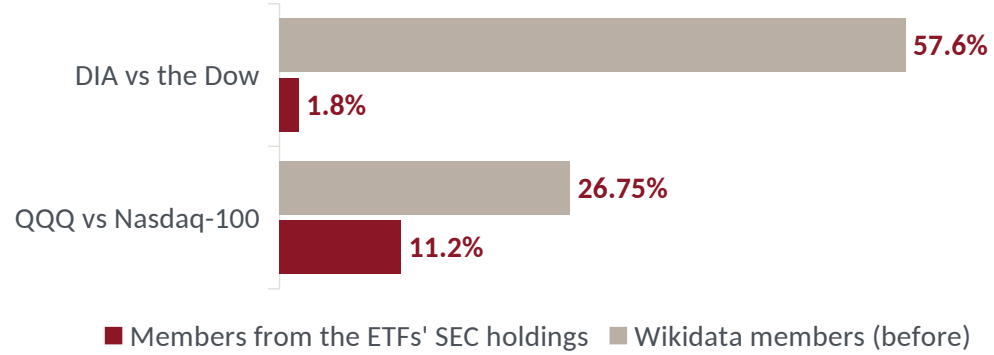
Index members from what the tracking ETFs report to the SEC



FUND QQQ: fact sheet, top holdings, look-through by SIC division (tutorial 13).

Earlier the S&P 500, Nasdaq-100 and Dow members came from Wikidata's lists, which lacked Exxon Mobil, BlackRock, NVIDIA in the Nasdaq-100 and about 100 more.

Active share against the index, before and after



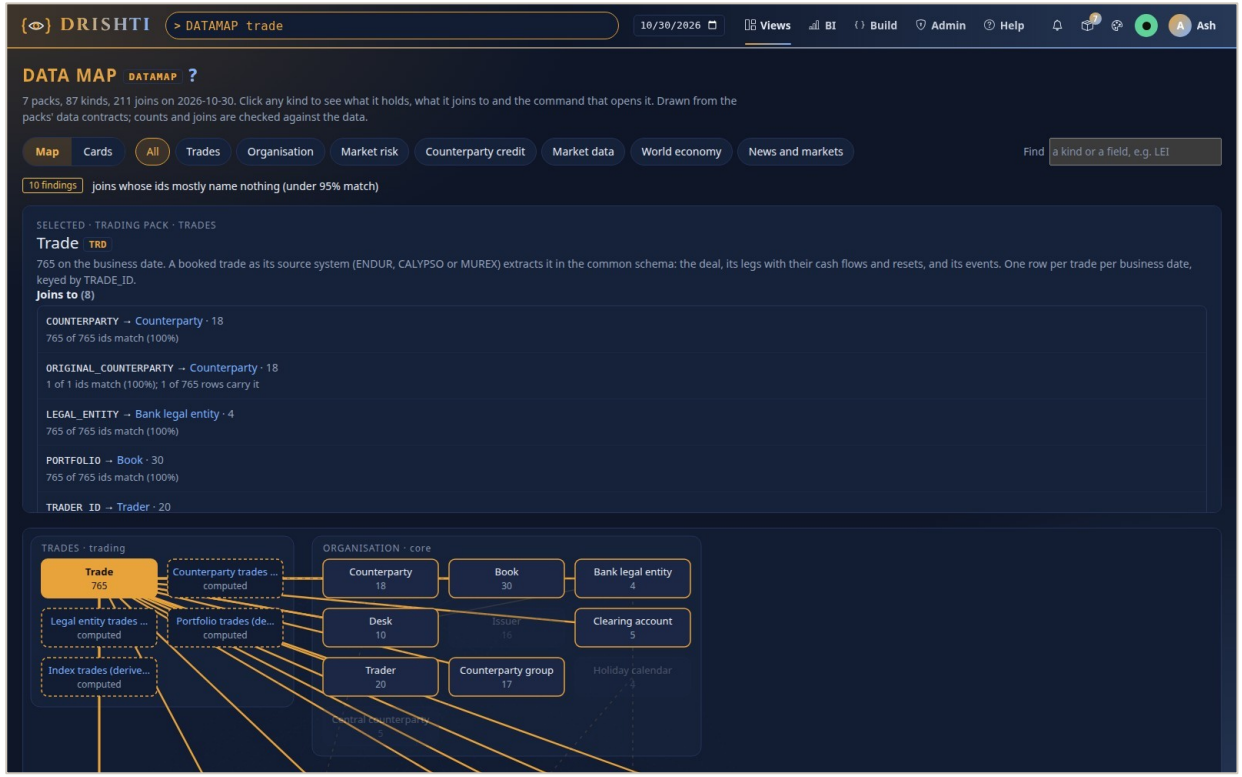
503 → 586
companies in the pack

98 of 99
listed Nasdaq-100 members held by QQQ

162
WHAT terms, 23 new for funds

Key insight The Dow is weighed by price and the Nasdaq-100 capped, as their methods do — the active shares fell to what an investor expects.

A map of all your data that draws itself — and one true surprise a day



DATAMAP trade: the selection across the top (Trade, its joins and match rates), the map under it.

- **Nothing hand-drawn**
  Kinds from the enabled packs, joins from the data contracts, counts from the lake; a new pack appears with no setup.
- **Every join checked**
  How many ids match; joins whose ids mostly name nothing are listed as findings (e.g. a jurisdiction code where an id is expected).

HOME · THE FACT OF THE DAY

One surprising, true sentence from your data, computed as you, on your business date.

Seven ship: a book over its VaR limit while its desk is within · a desk that lowers VaR · posted collateral raising exposure · VaR against peak PFE · the most-held company · two managers holding one portfolio · a fund's weights above its net assets

Key insight Each pack declares its facts; only those that hold today are shown, with 'Why does that happen?' and the command.

Feeds and public data

13

One feed file per source, pulled daily into the lakes

Piece	As built
The feed file	config/feeds/<feed>.yaml: datasets, the pack and table each loads, the lake, the schedule, the licence with its internal-only flag; API keys as environment names only
The pull	incremental and resumable; one polite User-Agent naming the product
What it writes	JSON Lines per dataset and business date under data/datafeeds/<feed>/, a manifest with rows and SHA-256, a run ledger
Three load routes	the Java loader, the Python loader, or the data inbox
Run and watch	drishti.py feed list | run | status | schedule; Admin → Data loads → Feeds; an alert when a run fails

Thirteen feeds ship

Public sector

World Bank WDI, central banks (FRED, Bank of Canada), Statistics Canada, the OECD, official press releases, GDELT world events

Reference and markets

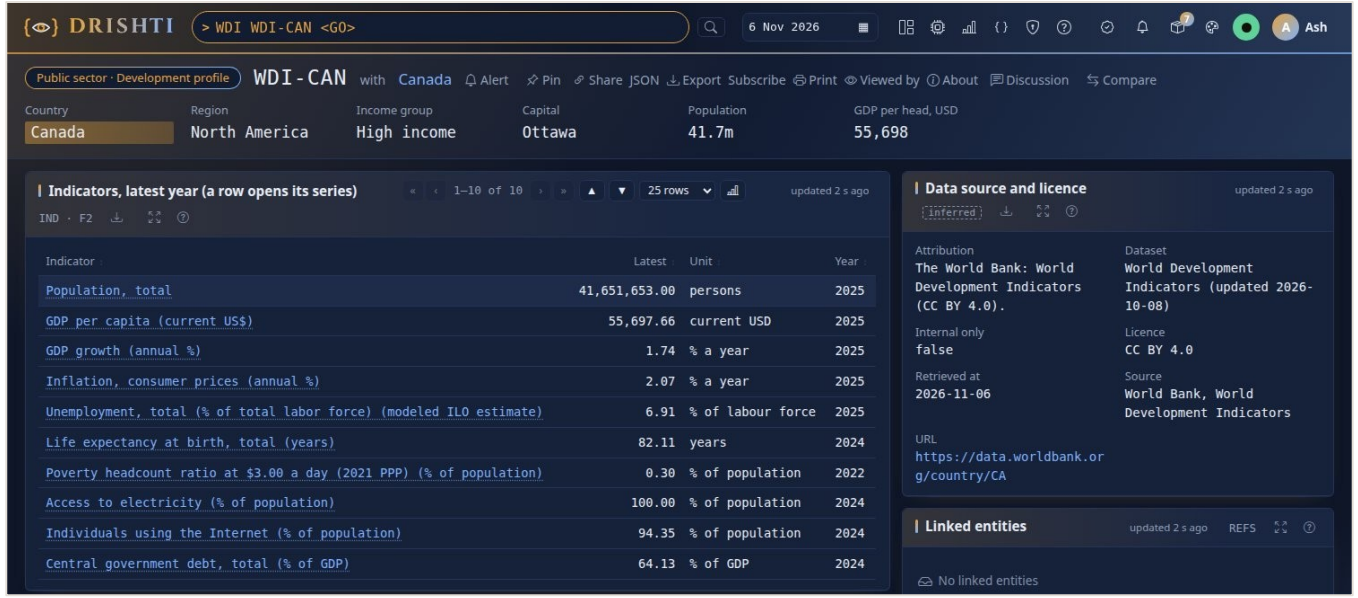
GLEIF legal entities, US equities, funds and ETFs, market data

Internal use only

the IMF forecasts, Yahoo prices (off as shipped), market headlines: each release says so

Why it matters. Outside data arrives the way a bank's own data does: by the same load rules, into named lakes, with every licence recorded and the internal-only sources marked, so nothing restricted reaches a client by accident. A daily entry for cron, systemd or Windows is one command (feed schedule).

Six packs and ten functions in the economy module, with a lens of its own



WDI WDI-CAN: a country's development indicators, with the source and its licence (tutorial 14, data of 2026-11-06)

Function	What it answers
WDI · WDIS	World Bank development indicators; the world map by GDP per head
CBANK · CBSER	central banks' series from FRED and the Bank of Canada
NSTAT	Statistics Canada
OECD · OECDS	the OECD's indicators and series
PRESS · PREL	official press releases: Fed, ECB, Bank of Canada, Bank of England, SEC, OSFI, Federal Register, Canada Gazette
GDELT	world events by country and tone

Built the way every domain will be

Packs in their domain folder, functions owned by the module, a lens, a guide and a tutorial, and data that arrives through drishti-feeds. The free news is in the World lens too.

Why it matters. The second industry proves the first was a module, not a special case.

US companies, legal entities and funds — linked, offline, refreshed by script

us-equities

586

US companies from SEC EDGAR, Nasdaq and GLEIF: financials, filings, index members

`USEQ · IDX · SIC · EDGAR`

LEI

gleif

9,512

legal entities in 540 ownership groups; groups computed from DIRECT_PARENT

`LEI · LEIG · LEIC`

LEI

funds

200

largest US mutual funds and ETFs from N-PORT, N-CEN and prospectuses; 5.6 MB gzipped

`FUND · HELD · TRACK · FUNDS`

Refresh script
`tools/publicdata/<pack>.py .sh and .ps1, status, dry-run`

→

Committed snapshot
`data/testdata/<domain>/<kind>/*.jsonl.gz`

→

Demo lakes
`build_demo_lakes → Delta`

→

A connector per pack
`served offline, like any source`

External data, one page: every source with its URL, licence and use; IMF, Yahoo and Google News marked internal only and refused in the repository; sectors are SIC divisions because GICS is proprietary — the pages say so.

Key insight **Public data arrives the way a bank's data does: a contract per kind, a lake, a connector — and no network at demo time.**

Security, drawn as flows

14



Authentication, authorization and audit are kept apart

Authentication
Who you are

- Local passwords: PBKDF2-SHA256, 240,000 iterations
- Or SSO over OIDC: Keycloak, Entra, Okta, Ping
- Lockout after 5 failures in 15 minutes
- Disable a user and their sessions end everywhere

Authorization
What you may see and do

- Roles name the kinds you may open
- Powers: raw, author, approve, calc, layout; BI: bi, bi-author, bi-steward
- Masked fields shown as ••• without raw
- Packs assigned per user

Audit
What happened

- Sign-ins, changes, token use, purges
- Access log: who viewed what, for which date
- Hash-chained collaboration, sealed
- Legal holds and an eDiscovery export

Key insight A mask is applied where a value leaves — in views, search, emails, chat bridges and snapshots alike.

Authentication — a person signs in to the console

Browser → Console → Server

```
1. Browser → console: any page but landing, help
   → 303 to /login (or to the identity provider)
2. Password: server checks PBKDF2-SHA256 hash
   or OIDC: server verifies the ID token itself
3. Server → signed session; console sets cookie
   HttpOnly, SameSite=Lax, Secure by default
4. Every write: same-origin check (else 403)
   and JSON only (else 415)
5. Every 10 s: is this user still enabled?
   disabled, deleted or reset → session ends
```

- **Lockout**
  Five failed sign-ins lock the account for fifteen minutes; an admin's reset unlocks it.
- **The last admin is protected**
  You cannot remove the last administrator, on any node of a cluster.
- **Forced change**
  The development admin is flagged in every page until its password changes.

Key insight A cross-site form cannot post: writes must come from the console and must be JSON.

Authorization — a script acts as its owner, never as more

Scope	Opens	Still required
(none)	Read: views, search, explain, read-only commands	The owner's roles and packs
design:write · bi:write	Create and save designs and BI items	The author or bi-author power
design:approve · bi:approve	Approve someone else's design or BI item	The approve power; never your own
packs:admin	Deploy, roll back and switch packs	The admin role
loads:write	Say a batch of data landed	The pack in the owner's packs

- **Hashes only**
  Secrets are stored as SHA-256; shown once.
- **Write scopes expire**
  At most 90 days.
- **Never open**
  Users, roles, tokens, sign-in, caches and collaboration state.
- **Audited**
  Every write and every refusal.

Key insight A scope only opens the endpoint — the owner's roles are still checked on every call.

Token exchange — the host proves itself, and the user

Host app → Server → Console → Browser

```
1. Host server → POST /api/v1/embed/token  (RFC 8693)
   client: secret (hash stored) or private_key_jwt
   subject: the IdP's ID token, or a host-signed
            assertion (≤ 60 s, used once)
2. Server → ES256 token, 300 s (max 900)
   audience = app, origin-bound, scopes embed:view
3. Browser → console /embed/v1/... with the token
   console checks signature (JWKS), audience, origin
4. Server checks again: app, user, origin, kind,
   rate limit — and always masks
5. Live stream: event: token → renewed in place
```

- **Off by default**
  drishti.embed.enabled; each host app registered by an admin.
- **Always masked**
  An embedded view never shows raw values, whatever the user's roles.
- **CORS, exactly**
  Only registered origins; never *, never credentials.

Key insight The token proves who and where; the server still decides what — on every call.

COMPLIANCE

Evidence, not just conversation

Chained

Every revision of a thread or share hashes the one before

Sealed

Seals cover comments, thread heads, holds and audit; verify on demand

Held

Legal holds by entity, user, thread or all (423 while held)

Exported

eDiscovery zip with a manifest; one-shot download

- **Retract and hide, never delete**

Retention keeps for ever by default; a hidden comment is still in the record.

- **Measured**

An export of 8,300 comments builds in 2.0 s and downloads (19.8 MB) in 3.1 s.

- **Only for compliance**

Legal holds and the export need the compliance power; an export downloads once.

Key insight **Tamper evidence is the honest claim: Drishti cannot stop a database administrator, but it shows when one acted.**

Evidence, by the numbers

15

Over 3,400 Java test methods, three browsers, one gate

3,400+
Java test methods (46 parameterised); the run expands them

~2,250
Console test functions, 640 of them in a browser

~640
CLI, generator and tool test functions

3
Browser engines for embedded views

- **Gates in every build**
  Error Prone and Spotless; ArchUnit for module direction; licence headers; no file over 1,500 lines; branding literals; JSONL from every generator and on every export; READMEs for every pack.
- **Pages at three widths**
  Every page at 390, 1,600 and 2,560 px; at 390 with touch: real content, 44 px targets, nothing clipped.
- **Numbers checked against an oracle**
  VaR and ES recomputed from the input files and compared with the server; intraday equals one run, to the cent; Delta equals Iceberg; a sharded cluster equals one server.

Key insight **Any failing test in any of the three suites stops the push — on Java 21 and on Java 25.**

VaR and exposure from Iceberg equal Delta, to the cent

Test	What it proves	Latest run
`VarFormatEqualityTest`	the same lines in Delta and Iceberg: VaR, ES, component and incremental VaR, worst scenarios, limits and limit letters, USD and EUR; intraday batches; official records	green
`ExposureFormatEqualityTest`	EE, PFE, EPE, profiles, contributions and official records, both formats	green
`LakeAppendsTest`	a Kafka window lands on an Iceberg risk lake as one append	green
`OfficialSinkTest` · `OfficialS3Test`	official tables in Delta and Iceberg, a folder or S3; six writers at once; a Delta commit on S3 is an exclusive create	green
`OfficialReadersTest`	BI's readers follow the official format by themselves	green

The Glue and S3 Tables runs on AWS (GlueLiveTest, check-glue.sh): written, never run; they need an AWS account.

On the demo data

Figure	Delta	Iceberg
VAR BANK, 2026-09-30 (USD)	112,041,646	112,041,646
Risk tables moved	—	14 tables, 955,124 rows, 579 s

A server on the Iceberg lake answered VAR BANK, the limit letter LL:LL-0003 and XPO BANK and published its official records.

Demo figures from Drishti's synthetic data, one workstation: an observation, not a sizing rule. Delta stays the default; Iceberg is chosen by configuration.

Key insight **One connector switched from plugin: delta to plugin: iceberg, the same rows, the same numbers.**

Trees rebuilt from columns; upgrades that keep every edit

`HierarchyEquivalenceTest`

MGMT and LEGAL rebuilt from columns

- The same node ids, parents, levels and members as the risk-hierarchy rows
- Aggregates equal to the bit; VaR, ES, components and worst scenarios to the cent, USD and EUR
- Limits equal; every limit letter the sum of its rows

Latest run: 3 tests green

`HierarchyUpgradeWithoutLettersTest`

Data written before the letters

- P&L vectors and exposure paths without LIMIT_LETTER
- Only LL is skipped, with a DRS-5211 warning and its fix
- VaR, exposure, MGMT, LEGAL and the official captures answer as before

Latest run: 1 test green

`ConnectorUpgradeTest`

A server on the previous release's files

- Every unedited file equals the current template
- The edits stay (an edited risk-store and var-whatif); missing keys added
- var-result, var-whatif with ccy and exposure-result answer

Latest run: 3 tests green

ColumnHierarchyBenchmark: 10,000 trades, 700 limit letters: the trees build in 65–99 ms against 53–197 ms without them; the VaR of all 1,412 nodes takes about 30 ms.

Key insight A new tree is configuration; an upgrade never loses an edit or a date.

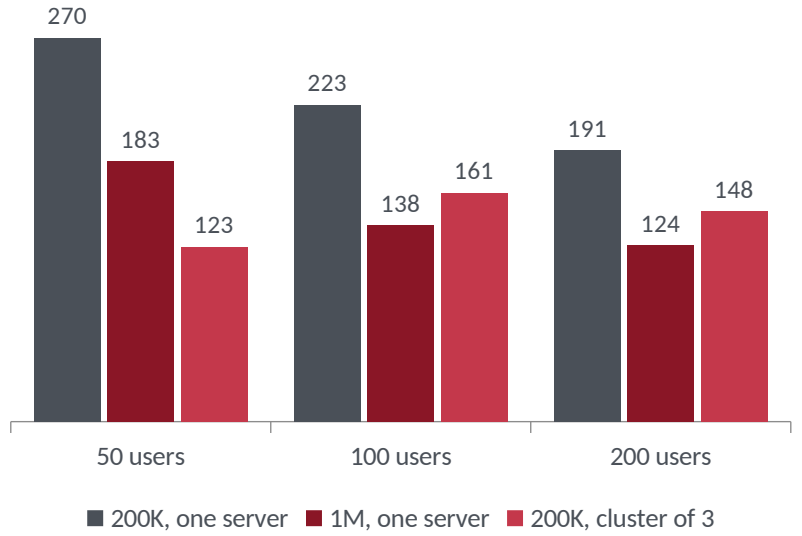
Measured: 200,000 and 1,000,000 positions side by side

One workstation: one server at each size, and a cluster of three at 200,000[1]

Measure	200K, one server	1M, one server	200K, cluster of 3
Requests a second, 50 / 100 / 200 users	270 / 223 / 191	183 / 138 / 124	123 / 161 / 148
p99 a VaR node, 50 / 200 users	0.72 / 5.39 s	1.08 / 7.33 s	0.46 / 5.08 s
p99 the bank minus a desk, 50 / 200 users	0.92 / 5.44 s	1.17 / 7.19 s	1.11 / 5.15 s
A date's first open	4.5 s	19–21 s	10.8 s
Load: five kinds, two dates, Java loader	13.9 s	28.4 s	the same lake
Heap after GC, two dates open	0.88 GB	2.2 GB	1.03 / 0.82 / 0.53 GB
Resident memory (RSS) at the end	5.0 GB	11.2 GB	4.9 / 5.6 / 5.4 GB
Disk a date: lake / hot file	0.29 / 0.56 GB	1.43 / 3.14 GB	the same lake

- **A cluster of three at 1,000,000 positions:** not yet measured[2]
- Mix: half VaR (nodes, the bank minus a desk, 50,000-position baskets, contributions, P&L paths), half trade pages; 60 s a step after a 45 s warm-up
- Single server -Xmx8g at 200K, -Xmx6g at 1M; the cluster -Xmx4g a member, VaR sharded, 2 replicas a shard; Java 21, G1

Requests a second, by users



- 200K, one server: 270 a second at 50 users, every p99 under 1 s; at 200 users the wait is queueing on 24 threads
- 1M, one server at -Xmx6g: 183 a second at 50 users with no failure at any step; every kind of question under 1.4 s at p99 with 50 users

[1] Measured on one workstation (24 threads, 61 GB, NVMe), Java 21, G1: BENCHMARK.md section 4 [2] Three servers at 1,000,000 positions: not yet run

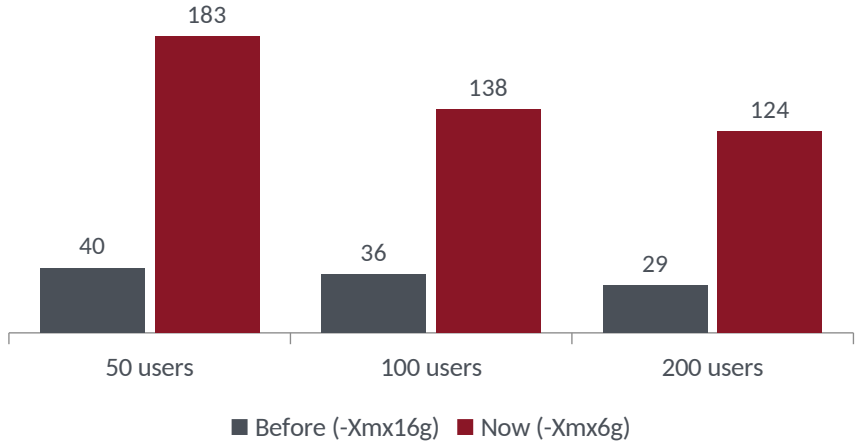
One server in 6 GB: under a third of the heap, four times the requests

The same data, machine and questions, measured twice[1]

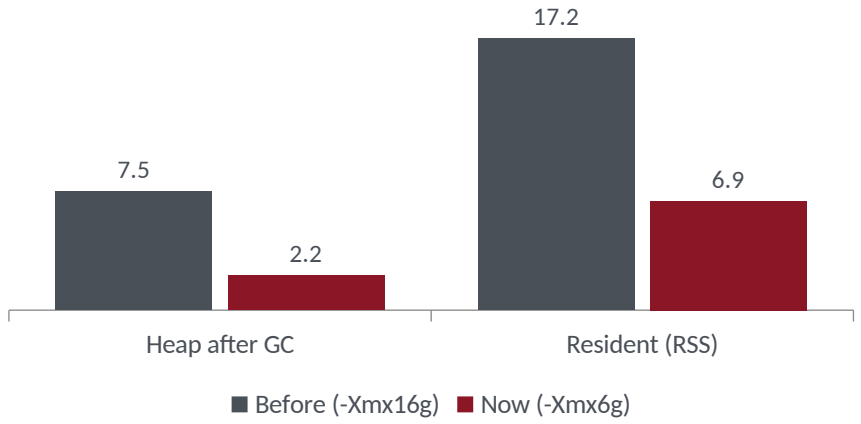
1,000,000 positions, one server	Before	Now
Heap setting (-Xmx)	16 GB	6 GB
Heap after GC, two dates open	7.5 GB	2.2 GB
Heap a date	about 3.2 GB	about 0.6–0.9 GB
Resident memory (RSS), two dates	17.2 GB	6.9 GB
A date's first open	25–27 s	19–21 s
Requests a second, 50 / 100 / 200 users	40 / 36 / 29	183 / 138 / 124
Failed requests, 50 / 100 / 200 users	0 / 23 / 77	0 / 0 / 0
The bank minus a desk, p50, 50 users	5.7 s	0.20 s
VaR answers, both dates	the reference	identical to the dollar

- **Node members as runs:** a node's members are kept as the sorted runs of its books' ordinals, not a bitmap as long as the date
- **Aggregates read from the hot file:** mapped like the position rows, not copied onto the heap
- **A scope's parts summed run by run:** the scan that took nine tenths of the CPU under load is gone

Requests a second, by users



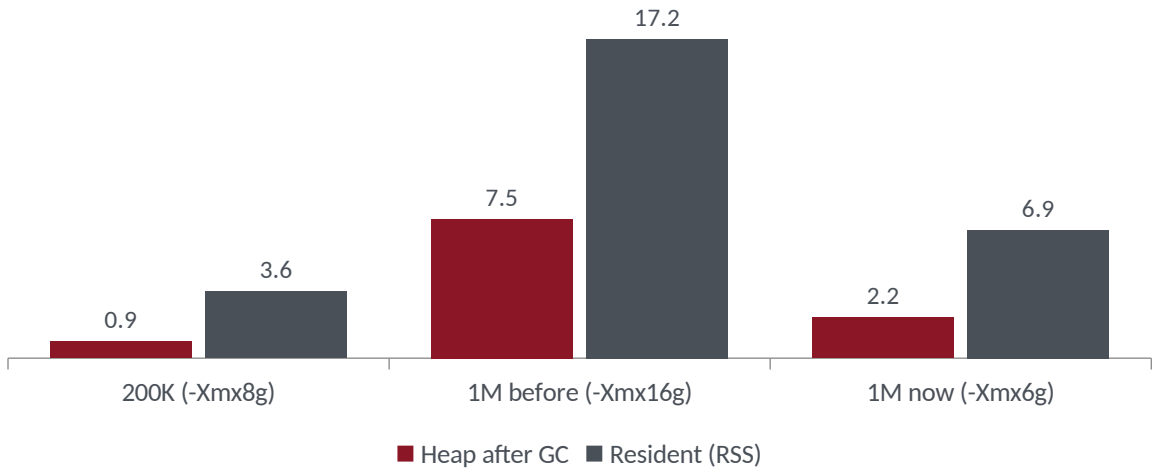
Memory with two dates open, GB



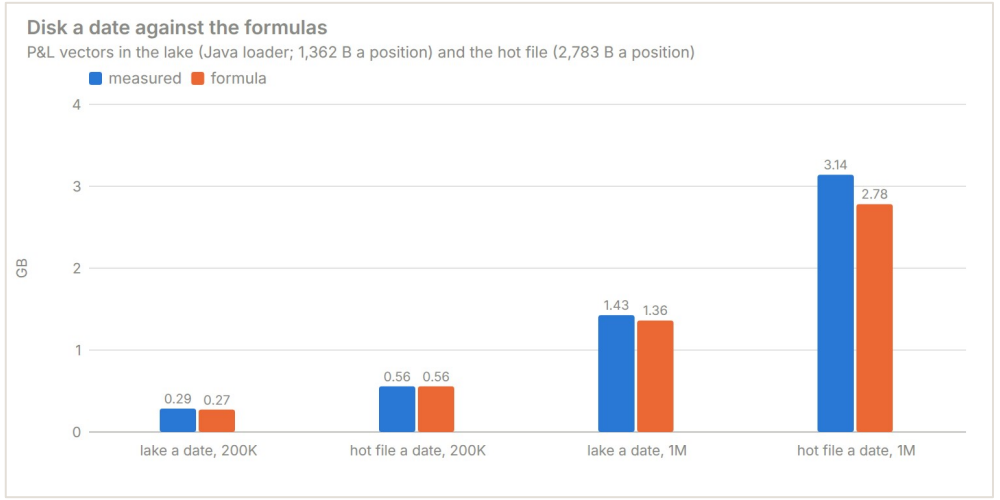
[1] Measured on one workstation (24 threads, 61 GB, NVMe), Java 21, G1: BENCHMARK.md section 4; 1,000,000 positions × 509 scenarios, two dates, 27,024 nodes

Heap and disk at both sizes, against the formula

Heap and resident memory, two dates open, GB[1]



Disk a date: the lake and the hot file



The heap a date[1]

Node members as runs, aggregates mapped from the hot file

At 1,000,000 positions and 27,024 nodes a date takes about 0.6–0.9 GB of heap, against 3.2 GB before, when each node's members were a bitmap as long as the date. The formula gives about 0.77 GB a date; two dates fit a 6 GB heap.

The capacity model's 80 MB of heap a date per 100,000 positions, 0.8 GB at a million, holds.

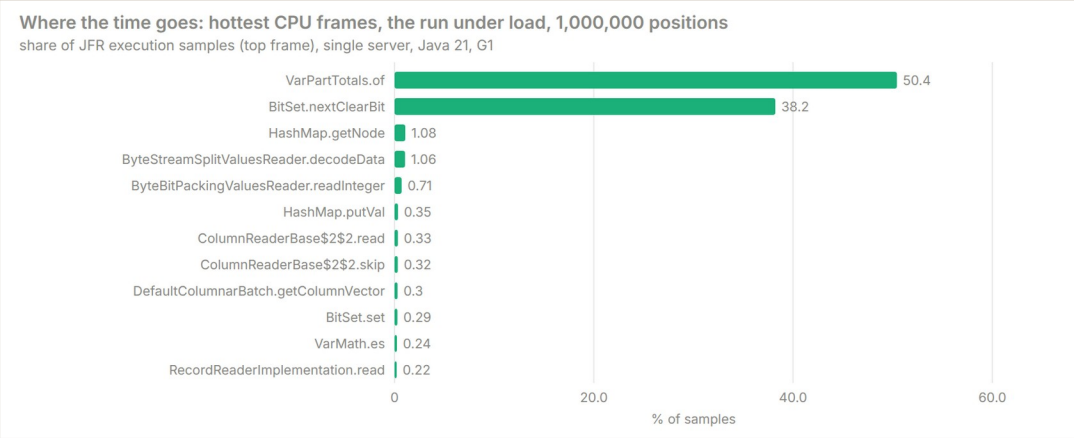
Disk a date[1]

The lake: 1,362 B a position, 5% above it at both sizes. The hot file: 2,783 B a position, met at 200,000, 13% above at 1,000,000 where 27,024 nodes add 639 MB of aggregates. Resident memory is about three times the heap: the mapped hot files count in it.

[1] Measured on one workstation (24 threads, 61 GB, NVMe), Java 21, G1: BENCHMARK.md section 4; the formula: VAR_GUIDE.md 27.1

Where the time went, and what the memory work changed

Before: hottest CPU frames under load, 1,000,000 positions[1]



- Before, at -Xmx16g: summing a scope's parts position by position, with its bit-set scan, took 89% of the CPU under load
- Now, at -Xmx6g: the bank minus a desk 0.20 s at p50 with 50 users, against 5.7 s
- Heap: about 0.6–0.9 GB a date, against 3.2 GB
- A date's first open: decoding the lake's Parquet is most of its CPU

What changed, and what comes next

1 A scope's parts summed by runs: done
the totals walk runs, not positions; the bank minus a desk no longer grows with the bank

2 The heap a date under a gigabyte: done[2]
node members as runs, aggregates mapped from the hot file: 3.2 GB a date became about 0.6–0.9 GB

3 Ten dates in memory: done
least recently used out first; month-, quarter- and year-ends pinned; a byte budget as the safety net

4 Many scopes in one pass; OpenBLAS bundled, off
each tile of rows read once for every scope; OpenBLAS gives the same VaR, turned on per node after measuring; profiling comes with the native work later

[1] Measured before the memory work: JFR execution samples, one server, Java 21, G1 (BENCHMARK.md 4.9) [2] Measured after it, one server at -Xmx6g (BENCHMARK.md section 4)

Many scopes in one pass; OpenBLAS bundled, and off as shipped

1,000,000 positions × 509 scenarios, 100 scopes summed at once[1]

Shape of the 100 scopes	One at a time	One pass, Java	One pass, OpenBLAS
Cube cells (each row in one)	291.1 ms	124.7 ms · 2.3×	121.2 ms · 2.4×
Overlapping portfolios	3,209.8 ms	2,645.4 ms · 1.2×	1,028.4 ms · 3.1×
Desks (contiguous ranges)	110.3 ms	33.0 ms · 3.3×	34.5 ms · 3.2×

At 200,000 positions[1]: cube cells 2.0× on both paths; overlapping portfolios 1.9× on Java and 2.4× on OpenBLAS; desks 2.2× on Java. At 10 or 1,000 scopes the gain is small (0.9× to 1.2×).

The same bits

The Java path reads each tile of rows once for every scope and gives the same bits as one scope at a time; the benchmark checks every vector. OpenBLAS agrees within a relative 1e-12, with the same VaR scenario date.

Read it with care

One workstation, with a release check running beside it: the ratios are the result, not the milliseconds. The dgemm thresholds were tuned on this machine; measure again on the bank's hardware.

OpenBLAS: bundled, off as shipped

Linux and Windows x86-64, nothing to install. A 250,000-position run gave the same VaR to the dollar on and off, with no clear end-to-end gain at request level[2].

On per node, after measuring

Admin → Engines turns it on live, after a self-test; a failure falls back to Java with a warning, never a stop. In a cluster, one member on Java takes the many-scope sums to Java.

Kept honest

Every release's VaR check runs on Java (bit-exact against the previous figures) and with OpenBLAS (within 1e-12, the same VaR dates).

Next

Profiling comes with the native work of a later release; the request path is where the next gain is looked for.

[1] Measured 2026-11-03, AMD Ryzen AI 9 HX 370 (24 threads, 61 GB), ManyScopeBenchmark, best of the repeats (BENCHMARK.md 3, 17) [2] CHANGELOG: OpenBLAS off as shipped

Years on disk, ten dates in memory, and the dates people return to pinned

Open in memory
the mapped hot file and the date's structures; ten dates by default, least recently used out; a byte budget as the safety net

A hot file on local disk
the built file of a date not open; opens again in seconds, without reading the lake

The lake (Delta or Iceberg)
every date, for the years kept; opening builds the hot file once

The official record
each date's figures of every node; history, backtests, statistics and reports read it

At 1,000,000 positions a date

	Before	Now[1]
Heap a date open	3.2 GB	0.6–0.9 GB
Server heap (-Xmx)	16 GB	6 GB
Hot file a date	3.14 GB	3.14 GB
Dates in memory	2 newest + 1	10, LRU

Pinned, from the calendar

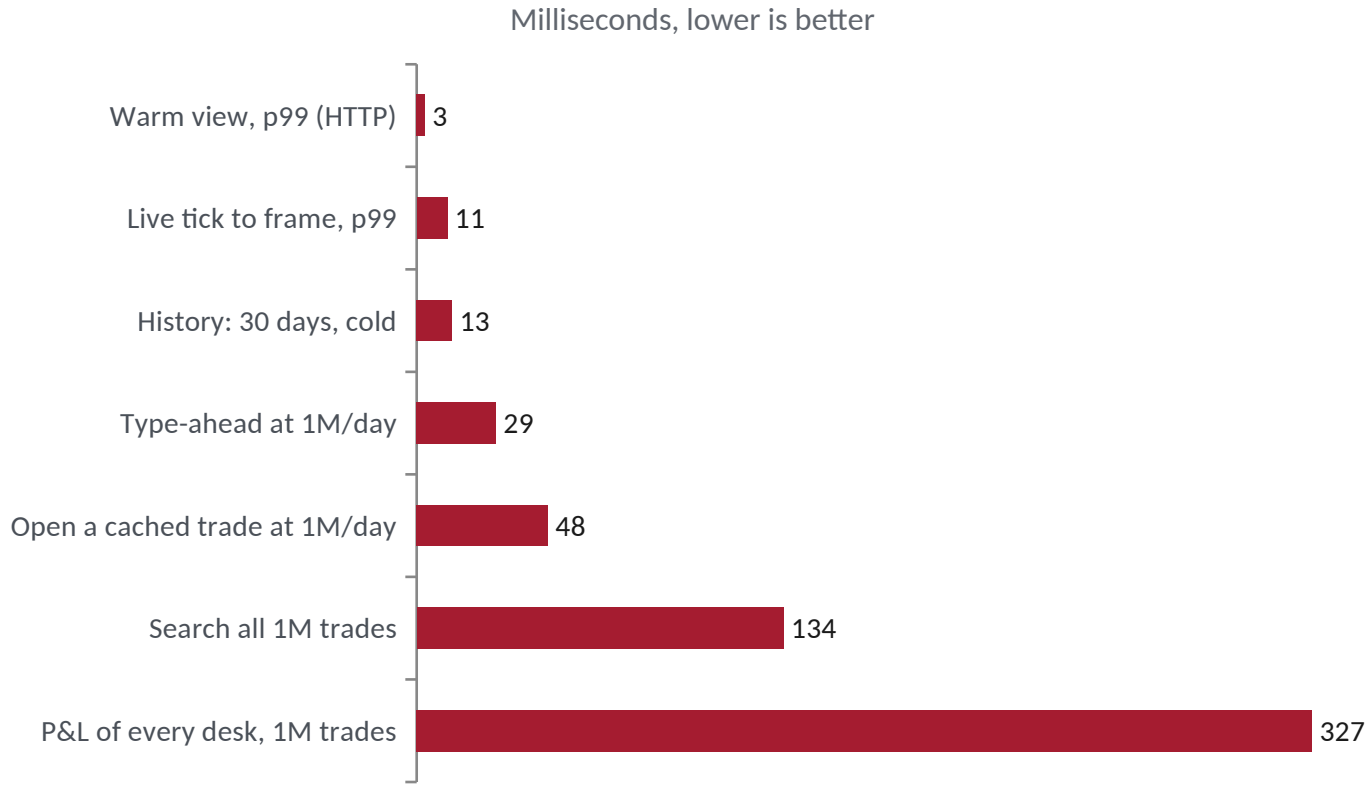
- month-, quarter-, year- and fiscal-year-ends (USNY+CATO)
- the stress window's first and last days
- any date the bank names: a crisis day, an audit date
- kept on disk by default, or held in memory

Key insight **Memory grows with the dates open at once, a setting, never with the years kept.**

[1] Measured: one server, Java 21, G1, two dates open, 1,000,000 positions (BENCHMARK.md section 4)

PERFORMANCE · MEASURED

Fast where people wait



Gate	Target	Measured
Warm view p99	< 50 ms	3.0 ms
Live p99	< 40 ms	11.2 ms
History, 30 days	—	13 ms cold, < 1 ms warm

- **One workstation**
  24 cores; 1,000,000 trades a day for three days, 5.2 GB.
- **Cold vs cached**
  Opening a trade cold is 137–157 ms; from the cache 48 ms.

Key insight **The gates are written down first and measured after — and every one has headroom.**

SCALE · 35.7 MILLION TRADES

Tens of millions of trades, fifty users, no failures

35.7M
Trades: ENDUR, CALYPSO, MUREX, 30 days, loaded while serving

179/s
Requests a second, 50 users against the server, 0 failures

0.21 s
Form search p50, the form kept per data version

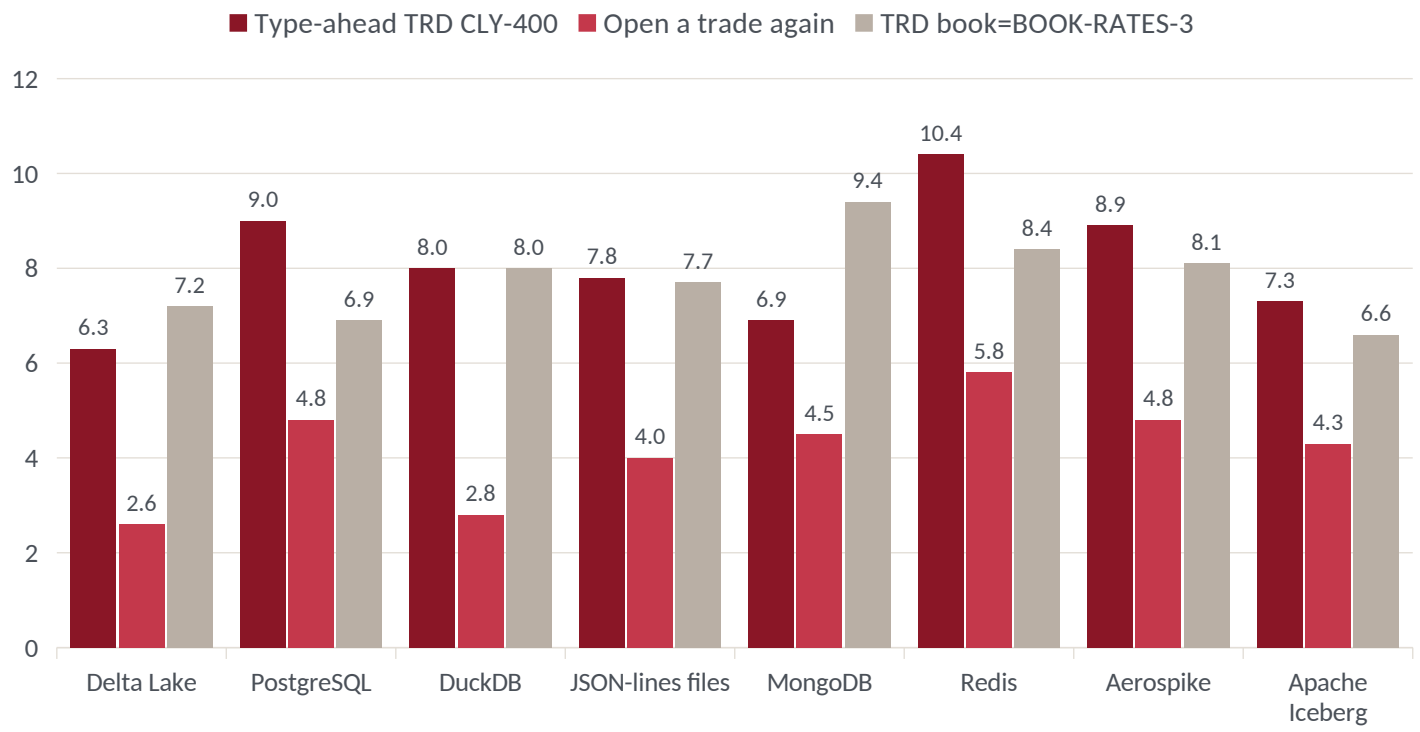
182/s
Console pages a second, 50 users, 0 failures

Data	Measure	Users	Result
35.7M trades	Single trade / grouped page, p50 / p95	50	0.16 / 0.73 s, 0.17 / 0.73 s
35.7M trades	Server heap while serving	50	3.7 GB of 8 GB
400,000 restated	A day reloaded, 10 % amended, while serving	30	2 min 54 s, 202 requests/s, 0 failures
8.2M trades	Single trade / grouped page from disk, p50	50	18 ms / 70 ms

Key insight One workstation, an 8 GB heap: the figures and their method are in CLUSTER_GUIDE.md section 16 and CLUSTER_GUIDE.md section 14.

The same book in eight stores, the same questions over HTTP

Median milliseconds at 50,000 trades a day (lower is better)



Store	Size	Load
Delta Lake	270 MB	10.2 s
PostgreSQL	739 MB	22.3 s
DuckDB	122 MB	22.3 s
JSON-lines	1,322 MB	15.4 s
MongoDB	202 MB	17.1 s
Redis	183 MB	20.2 s
Aerospike	1,176 MB	18.8 s
Iceberg	133 MB	26.4 s

Measured on one workstation, 2026-10-01. Four stores were also measured at a million trades a day.

Key insight Every store answers an interactive question in single-digit milliseconds at this size; the choice is about operations.

VaR and exposure answers in milliseconds, at 100,000 trades in memory

VaR (VarScaleBenchmark, VarTierBenchmark)

Measured	Time	On
A node's VaR and ES	0.48 ms	tiered store, 100k trades
The bank from float32 rows	3.4 ms	tiered store, 100k trades
The bank from float64 rows	58 ms	heap tier, 1M trades
A basket of 50,000 positions	2.6 ms + ≈ 45 ms ids	tiered store, 100k
First load of a date	0.8–1.2 s	tiered store, 100k
Loading the demo date	572 ms	9,572 trades, the server

Exposure (ExposureTierBenchmark)

Measured, 100k trades, 9,901 sets	Time
Load a date: rows into value matrices	785 ms
Collateral, E, N, hot file written	1,250 ms
Reopen the date from its hot file	157 ms
Scope BANK (every set)	61 ms
Scope CPTY: (20 sets) · NSET: (1 set)	0.22 · 0.06 ms
Pre-deal exact, Java · proxy	0.11 · 0.007 ms

Why it is fast

Vectors add: a node's vector is summed once and kept; a hot file per date is memory-mapped (float32 rows, float64 sums); sums run on every core in a fixed order — the same bits every time.

What is extrapolated, and said so

1M trades × 50 dates (VaR) and a 1M-trade, 100,000-set bank (exposure) are extrapolated from these runs[1]; a workstation run stays at 100,000 trades in memory.

Key insight Measured on one workstation (24 threads, 61 GB, JDK 25, NVMe); the full million-trade runs are for a real server.

[1] linear in trades (× 10 from 100,000 to 1M) and in dates from the measured runs above (VAR_GUIDE.md 27.3, EXPOSURE_GUIDE.md 27.4); not measured, to be confirmed on a server

Nothing reaches main without the full gate

Licences
A header on every file

→

Java 21
Full verify; bytecode pinned to 21

→

Java 25
Verify in a copy of the commit

→

Console
Tests on parallel workers

→

Tools
CLI tests

→

Push
develop, then main

~1 hour
A full drill: Java 21 verify, then Java 25, console and tools side by side

45
Tagged releases, 30 September to 29 October 2026

2,471
Commits, every push through the drill

Key insight **A failed test stops the push; a flaky one is retried once and written down — never ignored.**

The release drill: from about 91 to about 77 minutes, same proof

Lane	Before	After (measured alone)	What did it
Browser	about 88 min	74 min	shared read-only servers under a write guard, snapshots split, drawn signals instead of sleeps
Containers	about 95 min	10 min	images built and booted once; health, the VaR golden through the API, four console pages
Java 21	about 43 min	29 min	Docker hidden from the copy's tests, the copy built once
Gate	3.3 min	3.3 min	unchanged

Same proof, less waiting

- the VaR goldens are the proof of the numbers; a 12-case matrix runs in about 3 s
- a fix drill reruns only the lanes the change touches
- a page walk times every page against a 10 s budget

Projected full drill

- about 77 minutes on a loaded machine, about 57 on a quiet one, against about 91 before
- measured on a shared machine; confirmed by each release's full drill

Operating and deploying

16



From a clone to a live view in about fifteen minutes

Seven commands

```
./mvnw -q package -DskipTests -pl drishti-server -am
uv venv --python 3.13 drishti-console/.venv
uv pip install --python drishti-console/.venv/bin/python \
    -r drishti-console/requirements.txt
python tools/drishti.py lake demo
java -Djava.io.tmpdir=data/tmp/server -Xmx4g \
    -jar drishti-server/target/drishti-server-<v>-exec.jar
python drishti-console/run_drishti_web.py
# open :17480/t → TRD MX-20000001 <GO>
#                 VAR BANK <GO>   EXP BANK <GO>
```

GETTING_STARTED

About 15 minutes: build, the demo lakes, a view

IDE_GUIDE

Server in IntelliJ, console in PyCharm

CLI_GUIDE

pack make, history, var, exposure, convert

USER_GUIDE

Every page, every key; 29 tutorials beside it

Key insight Start with your own JSON Lines: one pack make, one deploy, and your records have a terminal.

Fewer documents, one home for every fact

172
documents under docs/, from 360

22
tutorials, one per major feature

32
guides and references

4
Knowledge pages: a glossary and three primers

Track	Tutorials, one per major feature
Start and analyse	01 the first 15 minutes · 07 pivot, cubes and Explain · 08 reports and dashboards · 09 the Notebook · 10 the AI assistant · 11 correct and approve
Risk	02 VaR · 03 exposure · 04 FRTB-SA · 05 SIMM · 06 groupings · 12 a day of bank data
World and public data	13 world data · 14 the public sector
Data and build	15 connect · 16 a lake of your own · 17 data out · 18 tables and models · 19 a pack and a screen · 20 embed
Operate	21 production · 22 a cluster of three

Every step three ways

Each tutorial step shows the screen, the command line and the YAML, on the shipped demo data.

Help by domain and lens

A Help page belongs to a module and may name its lenses: a module that is off for a person is not listed, searched or opened. Every old address still opens the page that took its subject over.

Link, don't copy

A field lives in its data contract, a record in its JSON example; guides and tutorials link both. Every contract has complete JSON examples, checked against its fields.

Checked in the gate

A register of features with their guide, tutorial, domain and lenses; counts capped; no copied record and no version history in a guide.

From a folder of JSON Lines to a deployed pack

One command

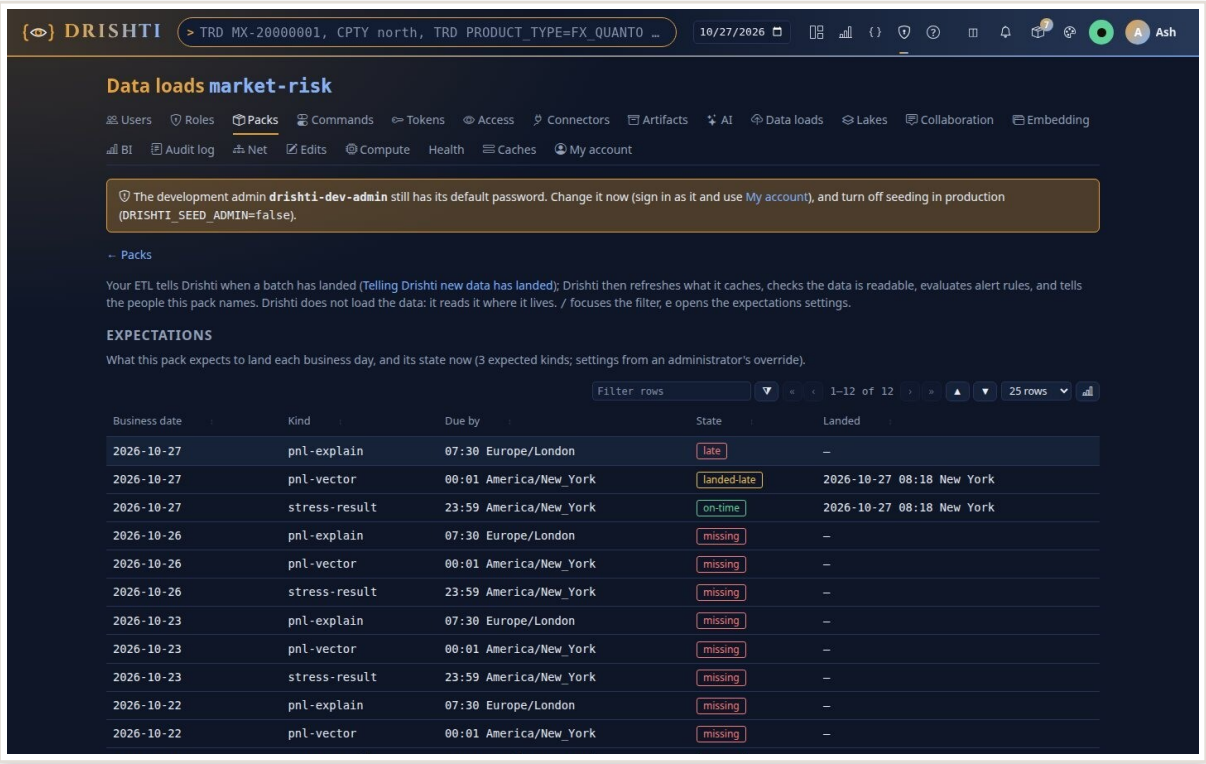
```
drishti.py pack make ./trades --pack desk \
    --kind trade --key TRADE_ID \
    --match PRODUCT_TYPE --date TRADE_DATE
# → desk/ pack, sutras per product type,
#   Delta lake by date, desk.tar.gz + sha256,
#   README.txt, run scripts
```



- **Checked before it lands**
  Checksums, paths, server version, Sutra tests and an optional Ed25519 signature.
- **Never stuck**
  The previous version is kept; a failed deploy rolls back; history has Roll back.

Key insight **Archives carry the pack, never the data — where data lives stays an admin's decision.**

Tell Drishti new data has landed — and it acts on it




```
drishti.py data landed --pack trading \
    --kind trade --date 2026-10-05 --rows 10000
# or POST /api/v1/packs/trading/loads
```

- **Refresh, verify, alert, notify**
  Caches refreshed, the date checked, alert rules run on it, bell, inbox and email.
- **Late is a fact, said once**
  Per-kind expectations with time zone and calendar; Health shows 'data late'.
- **Safe to repeat**
  Idempotent per pack, kind, date and batch; token scope loads:write.

Every build step from the command line

Group	What it does	Examples
sutra · pack	Lint, test, preview Sutras; make, sign, deploy packs	sutra test, pack make, deploy, rollback
data · lake · convert	Ingest JSONL, build lakes, turn CSV into JSONL	landed, lake demo --names, convert
server · connector · net	Packs and commands on or off, connectors, members	packs off, commands off, net members
bi · design · view	Models, SQL, runs, statements; designs; a view	bi sql --format jsonl, bi runs
history · contract	An entity over days; every command's contract	history TRD 1000015 30, contract check
var · exposure · compute	The risk engines as you see them; reactive kinds	var show … --ccy EUR, exposure profile
change · edits · doctor	Write-back changes, Admin → Edits, a setup check	change submit, edits status, doctor

Key insight Anything the console can build, a script can build — and a pipeline can check it.

Ready means ready; drain before stop; upgrade one node at a time

Each node, in turn: the rolling upgrade runner does exactly this



Order: consoles → BI and data-serving → coordinators → shard owners, one zone at a time. Any failure stops the run and names the node; resume with --from.

Readiness, for routing; liveness, for restarts

Readiness	HTTP	When
`UP`	200	joined, newest date open, owned shards ready
`DOWN`	503	joining, a date opening, or a shard not ready
`OUT_OF_SERVICE`	503	drained before a stop

- Liveness (the JVM answers) drives container restarts: a node waiting for a date is never restarted for waiting
- GET /api/v1/admin/node says what readiness waits for, and whether the node is drained and safe to stop
- Each gate can be switched off: drishti.node.readiness.joining, .engines, .shards

The runner and the stop hook

- tools/rolling-upgrade.sh --inventory my-cluster.yaml --version <v> --dry-run prints every step; without --dry-run it runs them (Windows: rolling-upgrade.ps1)
- The inventory (deploy/rolling.example.yaml): platform vm or ecs, the waits (drain 600 s, ready 1,800 s), zones, the nodes' real addresses
- The stop hook drains a node on every stop (systemd, Docker, ECS); Admin → Cluster has Drain for a stop
- The drain needs an administrator's token with the node:drain scope

Every server measures itself and raises its own alerts

Platform metrics, every 15 seconds

What	Prometheus names (examples)
Caches, the memory budget	`drishti_cache_bytes`, `drishti_memory_budget_*`
Disk folders and free space	`drishti_disk_folder_bytes`, `drishti_disk_free_bytes`
Sources and feeds	`drishti_source_up`, `drishti_feeds_attention`
Loads	`drishti_loads_last_ready_age_seconds`
Pages and alerts	`drishti_view_seconds`, `drishti_alerts_open`

- Exported on /actuator/prometheus with the JVM's and the HTTP server's own
- The CIO lens reads the same names
- Admin → Health shows them, with the alerts open now

With or without Prometheus: each server evaluates the rules itself; deploy/prometheus/drishti-alerts.yaml is the same rules in PromQL for a site's own Alertmanager.

The alert rules as shipped (config/alerts/platform-alerts.yaml)

Rule	Opens when	Severity
`cache-near-limit`	above 95% of its limit, an hour	warn
`heap-after-gc-high`	heap after GC above 85%, 10 min	warn
`direct-memory-high`	direct memory above 80%	critical
`disk-free-under-guard`	free disk under the guard	critical
`source-down`	a source down for 5 min	critical
`feeds-need-attention`	a load late, missing or failed	warn
`views-slow`	pages above 150 ms at p99, 5 min	warn
`member-suspect`	a member silent for 2 min	warn

- When one opens: logged, counted, and every administrator told in the inbox; listed on Health until it closes
- Rules are configuration: a metric, a threshold, for how long, a severity and a summary line; read again when changed

Tidy by design: scopes, one settings document, archives, API references

Admin → Ad-hoc scopes

every person's saved VaR baskets in one list

- Owners delete their own; an administrator may delete anyone's, with an optional reason
- Every delete audited (var-basket-delete: who, whose, the reason); the owner told in the inbox
- Also from the command line and the API (drishti.py var basket-delete)

One configuration document

docs/admin/CONFIGURATION.md, and Help → Configuration

- Every setting of every kind: key, type, default, environment variable, meaning, restart or live
- Generated from the files themselves and checked on every build: never out of date
- Administrators see the value in effect beside each default, and where it came from; secrets never shown

Loaders read archives

zip, tar.gz, tgz and tar, read in place, local or s3://

- JSON Lines members, plain or compressed, read in name order; other files skipped with a note
- No folders: one dataset named after the file; one level of folders: a dataset per folder
- One rule in the Python and the Java loaders, tested on the same archives

API references in the console

Help → Developer

- The REST API (OpenAPI), the renderer's TypeScript API
- Java and Python references generated from the code: plugin and connector SPI, shard machinery, compute SDK, drishti.py and its client
- The build fails on an undocumented public member

Each engine on or off per server; off means gone everywhere

Engine	Command or place	As shipped	Needs
VaR	VAR, SMAT, PLAY VAR, regulatory VaR	on	—
Exposure	EXP	on	—
Sensitivities	the sensitivities API, risk factors	on	—
FRTB-SA	FRSA, PLAY FRTB	on	VaR, sensitivities
SIMM	SIMM, PLAY SIMM	on	sensitivities
Pivot	PIVOT, Views → Pivot	on	—
MR cube	MCUBE	on	VaR, pivot
Data inbox	Admin → Data loads → Inbox	off	—
In-memory lake	memory-only tables	off	—
Arrow Flight	Admin → Data out	off	—
Linear algebra: OpenBLAS	many-scope sums	off	—

One file per engine

config/engines/<engine>.yaml, switched from Admin → Engines or the command line; written, audited and kept in History.

Off is gone everywhere

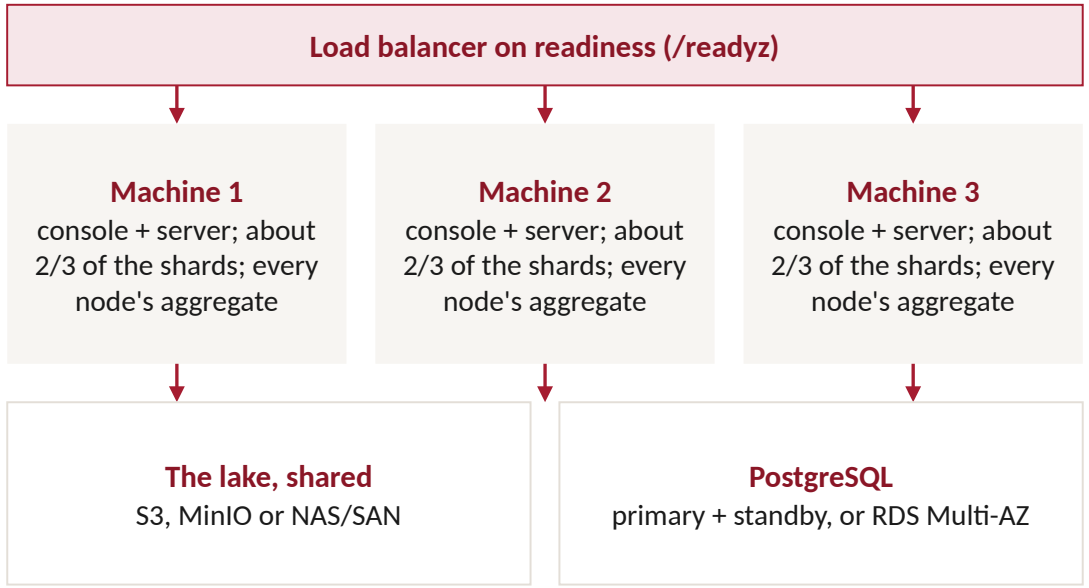
Commands, menus, Help, pages, playgrounds and the API; dependencies checked first; in a cluster a request goes to a node that runs the engine.

Busy protection

When memory is short, new heavy work (a new date, a download, a BI run, a bulk load) waits politely and retries; what is open keeps working. An operator may override, with a reason.

Three machines, on premises or on AWS

The smallest highly available Drishti: three machines, the rest shared



- One fails: every shard still has a replica; the others rebuild the second copy from the lake
- Upgrades one at a time: drain, stop, upgrade, ready; two copies throughout
- No consensus protocol: a split leaves read answers identical; writes go to the shared database
- Two machines: pilots and UAT only (no spare)

A machine (the heap measured at 1M, the rest extrapolated)[1]

	On premises	AWS
Machines	3 × 16 cores, 64 GB, 500 GB NVMe	3 × m7i.4xlarge, 16 vCPU, 64 GB; gp3 500 GB
Lake	S3-compatible (MinIO) or NAS/SAN	S3
Database	PostgreSQL + standby	RDS PostgreSQL Multi-AZ
Balancer	a pair + VIP, or F5	ALB across three zones

- 64 GB carries 1-day VaR to 2M (heap 4.3 GB, -Xmx 12g; 6g at 1M, measured); every horizon: 128 GB to about 1M, 192 GB at 2M
- The lake is small: about 1–3 TB for several years at 1–2M positions (1-day)

AWS, 1-day VaR to 2M, a month (Canada (Central))[2]

On demand: USD 3,300–4,000[3]

1-year Savings Plan: USD 2,800–3,500[4]

3-year Savings Plan: USD 2,300–3,000

Every horizon to 1M (3 × m7i.8xlarge): USD 5,400–6,100 on demand. Check every price in the AWS Pricing Calculator.

[1–4] How these figures were derived: A-1, in the appendix (Three machines)

Where it is going

17



ROADMAP

What is built, and what comes next

Built · unlabelled	Not built yet
Any input: PVs or P&L, trade or position files and Kafka; intraday as new – old; mixed per system	**Trust, RBAC and controls** security on by default; data and module licensing; one breach queue
1-day, 10-day and stressed VaR; parametric beside it √10 or the sum; SVAR; SMAT; why VaR moved; the VaR Playground	**Memory and speed** the full drill under an hour; measured everywhere, then fixed
Any currency; official records, 7 years every hierarchy; Delta or Iceberg, folder or S3	**Counterparty engines and stress** SA-CCR in pre-deal; the CCR cube; a stress engine
Totals over every position; sharded cluster 1,000,000 positions in a 6 GB heap; many scopes at once; engines per server	**Collateral and margin** valuation and eligibility; margin calls; wrong-way flags
Regulatory VaR; FRTB-SA, SIMM; the MR cube backtests, sign-off, the stress window; each measure with its playground	**CVA, then hardening** BA-CVA from SA-CCR; FVA; adversarial and soak tests
The pre-deal check; one limit rule approve, refer or decline from the exposure paths; within, near, breach everywhere	**Lakehouse interoperability** Unity Catalog, Delta Sharing, plain Delta and Iceberg tables
BI on one table layer; Pivot → Report the editors; the Notebook; one Python runtime on the server	**Onboarding: role doors** four role doors; the remaining guides and tutorials
Drishti Elements; one way per job every screen on eleven libraries; one search, Approvals, Explain, Build home	**Arrow Flight on three nodes, end to end** single-node reads and streams are run
Domain modules, lenses, the access registry banking, capital markets, economy; ten lenses; functions one by one	**Glue and S3 Tables on AWS** code written; first live run on an account
A lake of your own; feeds; the docs rewritten thirteen feeds, the public sector; one guide and one tutorial per feature	

At scale: lake clustering by book and portfolio (Z-order or liquid clustering); a published table of node and portfolio vectors, built in an overnight batch.

Not scheduled: QuantLib; the PostgreSQL wire protocol; FRTB internal models. New domains come as modules, as the public sector did.

Lean & streamlined: trust and speed, then new engines

1 Next: Trust, RBAC, security and controls

- Security on by default, one entitlement service for every door, visibility by role; data licensing and module licensing on the clean modules
- Run status and sign-off, one breach and escalation queue, computed credit-limit usage, number honesty; the pilot kit

2 Memory and speed

- the full drill under an hour
- measured everywhere, then fixed where it matters

3 Counterparty engines and stress

- SA-CCR with incremental EAD in pre-deal
- the CCR cube; a stress engine

4 Collateral and margin

- valuation and eligibility
- margin calls and disputes; wrong-way flags

5 CVA

- BA-CVA from SA-CCR exposure
- FVA

6 Hardening

- adversarial tests at every door
- soak tests at production volumes

7 Onboarding

- four role doors
- the remaining guides and tutorials

8 More domains, as modules

- each with its packs, lens and feeds
- anti-money laundering next in line

Not scheduled: QuantLib; the PostgreSQL wire protocol; FRTB internal models, SA-CVA, full xVA. New domains come as modules, as the public sector did.

Appendix: reference

A

248 error codes, every one documented

```
$ curl -s server/api/v1/views/trade/NOPE-404
Content-Type: application/problem+json
{ "status": 404,
  "code":   "DRS-1001",
  "title":  "entity not found",
  "detail": "no source holds the entity" }
```

Family	Area	Codes
DRS-1xxx	Entities and sources	14
DRS-2/3xxx	Grammar, Sutras, inference	11
DRS-4xxx	Commands and search	19
DRS-5xxx	Requests, Drishti*net, VaR, exposure	51
DRS-6xxx	Users, roles, tokens	11
DRS-7xxx	Collaboration	16
DRS-8xxx	Embedding	6
DRS-9xxx	Business intelligence, runs	120

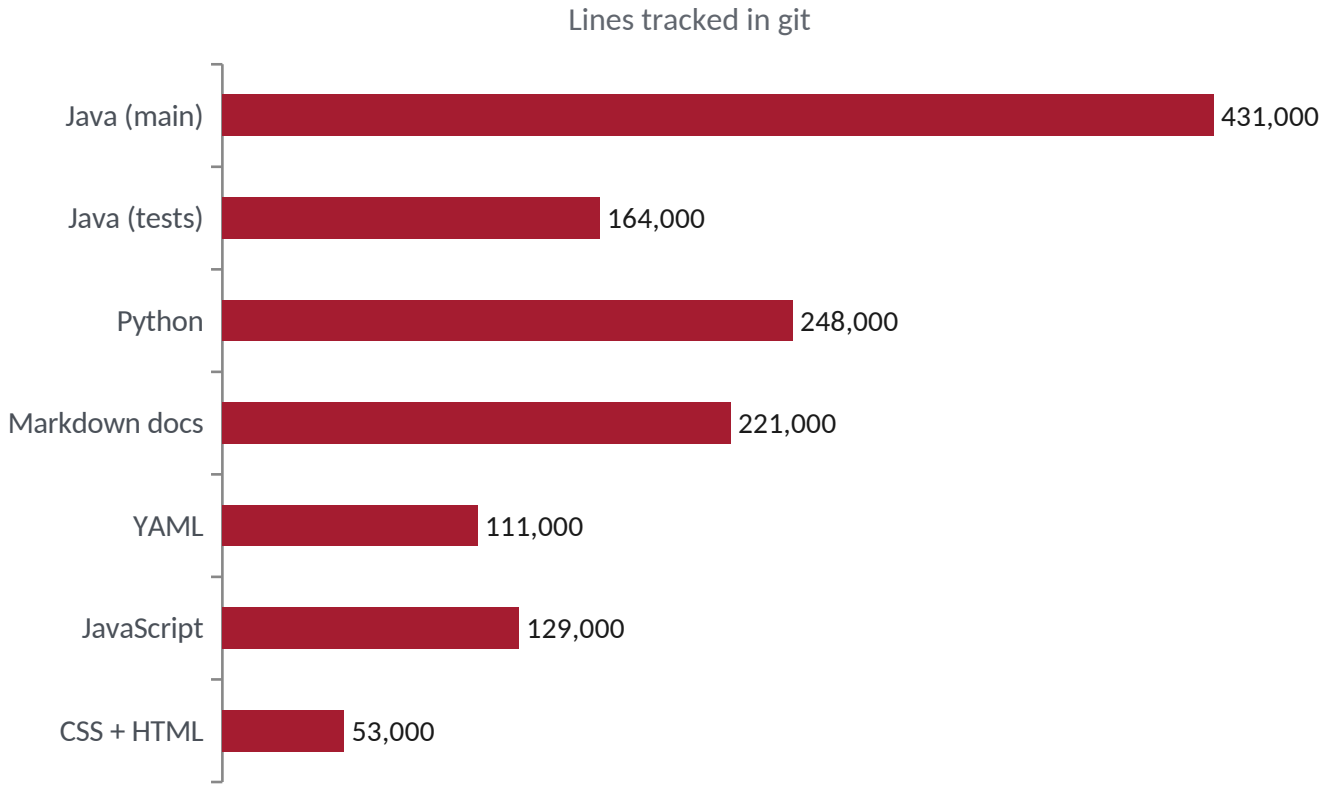
248
Codes in nine families, one table

47
Sutra problem codes in the grammar reference

RFC 7807
problem+json, the same shape everywhere

Key insight **Errors are a contract: a stable code, an HTTP status and a row that says what to do.**

Where the work is



- **172 documents, one guide each**
  32 guides and references, 22 tutorials (one per major feature), 4 Knowledge pages, 22 ADRs; 51 contract pages, every one with JSON examples.
- **Every screen documented**
  Over 900 pictures under docs/, taken by scripts on the current UI over the demo lakes.
- **52 modules, 19 plugins**
  Four domain modules over a core that knows no industry; eleven Elements libraries.

How the numbers were derived

How it scales: the algorithms

[1] A scope's set: one bit a trade, N ÷ 8 bytes: 200,000 ÷ 8 = 25,000 B ≈ 25 KB; 2.3M ÷ 8 = 287,500 B ≈ 290 KB (VAR_GUIDE §27a.1). The standalone cache: 8 B a position and lens: 200,000 × 8 = 1.6 MB; 2.3M × 8 = 18.4 MB ≈ 18 MB (§27a.3).

[2] Is a position whole: a scan of its own bits, (to − from) ÷ 64 machine words (a position of up to 64 trades: one word); a scope: one pass over its positions. A standalone VaR keeps the k worst of S values, O(S × k): k = ⌈509 × 0.01⌉ = 6 at 99% (§27a.1, §27a.3).

[3] Measured, one user, p50 (BENCHMARK.md §4.5): a node 3 ms at 200,000 and 4 ms at 1,000,000 positions, its aggregate whatever its size; the bank minus a desk 68 ms and 1.46 s, linear in the scope's positions while its parts are summed position by position: × 2.3 = 3.4 s at 2.3M (EXTRAPOLATED, §12.1), the walk that summing by runs removes.

Three machines

[1] Per member of three, replicas 2 (each holds 2/3 of the shards), two dates open; the heap scaled from one server measured at 1M (-Xmx6g, 2.2 GB with two dates): H = 128 MB + 641 B × P + 2 × 722 B × P = 2.2 GB at 1M, 4.3 GB at 2M; -Xmx 2.7 × H (the measured ratio) = 11.6 → 12g; RAM = H + 0.13 GB + two open dates of hot files (2,783 B × P × 2/3) + a load + 7 GB = 28 GB ÷ 0.61 → 48 GB; every horizon (the capacity model × P ÷ 2.3M): 55 GB at 1M, 97 GB at 2M → 192 GB.

[2] AWS list prices, Canada (Central), on demand, 730 h (ASSUMPTION: verify before quoting). 1-day VaR to 2M: 3 × m7i.4xlarge (a quarter of m7i.16xlarge) = $1,968, gp3 3 × 500 GB = $231, about 3,300–4,000 with RDS. Every horizon to 1M: m7i.8xlarge = m7i.16xlarge $3.5952 ÷ 2 × 3 × 730 = $3,937; gp3 3 × 1 TB, 6,000 IOPS, 500 MiB/s = $363; S3 2 TB $60; ALB, zones, loads, NAT and endpoints, CloudWatch, secrets as the cost lines = $476.

[3] RDS PostgreSQL Multi-AZ $600–1,300 (ASSUMPTION, not in the price constants: db.m7g.xlarge to 2xlarge, us-east-1 list + ~10%): total $5,436–6,136, about 5,400–6,100.

[4] Compute Savings Plans, no upfront, on EC2 and Fargate only: 1 year −26.9% / −20.0% = −$1,065; 3 years −49.5% / −45.0% = −$1,964.

Related documents

Document	What it holds
README.md · docs/tutorials/README.md	what Drishti is; 22 tutorials, one per major feature
docs/guides/USER_GUIDE.md · CLI_REFERENCE.md	every page and key; the command line
docs/guides/OPERATIONS_GUIDE.md · CLUSTER_GUIDE.md	install, run and operate; several servers, AWS, sizing and tuning
docs/guides/ADMIN_GUIDE.md · docs/architecture/ACCESS_MODEL.md	people, roles, lenses and modules; the access registry
docs/guides/DATA_GUIDE.md · DATA_LAKE_GUIDE.md	JSON Lines, connectors and the demo data; lakes and loading
docs/guides/VAR_GUIDE.md · EXPOSURE_GUIDE.md	risk computed on demand: VaR, exposure and the pre-deal check
docs/guides/REPORTS_GUIDE.md · PIVOT_GUIDE.md	reports and dashboards; the Pivot, the cubes and Explain
docs/admin/BENCHMARK.md · docs/architecture/ARCHITECTURE.md	the measured numbers; the design underneath
docs/architecture/IMPLEMENTATION_PLAN.md · RELEASE_NOTES.md	what comes next; the release history (this deck is version-free)
docs/publications/Drishti-VaR-Engine.pptx	the VaR engine in depth
docs/publications/Drishti-on-AWS.pptx	Drishti on AWS: the lake, Delta or Iceberg



Drishti दृष्टि

Thank you

See what your data hasn't told you yet.

ANY DATA. ANY DOMAIN. ONE GRAMMAR.

Ashutosh Sinha • November 2026